



République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université Mouloud Mammeri de Tizi-Ouzou
Faculté de Génie Électrique et d'Informatique
Département d'Informatique

MÉMOIRE

En vue de l'obtention du diplôme
de Master 2 en Informatique

Spécialité : Systèmes Informatiques Intelligents

Présenté par : **Idir REBHI** et **Siham LARBI**

Détection de stéganographie issue de modèles génératifs : distinction entre images stéganographiées et images synthétiques

Mémoire encadré par : **Samy SADI**

Co-encadré par : **Tassadit HEDIR**

Soutenu publiquement devant le jury composé de :

Mehammed DAOUI	Pr. Univ. de Tizi-Ouzou	Président
Idir FILALI	Pr. Univ. de Tizi-Ouzou	Examineur
Samy SADI	MCB. Univ. de Tizi-Ouzou	Encadrant
Tassadit HEDIR	Doct Univ. de Tizi-Ouzou	Co-Encadrante

REMERCIEMENTS

Au terme de ce travail, nous souhaitons adresser nos sincères remerciements à toutes les personnes qui nous ont accompagnés, soutenus et encouragés tout au long de cette aventure.

Nous tenons tout d'abord à nous remercier mutuellement. Ce mémoire a représenté un défi important, marqué par de nombreuses difficultés, des périodes de doute et un travail exigeant. Cette collaboration, fondée sur l'écoute et les efforts partagés, a joué un rôle essentiel dans l'aboutissement de ce projet.

Nous adressons également nos plus sincères remerciements à nos parents et à nos proches, pour leur soutien constant, leur patience et leurs encouragements tout au long de nos études.

Nous remercions l'ensemble du corps enseignant et administratif de l'université pour la qualité de la formation qui nous a permis de grandir aussi bien sur le plan académique que personnel.

Nous exprimons une gratitude particulière à M.Sadi et à Mlle.Hedir pour leur accompagnement. Nous les remercions pour leur disponibilité, leur patience, leurs précieux conseils ainsi que pour la rigueur et l'exigence qu'ils ont constamment encouragés dans notre démarche. Leurs remarques et leurs orientations ont contribué de manière déterminante à la qualité de ce mémoire.

Enfin, nous leur sommes reconnaissants de nous avoir fait découvrir l'univers fascinant de la stéganalyse. Ils nous ont permis d'explorer un domaine passionnant de la recherche et de développer un intérêt profond pour les problématiques scientifiques qu'il soulève.

À toutes ces personnes, nous adressons notre profonde reconnaissance et nos plus sincères remerciements.

*En vous souhaitant une agréable lecture,
Idir REBHI et Siham LARBI.*

Résumé

Le développement de l'intelligence artificielle générative, et particulièrement des modèles de diffusion, a fait émerger la stéganographie générative basée sur la diffusion (DM-GIS). Cette méthode intègre un message secret directement lors de la création de l'image, rendant les outils de détection classiques inefficaces. Plusieurs approches de stéganalyse ont récemment été proposées pour faire face à cette menace, parmi lesquelles NS-D Ser [41] constitue l'une des références les plus avancées. Cette approche repose sur l'extraction du bruit de l'image afin d'en analyser les propriétés statistiques globales. Cependant, elle est mise en échec par des techniques stéganographiques et de tatouage numérique avancées telles que le Gaussian Shading [36], qui parviennent à camoufler l'information cachée en préservant la distribution naturelle du bruit.

L'objectif de ce mémoire est d'explorer de nouvelles pistes pour améliorer les performances de détection. Dans un premier temps, nous réimplémentons et évaluons la méthode de référence NS-D Ser afin d'établir une base de comparaison. Ensuite, nous étendons cette approche en y ajoutant de nouvelles couches algorithmiques. Nous explorons ainsi différentes méthodes de classification, l'utilisation de caractéristiques statistiques supplémentaires, ainsi qu'une approche purement neuronale basée sur le Deep Learning. Enfin, ce travail propose une étude comparative de ces différentes approches afin d'analyser leurs performances et d'identifier la stratégie la plus efficace.

Mots clés : Stéganalyse, Modèles de diffusion, Stéganographie générative, Bruit latent, Deep Learning, Stable Diffusion.

Abstract

The advancement of generative AI, particularly diffusion models, has led to the rise of Diffusion Model-based Generative Image Steganography (DM-GIS). This method embeds secret messages directly during the image generation process, rendering conventional detection tools ineffective. Several steganalysis approaches have recently been proposed to address this threat, among which NS-D Ser [41] stands as one of the most advanced reference frameworks. It relies on extracting image noise to analyze its global statistical properties. However, this approach fails against advanced steganographic and watermarking techniques such as Gaussian Shading [36], which successfully camouflages hidden information by preserving the natural distribution of the noise.

The objective of this work is to explore new avenues for improving detection performance. First, we re-implement and evaluate the original NS-D Ser framework to establish a benchmark. Then, we extend this baseline by introducing new algorithmic layers. We investigate alternative classification methods, incorporate additional statistical features, and explore a purely deep learning-based approach. Finally, we provide a comparative study of these methods to analyze their performance and identify the most effective strategy.

Keywords: Steganalysis, Diffusion Models, Generative Image Steganography, Latent Noise, Deep Learning, Stable Diffusion.

TABLE DES MATIÈRES

REMERCIEMENTS	II
RÉSUMÉS	III
TABLE DES MATIÈRES	IV
INTRODUCTION GÉNÉRALE	1
1 Contexte	1
2 Problématique : Aveuglement des outils classiques	1
3 Contributions	2
4 Organisation du document	2
I État de l’art	3
1 FONDEMENTS THÉORIQUES	4
1.1 Introduction	4
1.2 Les Réseaux de Neurones Artificiels	4
1.2.1 Structure d’un neurone artificiel	5
1.2.2 Entraînement d’un réseau de neurones	11
1.2.3 Boucle d’apprentissage d’un réseau de neurones	15
1.2.4 Évaluation des réseaux de neurones	15
1.2.5 Quelques Architectures de Réseaux de Neurones	16
1.3 Génération d’images synthétiques	19
1.3.1 Quelques rappels statistiques	19
1.3.2 Les Modèles de diffusion	21

1.4	Conclusion	28
2	ÉTUDE DES MÉTHODES DE RÉFÉRENCE	29
2.1	Introduction	29
2.2	La stéganalyse	29
2.2.1	Définition de la stéganalyse	29
2.2.2	Définition de la stéganographie	30
2.2.3	Les caractéristiques fondamentales de la stéganographie	30
2.2.4	Enjeux de la stéganalyse et de la stéganographie	31
2.2.5	Terminologie	31
2.3	Approches existantes	33
2.3.1	Deux voies pour la stéganalyse	33
2.3.2	Quelques techniques de stéganographie	34
2.3.3	Taxonomie de la stéganalyse	35
2.3.4	SPAM et SRM : Rich Models en Stéganalyse (méthodes classiques)	37
2.3.5	Le changement de paradigme (Paradigm Shift)	37
2.3.6	Historique de la stéganalyse basé sur les CNN	38
2.3.7	Architectures des modèles QianNet, XuNet et YeNet	40
2.3.8	La stéganalyse basée sur les GAN	41
2.4	NS-Dser	41
2.4.1	NS-Dser : Vers une Stéganalyse dans l'Espace de Bruit	41
2.4.2	L'espace des distributions des images	42
2.4.3	L'équivalence des divergences : le fondement de NS-Dser	42
2.4.4	Le pipeline de l'architecture NS-Dser	43
2.4.5	Le vecteur de statistiques globales et le FLD	44
2.4.6	Mécanismes d'insertion des méthodes DM-GIS	46
2.4.7	Le compromis entre sécurité et précision d'extraction	46
2.4.8	Évaluation expérimentale : Scénarios, résultats et limites de NS-Dser	47
2.5	Conclusion	49
II	Contributions	50
3	APPROCHE PROPOSÉE	51
3.1	Introduction	51
3.2	La baseline	51
3.3	Défis de l'approche expérimentale	53
3.3.1	L'émergence d'un domaine nouveau et l'absence de datasets de référence	53
3.3.2	La double surcharge computationnelle : génération et inversion	53
3.4	La génération des données et constitution du dataset	54
3.4.1	Choix des méthodes de stéganographie	55

3.5	Première Extension - Extraction des caractéristiques internes de U-Net . . .	55
3.5.1	Architecture globale de cette approche	55
3.6	Deuxième extension - Explorations de différents classifieurs et statistiques .	57
3.6.1	Ajout de nouvelles caractéristiques sur le bruit latent	58
3.6.2	Déploiement de classifieurs avancés	58
3.6.3	Apprentissage de la normalité : Approches PCRM et RPCM	59
3.7	Conclusion	60
4	IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX	61
4.1	Introduction	61
4.2	Environnements de Développement, Matériel et Outils	61
4.2.1	Google Colab (Génération et Approche Neuronale)	62
4.2.2	Kaggle (Extraction NS-Dser et Deuxième Extension)	63
4.2.3	Outils et Bibliothèques Logiciels	63
4.3	Pipeline Central : Fonctions d'Extraction et d'Inversion	64
4.3.1	Chargement de l'architecture Stable Diffusion 2.1	64
4.3.2	Logique d'Inversion et Extraction	65
4.3.3	Le choix du scheduler Heun	65
4.4	Génération et Organisation Structurée des Données	66
4.4.1	Implémentation des DataLoaders (Approche Neuronale)	67
4.5	Stratégie d'Entraînement et Hyperparamètres	67
4.5.1	Stratégie d'Entraînement de la Première Extension (CNN)	67
4.5.2	Stratégie d'Entraînement de la Deuxième Extension (Machine Learning & One-Class)	68
4.6	Résultats Expérimentaux et Évaluation	68
4.6.1	Évaluation de la Baseline NS-Dser	68
4.6.2	Évaluation de la Première Extension : Approche Neuronale (CNN) .	70
4.6.3	Évaluation de la Deuxième Extension : Modèles Avancés et Nouvelles Caractéristiques	71
	CONCLUSION GÉNÉRALE	73
1	Synthèse	73
2	Perspectives	74
3	Disponibilité du code et reproductibilité	74
	Annexes	76
A	ARCHITECTURE DÉTAILLÉE DE L'APPROCHE	77
B	RÉSULTATS BRUTS DE CLASSIFICATION SUR LA BASE MAS-GRDH	79

B.1	Tableau comparatif des performances	79
B.2	Matrices de corrélation et analyses graphiques	79
BIBLIOGRAPHIE		82
LISTES		86
	Liste des figures	86
	Liste des tableaux	88

INTRODUCTION GÉNÉRALE

1 Contexte

Le développement rapide de l'Intelligence Artificielle (IA) générative et notamment les modèles de diffusion a transformé la manière de générer les images et toute sorte de contenu visuel, mais a également fait émerger de nouvelles méthodes de dissimulation d'information. La stéganographie générative basée sur la diffusion (DM-GIS pour Diffusion Model based Generative Image Steganography) permet désormais de générer des images contenant intrinsèquement un message secret. La stéganographie d'images générative se définit comme une approche capable de dissimuler des messages secrets au sein d'images synthétisées à la volée, s'affranchissant ainsi totalement de l'utilisation de covers préexistantes. [41] Ceci signifie entre autres que, là où il fallait auparavant disposer d'une image support pour y dissimuler une information, il n'est désormais plus nécessaire d'en posséder une au préalable : il devient possible de générer directement une image contenant l'information encastree à l'intérieur. Cela change considérablement la donne à plusieurs niveaux, notamment en matière de discrétion, d'automatisation et de détection. En effet, l'information cachée ne se contente plus d'être ajoutée après coup sur une image existante, mais participe indirectement au processus même de génération.

2 Problématique : Aveuglement des outils classiques

Le problème fondamental posé par ces méthodes est l'inefficacité totale des stéganalyses classiques. Ces derniers sont conçus pour détecter des traces de modification dans les pixels ou les domaines fréquentiels d'un support préexistant. Or, dans le cadre du *Diffusion Model-based Generative Image Steganography* (DM-GIS), il n'existe plus réellement de « support modifié », ce qui rend les approches traditionnelles incapables de distinguer une image saine d'une image porteuse d'un message caché. Pour répondre à cette problématique, le stéganalyste de référence actuel, NS-DSer [41], analyse la distribution statistique du bruit latent à l'origine de l'image générée, lequel suit généralement une distribution gaussienne. En effet, dans les approches de stéganographie générative, le message secret est le plus souvent

encodé directement dans ce bruit latent, ce qui entraîne inévitablement des perturbations dans ses statistiques ainsi que dans sa structure interne. Cependant, cette approche présente une limite importante : elle devient inefficace face aux méthodes conçues pour préserver la distribution gaussienne du bruit, comme le watermarking Gaussian Shading [36]. Dans ce cas, les modifications introduites par la dissimulation restent statistiquement proches du bruit naturel attendu, rendant la détection beaucoup plus difficile pour les méthodes de stéganalyse traditionnelles.

3 Contributions

L’objectif de ce mémoire est d’explorer de nouvelles pistes pour renforcer la stéganalyse face aux techniques de camouflage avancées. Notre démarche consiste, dans un premier temps, à réimplémenter et tester la méthode de référence NS-DSer afin de servir de base de comparaison. Dans un second temps, nous cherchons à améliorer cette détection en y intégrant de nouvelles couches algorithmiques. Nous explorons ainsi différentes méthodes de classification, l’utilisation d’autres propriétés statistiques du bruit extrait, ainsi qu’une approche alternative purement neuronale basée sur le Deep Learning. Ce travail propose une étude comparative de ces différents paradigmes afin de confronter leurs résultats et d’identifier la solution la plus performante.

4 Organisation du document

Outre l’introduction et la conclusion générale, le présent document est organisé en deux parties.

Dans la première partie, nous présentons un état de l’art réparti sur deux chapitres :

- Le **chapitre 1** expose les fondements théoriques et les architectures de base nécessaires à cette étude, notamment les réseaux de neurones (CNN, MLP), l’architecture U-Net, ainsi que les principes mathématiques de la loi normale et des modèles de diffusion.
- Le **chapitre 2** dresse un vaste panorama de l’évolution de la stéganalyse, en mettant en évidence le changement de paradigme introduit par le deep learning.

Dans la deuxième partie, nous présentons nos contributions qui sont répartis sur deux chapitres :

- Le **chapitre 3** présente nos contributions. Nous y expliquons comment nous avons recréé la baseline NS-DSer pour la tester, avant de détailler les différentes pistes explorées pour l’améliorer : l’ajout de nouvelles couches, le test d’autres classifications et statistiques, et l’évaluation d’une approche purement neuronale.
- Le **chapitre 4** est consacré à l’implémentation technique, à l’environnement expérimental et à l’analyse des résultats obtenus ainsi que les perspectives d’amélioration de notre modèle.

Première partie

État de l'art

CHAPITRE 1

FONDEMENTS THÉORIQUES

1.1 Introduction

Ce premier chapitre pose les bases théoriques indispensables pour comprendre les travaux de ce mémoire, situés au croisement de l'apprentissage profond, de la génération d'images et de la stéganalyse. Dans un premier temps, nous introduirons les principes fondamentaux des réseaux de neurones artificiels et les mécanismes de l'apprentissage profond. Dans un second temps, nous étudierons le fonctionnement des modèles de diffusion, notamment à travers l'exemple de Stable Diffusion, afin de comprendre comment la manipulation et l'analyse du bruit permettent de générer des images ou de détecter des messages cachés.

1.2 Les Réseaux de Neurones Artificiels

Les réseaux de neurones artificiels sont des modèles computationnels qui s'inspirent du fonctionnement du cerveau humain. Composés de plusieurs unités de calcul interconnectées (les neurones), ces réseaux possèdent la capacité d'apprendre automatiquement à partir des données pour réaliser des tâches complexes, telles que la classification d'informations ou la prédiction.

Lorsque ces réseaux sont structurés avec un grand nombre de couches superposées, on parle d'apprentissage profond (*Deep Learning* ou DL). Cette approche permet aux modèles d'apprendre des représentations de données à plusieurs niveaux d'abstraction. Ces méthodes ont considérablement amélioré l'état de l'art dans la reconnaissance vocale, la détection d'objets visuels, ainsi que dans de nombreux autres domaines scientifiques et technologiques [20].

Pour comprendre le fonctionnement de ces architectures complexes, il convient d'étudier d'abord la structure d'un réseau de neurones simple. Celui-ci est généralement composé de trois types de couches comme le montre la figure 1.1 :

- **Couche d'entrée** → reçoit les données brutes ;

- **Couches cachées** → une ou plusieurs couches qui réalisent les traitements et l'extraction de caractéristiques ;
- **Couche de sortie** → fournit le résultat final.

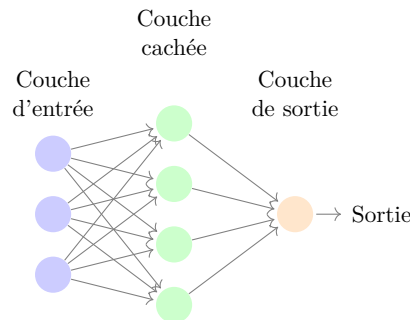


Figure 1.1 : Architecture simplifiée d'un réseau de neurones multicouche (MLP).

1.2.1 Structure d'un neurone artificiel

Le neurone artificiel constitue l'unité de base des réseaux de neurones artificiels. Son fonctionnement est inspiré du neurone biologique du cerveau humain. Il reçoit plusieurs valeurs en entrée, effectue des opérations mathématiques sur celles-ci, puis produit une sortie. L'objectif principal d'un neurone artificiel est de transformer les données d'entrée afin d'extraire des informations utiles pour la tâche à accomplir.

1.2.1.1 Entrées, poids et biais

Un neurone artificiel reçoit un ensemble de données appelées entrées. Ces entrées peuvent représenter différentes caractéristiques d'un problème, comme les pixels d'une image, des valeurs numériques ou des caractéristiques extraites d'un signal. Chaque entrée est associée à un poids. Les poids représentent l'importance de chaque information dans la décision prise par le neurone. Une entrée ayant un poids élevé influence davantage la sortie finale qu'une entrée ayant un poids faible.

Le neurone possède également un biais (bias), comme le montre la figure 1.2, qui permet d'ajuster le résultat obtenu indépendamment des entrées. Le biais améliore la flexibilité du modèle et facilite l'apprentissage de relations complexes entre les données.

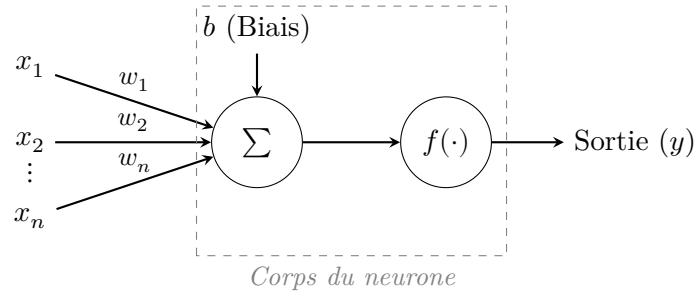


Figure 1.2 : Modèle mathématique d'un neurone artificiel : entrées pondérées, biais, sommation et activation.

1.2.1.2 Somme pondérée

Après la réception des entrées, le neurone calcule une combinaison linéaire appelée somme pondérée. Cette opération consiste à multiplier chaque entrée par son poids correspondant, puis à additionner tous les résultats avec le biais.

Mathématiquement, cette opération peut être exprimée comme suit :

$$z = \sum_{i=1}^n w_i x_i + b \quad (1.1)$$

Où :

- x_i : représente les entrées ;
- w_i : représente les poids associés ;
- b : représente le biais ;
- z : représente le résultat de la somme pondérée.

→ Cette étape permet au neurone de combiner les informations reçues avant de produire une réponse.

1.2.1.3 Fonctions d'activation

La fonction d'activation est la formule mathématique qui détermine la valeur de sortie d'un neurone à partir de ses entrées. C'est un élément essentiel : sans elle, le réseau se limiterait à une simple suite de calculs linéaires, ce qui bloquerait ses capacités d'apprentissage.

Plus précisément, les fonctions d'activation permettent :

- d'introduire de la non-linéarité dans le modèle, comme illustré dans la figure 1.3 ;
- de faciliter l'apprentissage de motifs complexes ;
- de contrôler la propagation des informations dans le réseau ;
- d'améliorer les performances du modèle sur des tâches complexes telles que la reconnaissance d'images ou le traitement du langage.

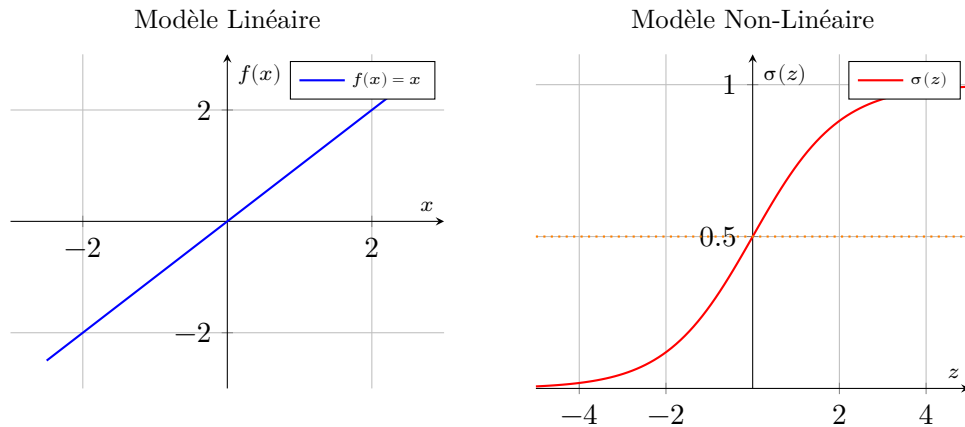


Figure 1.3 : Comparaison entre la linéarité et la non-linéarité (Fonction Sigmoidale).

Le fonctionnement complet d'un neurone artificiel peut être résumé par l'expression suivante :

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right) \quad (1.2)$$

où :

- x_i sont les entrées ;
- w_i sont les poids ;
- b est le biais ;
- f est la fonction d'activation ;
- y représente la sortie du neurone.

Cette formulation mathématique constitue la base du fonctionnement des réseaux de neurones artificiels et des modèles d'apprentissage profond.

Plusieurs fonctions d'activation existent, chacune possédant ses avantages et ses limites.

Fonction Sigmoidale

La fonction sigmoïde est l'une des premières fonctions d'activation utilisées dans les réseaux de neurones. Elle transforme toute valeur réelle en une valeur comprise entre 0 et 1.

Sa représentation mathématique est donnée par :

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1.3)$$

Caractéristiques

- La sortie appartient à l'intervalle $[0, 1]$;
- Elle est souvent utilisée pour les problèmes de classification binaire ;
- Elle possède une courbe en forme de “S”.

Avantages

- Fonction simple et facile à interpréter ;
- Permet de représenter des probabilités.

Inconvénients

- Souffre du problème du gradient qui disparaît (*vanishing gradient*) ;
- Les calculs exponentiels augmentent le coût computationnel ;
- Les sorties ne sont pas centrées autour de zéro.

Fonction Tangente Hyperbolique (Tanh)

La fonction tangente hyperbolique est similaire à la fonction sigmoïde, mais sa sortie varie entre -1 et 1 .

Elle est définie par :

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.4)$$

Caractéristiques

- Les sorties appartiennent à l'intervalle $[-1, 1]$;
- Fonction symétrique autour de zéro.

Avantages

- Généralement plus performante que la sigmoïde ;
- Les données centrées autour de zéro facilitent l'apprentissage.

Inconvénients

- Elle souffre également du problème du gradient qui disparaît ;
- Peut ralentir l'apprentissage dans les réseaux très profonds.

Fonction ReLU (Rectified Linear Unit)

La fonction ReLU est aujourd'hui la fonction d'activation la plus utilisée dans les réseaux de neurones profonds.

Elle est définie par :

$$f(x) = \max(0, x) \quad (1.5)$$

Fonctionnement

- Si $x < 0$, la sortie est égale à 0 ;
- Si $x > 0$, la sortie est égale à x .

Avantages

- Très simple et rapide à calculer ;
- Réduit le problème du gradient qui disparaît ;
- Accélère l'apprentissage des réseaux profonds.

Inconvénients

- Peut provoquer le problème des “neurones morts” ;
- Les valeurs négatives sont complètement supprimées.

Fonction Leaky ReLU

La fonction Leaky ReLU est une amélioration de la fonction ReLU. Elle permet de conserver une petite pente pour les valeurs négatives.

Elle est définie par :

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases} \quad (1.6)$$

où α représente une petite constante positive.

Avantages

- Réduit le problème des neurones morts ;
- Conserve les avantages de la ReLU.

Inconvénients

- Le choix du paramètre α peut influencer les performances.

Fonction Softmax

La fonction Softmax est principalement utilisée dans la couche de sortie des problèmes de classification multi-classes. Elle transforme un vecteur de valeurs en probabilités dont la somme est égale à 1.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}} \quad (1.7)$$

Caractéristiques

- Chaque sortie représente une probabilité ;
- La somme des probabilités est égale à 1.

Avantages

- Très adaptée aux tâches de classification ;
- Permet d'identifier la classe la plus probable.

Inconvénients

- Sensible aux grandes valeurs d'entrée ;
- Plus coûteuse en calcul que certaines autres fonctions.

Comparaison des principales fonctions d'activation

Fonction	Intervalle	Avantages	Inconvénients
Sigmoïde	$[0, 1]$	Interprétation probabiliste	Gradient qui disparaît
Tanh	$[-1, 1]$	Données centrées sur zéro	Gradient qui disparaît
ReLU	$[0, +\infty[$	Rapide et efficace	Neurones morts
Leaky ReLU	$] -\infty, +\infty[$	Corrige les neurones morts	Paramètre α
Softmax	$[0, 1]$	Classification multi-classes	Calcul plus coûteux

Tableau 1.1 : Comparaison des principales fonctions d'activation

Les fonctions d'activation jouent un rôle essentiel dans les performances des réseaux de neurones. Le choix de la fonction dépend principalement du type de problème traité ainsi que de l'architecture du réseau utilisé.

1.2.2 Entraînement d'un réseau de neurones

L'entraînement d'un réseau de neurones est le processus par lequel le modèle apprend à effectuer une tâche à partir de données. Durant cette phase, le réseau ajuste progressivement ses paramètres internes afin de réduire les erreurs entre les prédictions produites et les résultats attendus.

L'apprentissage repose principalement sur l'exploitation de grandes quantités de données ainsi que sur des méthodes d'optimisation permettant d'améliorer les performances du modèle au fil des itérations.

1.2.2.1 Jeux de données

Les données utilisées pour entraîner un réseau de neurones sont généralement divisées en plusieurs ensembles afin d'évaluer correctement les performances du modèle et d'éviter le surapprentissage (*overfitting*).

Jeu d'entraînement (*Training Set*)

Le jeu d'entraînement contient les données utilisées directement pour apprendre au modèle. Durant cette phase, les paramètres du réseau sont ajustés afin de minimiser les erreurs de prédiction.

Jeu de validation (*Validation Set*)

Le jeu de validation permet d'évaluer les performances du modèle pendant l'apprentissage. Il est principalement utilisé pour ajuster les hyperparamètres et surveiller le comportement du réseau afin d'éviter le surapprentissage.

Jeu de test (*Test Set*)

Le jeu de test est utilisé après l'entraînement afin de mesurer les performances finales du modèle sur des données jamais vues auparavant. Il permet d'évaluer la capacité de généralisation du réseau de neurones.

1.2.2.2 Fonction de perte (*Loss Function*)

La fonction de perte mesure l'erreur entre les prédictions du réseau et les valeurs réelles attendues. L'objectif de l'apprentissage est de minimiser cette erreur afin d'obtenir les meilleures performances possibles.

Plus la valeur de la fonction de perte est faible, plus les prédictions du modèle sont proches des résultats attendus.

Parmi les fonctions de perte les plus utilisées, on retrouve :

- l'erreur quadratique moyenne (*Mean Squared Error*) pour les problèmes de régression ;
- l'entropie croisée (*Cross-Entropy Loss*) pour les problèmes de classification.

Un exemple de fonction d'erreur quadratique moyenne est donné par :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1.8)$$

où :

- y_i représente la valeur réelle ;
- $\hat{y}_i$ représente la prédiction du modèle ;
- n représente le nombre d'échantillons.

1.2.2.3 Descente de gradient

La descente de gradient, figure 1.4, (*Gradient Descent*) est l'algorithme d'optimisation le plus utilisé pour entraîner les réseaux de neurones. Son objectif est de minimiser la fonction de perte en ajustant progressivement les poids du réseau.

Le principe consiste à calculer le gradient de la fonction de perte par rapport aux paramètres du modèle puis à modifier ces paramètres dans la direction opposée au gradient.

La mise à jour des poids peut être exprimée par :

$$w = w - \eta \frac{\partial L}{\partial w} \quad (1.9)$$

où :

- w représente les poids du réseau ;
- η représente le taux d'apprentissage (*learning rate*) ;

- L représente la fonction de perte.

La rétropropagation du gradient (*backpropagation*) est utilisée pour calculer efficacement les gradients dans les réseaux profonds.

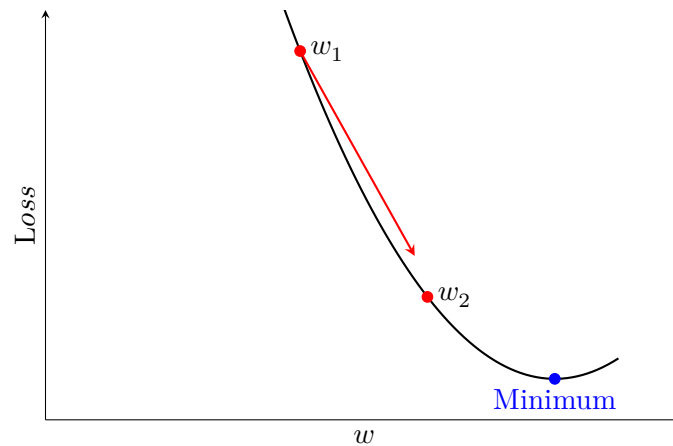


Figure 1.4 : Représentation simplifiée de la descente de gradient.

1.2.2.4 Optimisation

L'optimisation correspond à l'ensemble des techniques utilisées pour améliorer l'apprentissage du réseau de neurones et accélérer la convergence.

Plusieurs algorithmes d'optimisation existent :

- SGD (*Stochastic Gradient Descent*) ;
- Adam (*Adaptive Moment Estimation*) ;
- RMSProp ;
- AdaGrad.

L'optimiseur Adam est particulièrement populaire grâce à sa rapidité de convergence et sa stabilité lors de l'apprentissage des réseaux profonds.

1.2.2.5 Paramètres et hyperparamètres

Les paramètres et les hyperparamètres jouent un rôle essentiel dans le fonctionnement des réseaux de neurones.

Paramètres

Les paramètres sont les valeurs apprises automatiquement par le réseau durant l'entraînement. Ils correspondent principalement :

- aux poids ;
- aux biais.

Ces paramètres sont ajustés automatiquement par les algorithmes d'optimisation.

Hyperparamètres

Les hyperparamètres sont définis avant le début de l'entraînement et ne sont pas appris automatiquement par le modèle.

Parmi les hyperparamètres les plus importants :

- le taux d'apprentissage ;
- le nombre de couches ;
- le nombre de neurones ;
- la taille des lots (*batch size*) ;
- le nombre d'époques ;
- le choix de la fonction d'activation ;
- le choix de l'optimiseur.

Le choix des hyperparamètres influence fortement les performances finales du modèle.

1.2.2.6 Époques et lots de données

Époque (*Epoch*)

Une époque correspond à un passage complet de l'ensemble du jeu d'entraînement dans le réseau de neurones. Généralement, plusieurs époques sont nécessaires afin d'obtenir un apprentissage satisfaisant.

Lot de données (*Batch*)

Les données sont souvent divisées en petits groupes appelés lots (*batches*) afin de réduire la consommation mémoire et accélérer les calculs.

1.2.2.7 Surapprentissage et généralisation

Le surapprentissage (*overfitting*) se produit lorsque le modèle apprend excessivement les données d'entraînement et perd sa capacité à généraliser sur de nouvelles données.

À l'inverse, un modèle capable de produire de bonnes performances sur des données inconnues possède une bonne capacité de généralisation.

Plusieurs techniques permettent de limiter le surapprentissage comme :

- le dropout (*Désactivation aléatoire de neurones*) ;
- l'arrêt anticipé (*early stopping*).

L'objectif principal de l'entraînement est donc d'obtenir un modèle capable d'apprendre efficacement tout en conservant une bonne capacité de généralisation.

1.2.3 Boucle d'apprentissage d'un réseau de neurones

La boucle d'apprentissage représenté par la figure 1.5, décrit le fonctionnement global d'un réseau de neurones durant l'entraînement. Dans un premier temps, les données d'entrée sont introduites dans la couche d'entrée du réseau. Ces données transitent ensuite à travers les différentes couches cachées où chaque neurone effectue des calculs afin d'extraire des caractéristiques pertinentes. À la fin de cette propagation, le réseau produit une sortie correspondant à sa prédiction.

La sortie obtenue est ensuite comparée à la valeur réelle attendue afin de calculer l'erreur du modèle, appelée *loss*. Cette erreur permet d'évaluer la qualité des prédictions réalisées par le réseau. À partir de cette erreur, un algorithme d'optimisation ajuste progressivement les paramètres internes du modèle afin de réduire les erreurs lors des itérations suivantes.

Ce processus est répété plusieurs fois sur l'ensemble des données d'entraînement durant plusieurs époques (*epochs*) jusqu'à obtenir des performances satisfaisantes.

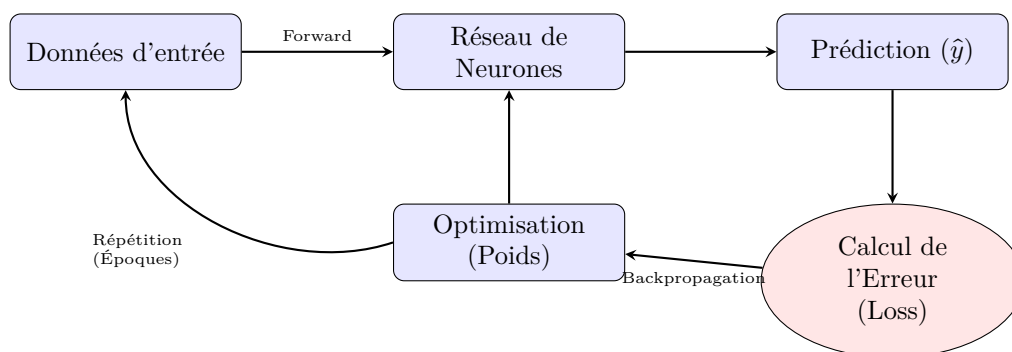


Figure 1.5 : Schéma de la boucle d'apprentissage d'un réseau de neurones.

1.2.4 Évaluation des réseaux de neurones

Une fois l'entraînement terminé, il est essentiel de mesurer les performances du modèle sur des données qu'il n'a jamais vues (le jeu de test). Cette évaluation permet de quantifier sa capacité de généralisation.

1.2.4.1 L'Accuracy

L'accuracy est la mesure la plus intuitive. Elle représente la proportion de prédictions cor-

rectes par rapport au nombre total d'échantillons traités.

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}} \quad (1.10)$$

Bien qu'utile, cette métrique peut être trompeuse si les classes sont déséquilibrées (par exemple, si 99% des données appartiennent à une seule classe).

1.2.4.2 La Matrice de Confusion

C'est un outil fondamental pour analyser les erreurs du modèle de manière détaillée. Elle permet d'identifier :

- **Vrais Positifs (VP)** : Prédits positifs, réellement positifs.
- **Faux Positifs (FP)** : Prédits positifs, réellement négatifs.
- **Faux Négatifs (FN)** : Prédits négatifs, réellement positifs.
- **Vrais Négatifs (VN)** : Prédits négatifs, réellement négatifs.

1.2.4.3 Précision et Rappel

Dans de nombreuses applications (comme le diagnostic médical), l'accuracy ne suffit pas. On utilise alors :

- **La Précision** : Parmi toutes les prédictions positives, combien étaient réellement correctes ?
- **Le Rappel** : Parmi tous les cas réels positifs, combien le modèle en a-t-il détectés ?
- **Le F1-Score** : La moyenne harmonique de la précision et du rappel, offrant un équilibre entre les deux.

1.2.5 Quelques Architectures de Réseaux de Neurones

Le domaine de l'apprentissage profond a vu l'émergence de diverses architectures, chacune étant optimisée pour des types de données spécifiques.

1.2.5.1 Le Perceptron Multicouche (MLP)

Le *Multi-Layer Perceptron* est l'architecture de base du deep learning. Il est composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie.

- **Fonctionnement** : Chaque neurone d'une couche est connecté à tous les neurones de la couche suivante (*Fully Connected*).
- **Usage** : Idéal pour des données tabulaires simples ou des problèmes de classification non complexes.

1.2.5.2 Les Réseaux de Neurones Convolutifs (CNN)

Les réseaux de neurones convolutifs sont particulièrement performants pour le traitement d'images et de données ayant une structure spatiale.

- **Convolution** : Utilise des filtres (noyaux), tels qu'on les perçoit dans la figure 1.6, pour extraire des caractéristiques locales (bords, textures).
- **Pooling** : Réduit la dimensionnalité des données tout en conservant les informations importantes.
- **Usage** : Reconnaissance faciale, détection d'objets, imagerie médicale.

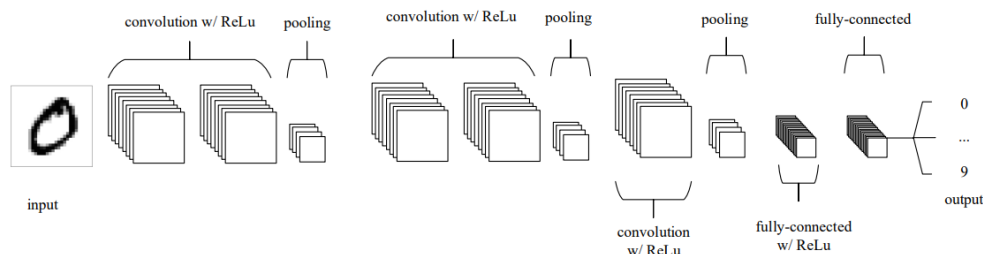


Figure 1.6 : Architecture d'un réseau de neurones convolutif (CNN) appliqué à la reconnaissance de chiffres manuscrits [21].

1.2.5.3 Les Transformers [33]

La nécessité du traitement textuel dans la génération d'images

Bien que l'objectif principal de ce travail porte sur la génération ou l'analyse d'images via des modèles comme *Stable Diffusion*, l'intégration d'une architecture de traitement du langage est devenue indispensable. En effet, pour qu'un modèle puisse générer une image fidèle à la volonté de l'utilisateur, il doit d'abord "comprendre" textuellement la description fournie (le *prompt*). C'est précisément à cette étape qu'interviennent les *Transformers* : ils servent d'encodeurs de texte textuels permettant de traduire les mots du *prompt* en représentations vectorielles denses (embeddings) que le modèle de génération d'images peut ensuite exploiter. Introduite initialement pour la traduction automatique, l'architecture des *Transformers* a révolutionné le domaine de l'apprentissage profond en remplaçant les structures récurrentes (RNN) et convolutives (CNN) par un mécanisme entièrement basé sur l'attention.

- **Définition** : C'est une architecture de réseau de neurones qui traite les données d'entrée de manière globale et parallèle. Elle repose sur le mécanisme d'*auto-attention*

(*self-attention*), qui permet au modèle de capturer les relations et les dépendances entre tous les éléments d'une séquence, quelle que soit leur distance.

- **Usage et Exemples :** Bien qu'ils soient la fondation des modèles de langage modernes (comme GPT [25], BERT [9] ou LLaMA [31]), les Transformers sont également utilisés en vision par ordinateur (*Vision Transformers* [10]) et pour le traitement du signal audio.

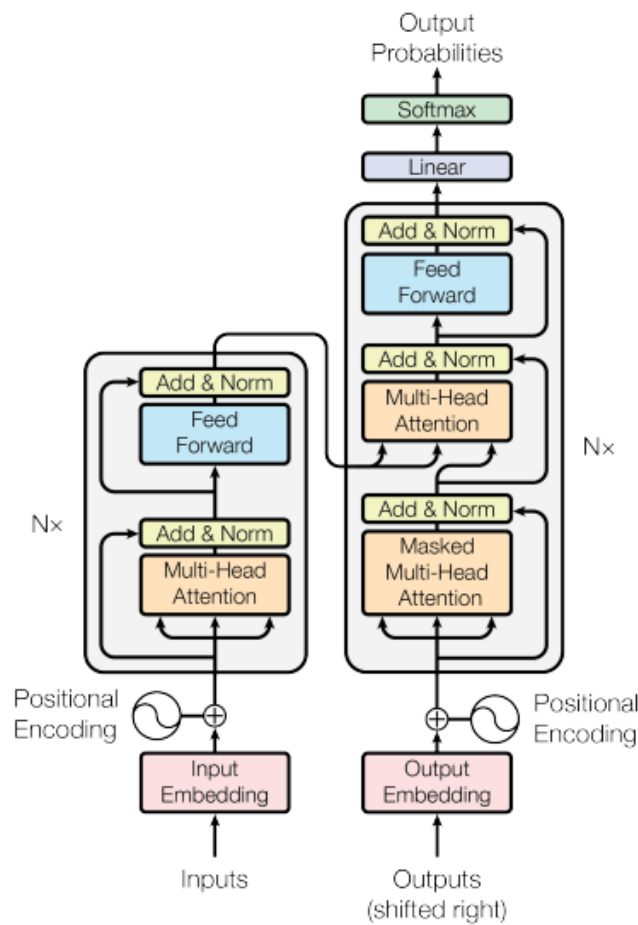


Figure 1.7 : Architecture générale du modèle Transformer (Encodeur-Décodeur) [33].

L'architecture du Transformer (Figure 1.7) repose sur une structure de type encodeur-décodeur (*Encoder-Decoder*) qui abandonne les mécanismes séquentiels traditionnels au profit du mécanisme d'auto-attention.

Le Bloc Encodeur (Partie Gauche)

L'encodeur traite la séquence d'entrée (le texte du *prompt*) pour en extraire les caractéristiques importantes.

- **Multi-Head Attention** : Permet au modèle d'analyser les relations entre tous les mots de la phrase simultanément, peu importe leur distance.
- **Feed Forward (FFN)** : Un réseau de neurones standard qui traite l'information de manière indépendante pour chaque mot.

Le Bloc Décodeur (Partie Droite)

Le décodeur génère la séquence de sortie de manière itérative.

- **Masked Multi-Head Attention** : Analyse les mots déjà générés en masquant les mots futurs pour éviter que le modèle ne "triche".
- **Attention Encodeur-Décodeur** : Fait le lien entre les informations issues de l'encodeur et la génération en cours dans le décodeur.

Le flux d'information est complété par deux éléments clés :

- **Positional Encoding (Codage Positionnel)** : Ajoute une information mathématique sur l'ordre des mots, car le réseau traite tout en parallèle et n'a nativement aucune notion de structure temporelle.
- **Add & Norm** : Des connexions résiduelles et une normalisation qui stabilisent l'apprentissage et facilitent la circulation du gradient à travers les blocs.

1.3 Génération d'images synthétiques

À première vue, ce travail étant centré sur la stéganalyse, l'intérêt d'aborder l'Intelligence Artificielle (IA) générative peut sembler ambigu. Cependant, il s'agit d'un point fondamental : pour détecter des messages cachés dans des médias synthétiques, il est crucial de comprendre la distribution statistique de ces images. Le lien entre stéganalyse et génération d'images réside dans la manipulation des textures et du bruit au niveau du pixel.

Bien que plusieurs approches existent, nous nous concentrerons sur les **modèles de diffusion**. Avant d'en détailler le fonctionnement, rappelons quelques concepts statistiques de base.

1.3.1 Quelques rappels statistiques

1.3.1.1 La loi normale (Distribution Gaussienne)

La loi normale, est une distribution statistique très utilisée pour représenter des phénomènes naturels de la vie réelle. De nombreuses mesures suivent approximativement cette loi, comme la taille des personnes, les notes d'un examen ou certaines erreurs de mesure. Elle se caractérise par une forme en cloche (figure 1.8) où la majorité des valeurs se concentrent autour de la moyenne, tandis que les valeurs très petites ou très grandes deviennent plus rares. Sa fonction de densité de probabilité est donnée par l'équation :

$$f(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \quad (1.11)$$

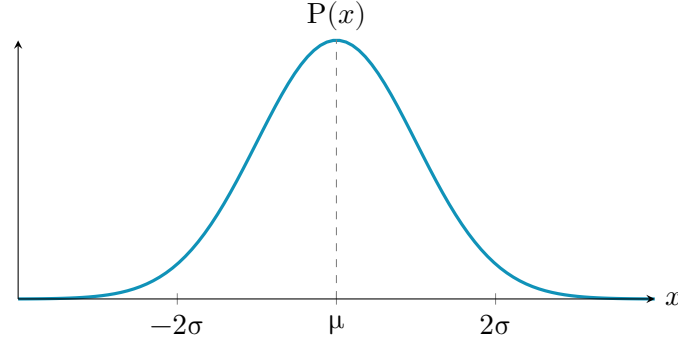


Figure 1.8 : Représentation de la loi normale centrée en $\mu = 0$ avec un écart-type σ .

1.3.1.2 L'écart-type

L'écart type mesure la dispersion des données autour de la moyenne. Il indique si les valeurs sont proches les unes des autres ou au contraire très étalées. Mathématiquement, il est la racine carrée de la variance :

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (1.12)$$

1.3.1.3 Une distribution de probabilité

Une distribution (ou loi) de probabilité est une fonction mathématique qui décrit la probabilité qu'un événement ou qu'une variable aléatoire prenne une valeur donnée.

1.3.1.4 Distribution des images

En apprentissage profond, une image n'est pas vue comme un simple ensemble de pixels, mais comme un échantillon provenant d'une distribution de probabilité complexe de haute dimension (l'espace des images réelles). Le but de l'IA générative est de modéliser et d'apprendre cette distribution.

1.3.1.5 Les chaînes de Markov

Une chaîne de Markov est un processus stochastique (aléatoire) où la transition d'un état à un autre dépend uniquement de l'état précédent. En d'autres termes, l'état futur dépend

uniquement de l'état actuel, sans tenir compte directement des états passés (propriété de Markov).

Mathématiquement, si l'on note $x_0, x_1, \dots, x_t$ la séquence des états successifs, la probabilité conditionnelle de l'état x_t se formalise par :

$$P(x_t \mid x_0, x_1, \dots, x_{t-1}) = P(x_t \mid x_{t-1}) \quad (1.13)$$

Dans le cadre des modèles de diffusion, le processus de dégradation (ajout de bruit) et le processus de reconstruction (débruitage) sont modélisés comme des chaînes de Markov, où chaque étape d'image x_t dépend uniquement de l'image à l'étape x_{t-1} ou x_{t+1} .

On peut représenter graphiquement les chaînes de Markov sous la forme d'un graphe orienté (ou automate) (figure 1.9), où les états sont reliés par des flèches indiquant les probabilités de transition d'un état à un autre.

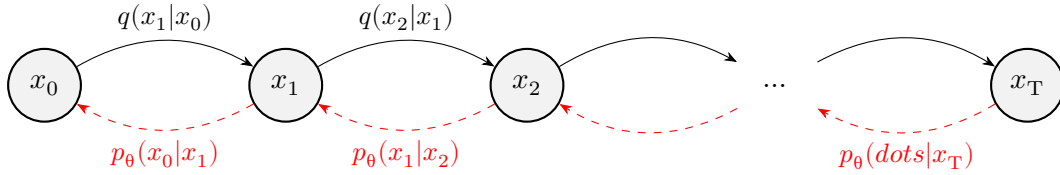


Figure 1.9 : Représentation d'une chaîne de Markov appliquée au processus de diffusion. Les flèches pleines décrivent le processus direct (ajout de bruit q), et les flèches pointillées rouges décrivent le processus inverse (génération/débruitage p_θ).

1.3.1.6 Le Bruit Gaussien

De manière générale, en traitement d'image, le **bruit** désigne une perturbation aléatoire et indésirable qui vient modifier la valeur d'origine des pixels, agissant comme un signal parasite imprévisible.

Le **bruit gaussien** est un cas particulier de bruit où la valeur de la perturbation injectée sur chaque pixel n'est pas complètement chaotique : elle suit mathématiquement la loi normale (la courbe en cloche présentée ci-dessus). La majorité des pixels subissent une modification mineure (proche de la moyenne $\mu = 0$), tandis qu'une minorité subit de fortes altérations. Dans les modèles de diffusion, ce bruit est qualifié de bruit blanc gaussien $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ et est ajouté de manière itérative selon la relation [15] :

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon \quad (1.14)$$

où β_t contrôle la quantité de bruit injectée à l'étape t .

1.3.2 Les Modèles de diffusion

Les modèles de diffusion (*Diffusion Models*) représentent une classe de modèles génératifs probabilistes qui surclassent aujourd'hui la plupart des architectures précédentes (comme

les GAN) dans la génération d'images haute fidélité. Le principe fondamental de ces modèles repose sur l'idée de détruire progressivement la structure d'une image par l'ajout de bruit, puis d'apprendre à inverser mathématiquement ce processus pour reconstruire l'image originale à partir d'un bruit pur.

1.3.2.1 L'origine thermodynamique

Bien que popularisés récemment, les fondements théoriques de ces modèles ont été posés par Sohl-Dickstein et al. [28] dans leur article précurseur intitulé « *Deep Unsupervised Learning using Nonequilibrium Thermodynamics* ». Inspirés par la physique statistique et la thermodynamique hors équilibre, les auteurs ont introduit le concept de diffusion en montrant qu'il était possible d'annihiler la distribution de probabilité d'une donnée complexe en utilisant un processus de diffusion (similaire à la diffusion d'un gaz). Ils ont prouvé qu'un réseau de neurones pouvait être entraîné à inverser ce système de diffusion pour restaurer la structure initiale, ouvrant ainsi la voie à une toute nouvelle approche générative.

1.3.2.2 La formalisation moderne

Le véritable saut de performance et la démocratisation de cette technologie ont eu lieu avec l'article « *Denoising Diffusion Probabilistic Models* » (DDPM) publié par Ho et al. [15]. Les auteurs ont reformulé l'approche thermodynamique initiale en établissant un lien d'équivalence direct avec le *score matching* et les techniques de débruitage progressif. C'est dans ce papier qu'est formalisée la paramétrisation moderne de la fonction de perte (*loss function*), simplifiant l'entraînement des réseaux de neurones (souvent basés sur une architecture U-Net) chargés de prédire le bruit gaussien injecté à chaque étape. Ce sont les équations de cet article DDPM qui servent aujourd'hui de socle mathématique aux diffuseurs modernes, incluant les modèles de pointe tels que *Stable Diffusion*.

1.3.2.3 Fonctionnement détaillé des Modèles de Diffusion (DDPM)

Les modèles de diffusion, et plus particulièrement les *Denoising Diffusion Probabilistic Models* (DDPM) introduits par Ho et al. [15], reposent sur une double dynamique stochastique : un processus direct (*Forward*) qui détruit l'information, et un processus inverse (*Reverse*) qui apprend à la reconstruire.

Le Processus Forward (Ajout de bruit)

Soit $x_0 \sim q(x)$ une image réelle issue de notre base de données. Le processus direct consiste à appliquer une chaîne de Markov qui, à chaque étape $t \in \{1, \dots, T\}$, injecte une légère perturbation sous forme de bruit blanc gaussien. Ce processus est régi par un calendrier de paramètres $\beta_t \in (0, 1)$ appelé *variance schedule*. La transition d'un état à l'autre est définie par la loi conditionnelle suivante :

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I}) \quad (1.15)$$

Où $\mathbf{I}$ représente la matrice identité, garantissant que le bruit est injecté de manière indépendante sur chaque pixel.

L'astuce de reparamétrisation

Calculer x_t en itérant l'équation (1.15) étape par étape serait très coûteux. Heureusement, les propriétés des variables gaussiennes permettent de définir une formulation directe depuis x_0 . En posant :

$$\alpha_s = 1 - \beta_s \quad \text{et} \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s \quad (1.16)$$

Nous pouvons exprimer la loi de x_t sachant uniquement l'image originale x_0 par la formule fermée :

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)\mathbf{I}) \quad (1.17)$$

En pratique, cela permet d'échantillonner directement n'importe quel état intermédiaire x_t durant l'entraînement via la relation :

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon \quad \text{où } \epsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (1.18)$$

Le terme $\sqrt{\bar{\alpha}_t}$ quantifie le signal d'origine préservé, tandis que $\sqrt{1 - \bar{\alpha}_t}$ mesure l'amplitude du bruit accumulé. Lorsque $t \rightarrow T$ (généralement $T = 1000$), $\bar{\alpha}_t \rightarrow 0$, transformant l'image x_0 en un bruit gaussien pur $x_T \sim \mathcal{N}(0, \mathbf{I})$.

Le Processus Reverse

Pour générer une nouvelle image, le modèle doit effectuer le chemin inverse : partir d'un bruit pur x_T et remonter la chaîne de Markov jusqu'à x_0 en estimant la distribution $q(x_{t-1} | x_t)$. Cependant, cette distribution globale est intraitable mathématiquement sans connaître l'ensemble des images existantes. La solution consiste à approximer ce processus inverse à l'aide d'un réseau de neurones paramétré par des poids θ , noté $p_\theta(x_{t-1} | x_t)$:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (1.19)$$

Si l'on conditionne ce calcul par rapport à l'image originale x_0 en utilisant le théorème de Bayes, la trajectoire inverse devient analytiquement calculable : $q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t\mathbf{I})$. Ho et al. ont démontré que l'objectif optimal pour le réseau est de prédire non pas directement l'image, mais le bruit exact ϵ injecté à l'étape t . La moyenne théorique estimée par le réseau se reformule alors ainsi :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \quad (1.20)$$

L'étape de génération (échantillonnage) consiste donc à itérer de $t = T$ à 1 en appliquant

la règle suivante :

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z \quad \text{où } z \sim \mathcal{N}(0, \mathbf{I}) \quad (1.21)$$

La figure 1.10 ci-dessous, montre très bien le déroulement des deux processus de diffusion qu'on a cité auparavant.

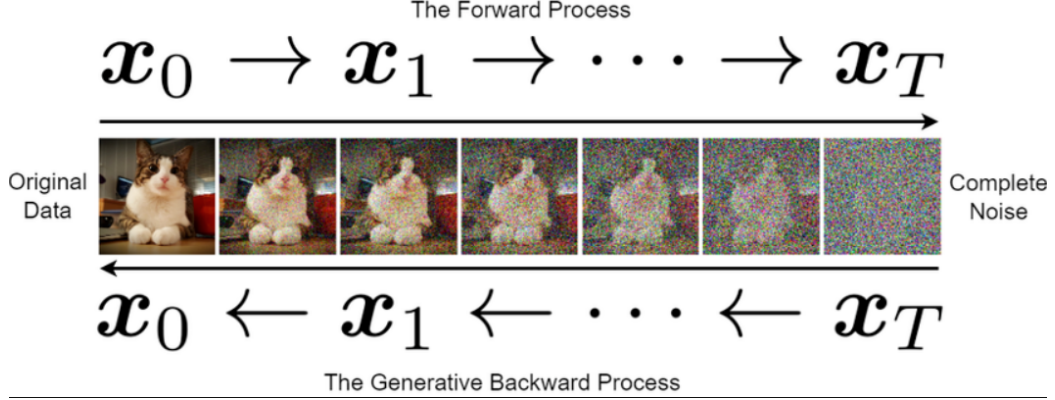


Figure 1.10 : Représentation visuelle du processus direct (*Forward Process*) et du processus inverse (*Generative Backward Process*) [13].

Architecture U-Net et Fonction de Perte (Loss)

Le réseau de neurones $\epsilon_\theta(x_t, t)$ prend en entrée l'image bruitée x_t ainsi que l'indice temporel t (encodé sous forme d'embedding sinusoïdal). Sa structure repose sur un réseau de type **U-Net** équipé de connexions résiduelles (*skip connections*) pour préserver la géométrie et les détails des pixels.

Pour entraîner ce réseau, la fonction de perte simplifiée établie par le modèle DDPM applique l'erreur quadratique moyenne (*Mean Squared Error* - MSE) entre le bruit réel ϵ ajouté à l'équation (1.18) et le bruit prédit par le modèle :

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta \left(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t \right) \right\|^2 \right] \quad (1.22)$$

C'est cette fonction de perte épurée qui assure la grande stabilité d'entraînement des modèles de diffusion par rapport aux réseaux antagonistes génératifs (GAN).

1.3.2.4 Processus d'inférence avec Stable Diffusion

Pour inférer une image avec Stable Diffusion, le modèle part d'une description textuelle (*prompt*) afin de créer une image de toutes pièces. Ce processus transforme un bruit aléatoire initial en une image propre en suivant un parcours géométrique précis.

Le mécanisme se déroule en trois étapes clés :

- **Le prompt et CLIP** : Le texte tapé par l'utilisateur est traduit en vecteurs de caractéristiques par le modèle linguistique **CLIP** [24]. Ces informations servent de guide pour orienter la création de l'image selon le sens du message.
- **L'U-Net** : À chaque étape du processus, le réseau **U-Net** (décrit précédemment) analyse l'état bruité actuel. Son rôle ici est de prédire le bruit résiduel afin d'indiquer la direction mathématique à prendre pour se rapprocher de l'image finale.
- **La PF-ODE et le Scheduler** : C'est ici que les mathématiques structurent le déplacement. Au lieu de retirer le bruit à l'aveugle, on modélise le chemin sous la forme d'une équation différentielle appelée **PF-ODE** (*Probability Flow ODE*). Cette équation trace une trajectoire courbe et continue qui relie mathématiquement le bruit initial x_T à l'image finale propre x_0 .

Comme un ordinateur ne peut pas suivre une courbe continue parfaite, on utilise un **scheduler** (échantillonneur). Le *scheduler* sert de guide de navigation : il découpe la trajectoire en un nombre réduit d'étapes discrètes (par exemple 20 ou 30 pas). À chaque pas t , il utilise la prédiction de l'U-Net pour calculer le vecteur de direction et déplacer le point le long de la courbe :

$$x_{t-1} = x_t + \text{Pas} \times \text{Direction de débruitage}(x_t, t) \quad (1.23)$$

Une fois ce voyage terminé, le décodeur du VAE convertit le point final x_0 de l'espace latent en une image visible dans l'espace des pixels.

L'ensemble de ce parcours géométrique est illustré par la figure 1.11. On y voit comment la PF-ODE définit le chemin idéal en courbe, tandis que le *scheduler* guide le modèle pas à pas, de point en point, pour atteindre l'image finale x_0 .

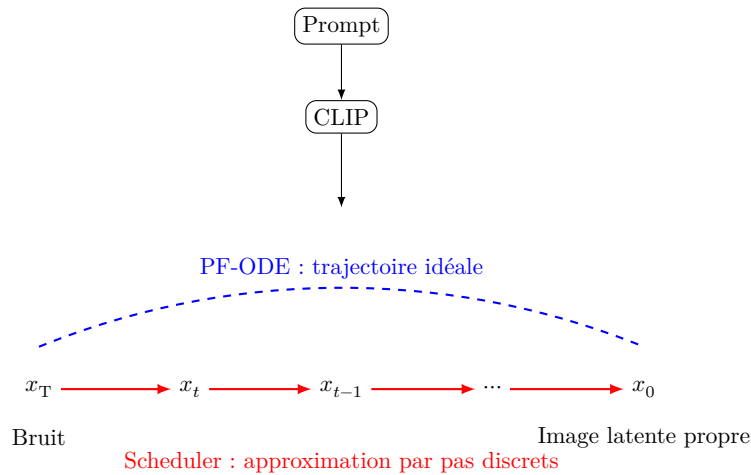


Figure 1.11 : Processus d'inférence de Stable Diffusion

1.3.2.5 Stable Diffusion et l'Espace Latent (LDM) [26]

Bien que les modèles DDPM soient mathématiquement puissants, leur principal défaut réside dans leur coût de calcul computationnel. Appliquer les processus de diffusion et de génération directement dans l'espace des pixels (*Pixel Space*) sur des images de haute résolution nécessite des ressources colossales, car le réseau U-Net doit traiter chaque pixel à chaque étape temporelle t .

Pour résoudre cette limitation, Rombach et al. [26] ont introduit les modèles de diffusion latente (*Latent Diffusion Models* - LDM), plus connus sous le nom commercial de **Stable Diffusion**. L'idée directrice est de diviser le processus de génération en deux phases distinctes : une phase de compression perceptive et une phase de génération statistique.



Figure 1.12 : Image synthétique générée par le modèle *Stable Diffusion v1.5* après $T = 20$ étapes d'échantillonnage, basée sur le prompt : « *a cat sitting on a chair* ».

La Figure 1.12 présente un exemple concret de génération obtenu avec l'architecture *Stable Diffusion v1.5*. Pour ce test, le processus de débruitage a été configuré sur $T = 20$ étapes d'échantillonnage (*sampling steps*).

Dans cette version spécifique du modèle, le *prompt* textuel « *a cat sitting on a chair* » est d'abord traduit en vecteurs de caractéristiques (*embeddings*) par l'encodeur de texte CLIP [24], un modèle s'appuyant directement sur l'architecture *Transformer*. Ces vecteurs guident ensuite l'U-Net à chaque étape de la chaîne de Markov inverse pour orienter la soustraction du bruit gaussien. En seulement 20 étapes, la distribution de probabilité initiale (bruit pur) converge vers une image synthétique stable, texturée et sémantiquement fidèle à la description fournie.

L'Auto-encodeur Variationnel (VAE)

Le cœur de cette architecture repose sur un *Variational Autoencoder* (VAE) préalablement entraîné sur un large ensemble d'images. Le VAE est composé de deux sous-réseaux :

- **L'Encodeur ($\mathcal{E}$)** : Il prend une image de l'espace des pixels et la projette dans un espace latent de dimension nettement inférieure. Généralement, le facteur de compression est de 8 (par exemple, une image $512 \times 512 \times 3$ devient un vecteur latent $64 \times 64 \times 4$). Cet espace latent est débarrassé des détails haute fréquence superflus tout en conservant la structure sémantique et géométrique essentielle de l'image.
- **Le Décodeur ($\mathcal{D}$)** : Une fois le processus de diffusion terminé dans l'espace latent, le décodeur prend le vecteur latent final et le reconstruit pour recréer une image haute résolution dans l'espace des pixels.

La Diffusion dans l'Espace Latent

Grâce au VAE, la chaîne de Markov (les processus *Forward* et *Reverse* étudiés précédemment) ne s'applique plus sur l'image x_t , mais directement sur la représentation latente z_t .

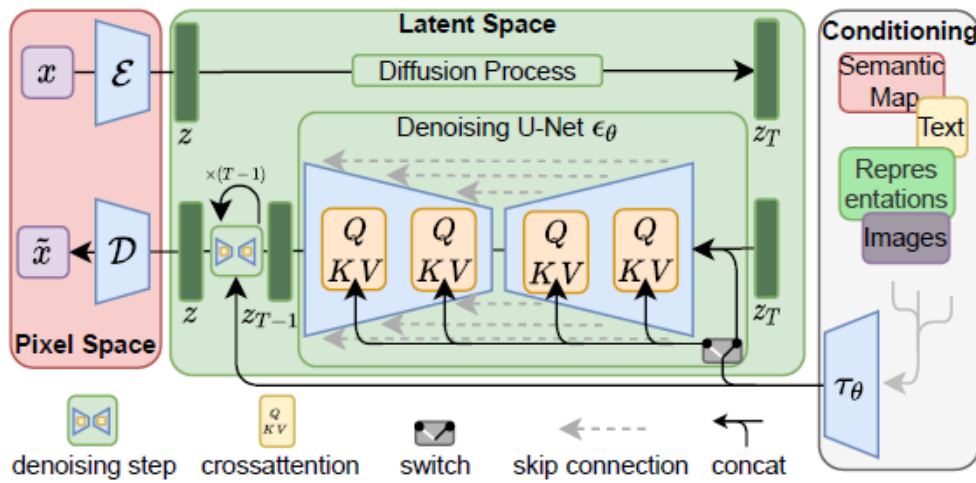


Figure 1.13 : Architecture globale d'un modèle de diffusion dans l'espace latent (*Latent Diffusion Model*) [26].

La Figure 1.13 synthétise le fonctionnement de l'architecture LDM et met en évidence la séparation entre l'espace des pixels (*Pixel Space*, à gauche) et l'espace latent compressé (*Latent Space*, au centre). Le mécanisme se décompose en trois grands blocs interconnectés :

- **La compression perceptive (Bloc rouge)** : L'encodeur $\mathcal{E}$ projette l'image initiale x dans l'espace latent sous la forme d'un vecteur compressé z . À l'inverse, une fois la génération finalisée, le décodeur $\mathcal{D}$ effectue la reconstruction pour projeter le vecteur propre z nettoyé vers l'image synthétique finale $\tilde{x}$.
- **Le cœur génératif (Bloc vert)** : Au lieu de bruiteur l'image pixel par pixel, le processus direct (*Diffusion Process*) ajoute du bruit gaussien au vecteur latent pour obtenir

z_T . Le réseau de neurones U-Net (ϵ_θ) prend le relais pour effectuer le débruitage étape par étape (*denoising step*). On remarque la présence de connexions résiduelles (*skip connections*) symbolisées par les flèches en pointillés gris, qui aident à préserver la cohérence des structures spatiales latentes.

- **Le mécanisme de conditionnement (Bloc gris, à droite) :** Pour guider la génération (via du texte, des cartes sémantiques ou d'autres images), les informations externes y passent par un encodeur spécifique τ_θ . Ces informations sont ensuite injectées directement au cœur de l'U-Net grâce à des modules d'attention croisée (*Cross-Attention*), matérialisés par les blocs oranges (Q, K, V). Ces blocs effectuent des projections de requêtes (*Query*), clés (*Key*) et valeurs (*Value*) pour forcer le bruit à s'aligner sémantiquement avec le prompt de l'utilisateur.

1.3.2.6 Variante des diffuseurs : Les modèles DDIM [29]

Bien que performante, l'architecture DDPM souffre d'une grande lenteur lors de la génération car elle impose de remonter la chaîne de Markov étape par étape sur la totalité des pas (souvent $T = 1000$). Pour surmonter ce coût computationnel, Song et al. [29] ont introduit les **Denoising Diffusion Implicit Models** (DDIM). En s'appuyant sur un processus de diffusion non-markovien déterministe (où la variance résiduelle σ_t est fixée à zéro), la trajectoire de génération devient entièrement implicite. Cette modification cruciale permet de "sauter" des étapes lors de l'inférence en n'échantillonnant que sur un sous-ensemble réduit (par exemple 20 ou 50 étapes au lieu de 1000). Cela accélère la génération d'un facteur 50 sans perte significative de qualité visuelle, tout en offrant une propriété d'inversibilité parfaite très utile pour l'analyse d'images.

1.4 Conclusion

Ce premier chapitre nous a permis de poser toutes les bases techniques nécessaires pour la suite. Nous avons d'abord vu le fonctionnement des réseaux de neurones, puis quelques concepts mathématiques indispensables. Enfin, nous avons détaillé les diffuseurs ainsi que tout le processus moderne de la génération des images.

Toutes ces notions sont essentielles pour comprendre la structure et les statistiques des images synthétiques. Maintenant que ce bagage technique est bien en place, nous pouvons passer à la suite. Le chapitre suivant sera entièrement dédié à l'état de l'art de la stéganalyse, où nous passerons en revue les méthodes classiques et les méthodes actuelles pour détecter ces fausses images et repérer les messages cachés.

CHAPITRE 2

ÉTUDE DES MÉTHODES DE RÉFÉRENCE

2.1 Introduction

Après avoir posé dans le chapitre précédent les bases théoriques de l'apprentissage profond, et des modèles de diffusion, ce chapitre est entièrement consacré à l'étude de l'état de l'art du domaine. Nous retracerons d'abord l'évolution historique de la stéganalyse, marquée par un véritable changement de paradigme (*paradigm shift*) des méthodes statistiques vers l'apprentissage profond. Nous analyserons ensuite l'émergence des nouvelles approches de stéganographie générative basées sur les modèles de diffusion, regroupées sous l'appellation DM-GIS (*Diffusion Model-based Generative Image Steganography*). Après avoir identifié les défis actuels posés par ces images synthétiques, nous introduirons le stéganalyste récent NS-DSer [41] en détaillant son architecture et ses principes pour la détection de ces contenus complexes.

2.2 La stéganalyse

La stéganalyse est une discipline de la sécurité informatique qui s'intéresse à la détection d'informations cachées. Pour bien comprendre ses mécanismes, il est indispensable de l'étudier en étroite relation avec sa contrepartie, la stéganographie.

2.2.1 Définition de la stéganalyse

La stéganalyse est l'art et la science de détecter la présence de messages cachés au sein d'un support numérique. Son objectif principal n'est pas nécessairement de déchiffrer ou de lire le message secret, mais d'établir avec certitude si un média (une image, un son ou un texte) a été modifié pour contenir une information dissimulée.

Dans la littérature scientifique, la stéganalyse est souvent formalisée comme un problème de classification binaire où un algorithme doit distinguer un support sain d'un support modifié. Comme le soulignent Fridrich et Kodovsky [12], la stéganalyse moderne repose massivement sur l'apprentissage automatique pour capturer les anomalies statistiques subtiles laissées par les processus d'insertion.

2.2.2 Définition de la stéganographie

La stéganographie est la pratique qui consiste à dissimuler un message secret au sein d'un autre message ou support physique, de telle sorte que personne d'autre que l'expéditeur et le destinataire ne soupçonne l'existence même du message. Contrairement à la cryptographie, qui rend le message illisible mais visible, la stéganographie cherche à rendre le message totalement invisible.

Le principe fondamental repose sur l'exploitation des redondances ou du bruit naturel d'un support numérique pour y injecter l'information secrète sans en altérer l'apparence visuelle ou sonore. Selon l'article de référence de Cox et al. [7], l'efficacité d'un algorithme stéganographique se mesure à sa capacité à maintenir l'illusion d'un média parfaitement ordinaire, garantissant ainsi l'indétectabilité du canal de communication.

2.2.3 Les caractéristiques fondamentales de la stéganographie

Pour évaluer la performance et l'efficacité d'un système stéganographique, on s'appuie généralement sur plusieurs critères essentiels. Voici les principales caractéristiques à prendre en compte :

- **L'imperceptibilité (ou invisibilité) :** C'est la caractéristique la plus importante. L'image modifiée (*stego*) doit être visuellement identique à l'image d'origine (*cover*). Aucun défaut, pixel suspect ou changement de couleur ne doit éveiller les soupçons d'un observateur humain.
- **La sécurité :** Contrairement à l'imperceptibilité qui trompe l'œil humain, la sécurité consiste à tromper les outils informatiques (les stéganalyseurs). Le fait de cacher un message ne doit pas modifier les propriétés ou les statistiques mathématiques de l'image (comme la distribution de son bruit).
- **La capacité d'insertion (ou charge utile / *Payload*) :** Cela représente la quantité maximale d'informations (en bits ou en octets) que l'on peut cacher à l'intérieur de l'image sans détruire sa qualité ni sa sécurité. Plus le message est gros, plus il est difficile de le garder secret.
- **La robustesse :** C'est la capacité du message caché à survivre si l'image subit des modifications ou des attaques après son envoi. Par exemple, si l'image est légèrement compressée, recadrée ou redimensionnée, un système robuste permettra au destinataire de retrouver quand même le message sans erreurs.
- **La confidentialité :** Elle garantit que même si on découvre qu'un message est caché dans l'image, il est impossible de le lire. En général, on combine la stéganographie avec la cryptographie.

2.2.4 Enjeux de la stéganalyse et de la stéganographie

Les interactions entre la stéganographie et la stéganalyse soulèvent des enjeux majeurs de sécurité et de confidentialité. Dans le domaine de la cybersécurité et de la criminalistique numérique (*digital forensics*), la stéganalyse est cruciale pour intercepter des communications malveillantes, comme la fuite de données d'entreprises ou la coordination d'attaques cybernétiques utilisant des images anodines sur les réseaux sociaux.

Ces deux disciplines vivent une véritable course aux armements : le développement de nouvelles techniques de dissimulation conduit continuellement au développement des méthodes de détection plus performantes. Récemment, l'apparition de l'intelligence artificielle générative a bouleversé cet équilibre. Les modèles capables de synthétiser des images de toutes pièces obligent désormais la stéganalyse à s'adapter pour repérer des messages cachés au cœur même des textures générées par les IA.

2.2.5 Terminologie

Pour étudier la stéganalyse de manière rigoureuse, il convient de définir précisément les termes techniques employés dans ce domaine.

2.2.5.1 Cover

Le *Cover* (ou support de couverture) désigne le média d'origine, totalement sain et vierge de toute modification. C'est l'image initiale issue d'un appareil photo ou d'une base de données avant qu'aucun message n'y soit dissimulé.

2.2.5.2 Stego

Le *Stego* désigne le support final qui contient le message caché. C'est le résultat obtenu après l'application de l'algorithme stéganographique sur le *Cover*. Visuellement, il est identique au *Cover*, mais ses statistiques internes sont différentes. Ces concepts sont bien illustrés dans la figure 2.1, où l'on voit également que la différence entre l'image stego et l'image cover donne le résidu contenant le message caché.

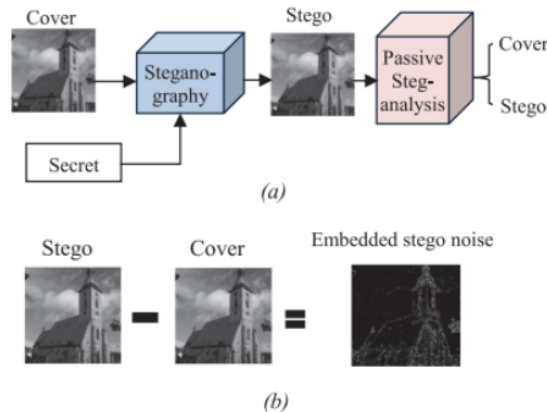


Figure 2.1 : Illustration des concepts de Cover et Stego [27]

2.2.5.3 DM-GIS (Diffusion Model-based Generative Image Steganography)

La stéganographie sans support (coverless steganography) est une approche moderne où l'on n'altère plus une image existante pour y cacher un message. À la place, l'image contenant le secret est entièrement générée de toutes pièces par une intelligence artificielle. Dans cette catégorie, la stéganographie générative basée sur les modèles de diffusion (DM-GIS) se distingue particulièrement. Parmi les techniques récentes qui exploitent cette approche, on retrouve notamment CroSS [40], mas-GRDH [18], ou encore le Gaussian Shading [36].

- **Intérêt pour la stéganalyse :** DM-GIS représente un défi majeur pour la stéganalyse statistique traditionnelle. Comme il n'y a pas d'image *Cover* d'origine ayant subi des modifications, il n'y a aucun bruit d'insertion classique à isoler. L'image générée possède des statistiques naturelles cohérentes. C'est précisément ce type de méthode qui pousse la stéganalyse moderne à s'orienter vers l'apprentissage profond et l'étude des artefacts spécifiques laissés par les modèles génératifs.

2.2.5.4 Payload

Le payload (ou charge utile) désigne la quantité maximale d'informations secrètes que l'on peut dissimuler dans une cover sans éveiller les soupçons. Qu'il s'agisse d'un texte, d'un fichier binaire ou d'une clé de chiffrement, ce volume de données numériques doit être soigneusement calibré pour garantir l'imperceptibilité et la sécurité du message transmis.

2.2.5.5 L'embedding payload

L'action d'*embedding payload* désigne simplement le fait d'injecter la charge utile (donc le *payload*) à l'intérieur du support d'origine (le *cover*). Cette opération donne naissance à l'objet modifié (le *stego*).

2.2.5.6 Spatial Domain

Le *Spatial Domain* (domaine spatial) fait référence à la manipulation directe des pixels de l'image. Travailler dans le domaine spatial signifie que l'algorithme stéganographique modifie directement les valeurs numériques de couleur ou de luminosité des pixels (comme la méthode du bit le moins significatif LSB) [32].

2.2.5.7 Frequency Domain

Le *Frequency Domain* (domaine fréquentiel) désigne la modification des coefficients mathématiques obtenus après une transformation de l'image (comme la transformée en cosinus discrète - DCT dans les fichiers JPEG). Les modifications y sont appliquées sur les fréquences de l'image plutôt que sur les pixels bruts, ce qui rend souvent le message plus robuste.

2.2.5.8 Feature Extraction

La *Feature Extraction* (extraction de caractéristiques) est le processus par lequel un algorithme de stéganalyse extrait les propriétés statistiques les plus pertinentes d'une image.

Ces descripteurs servent à mettre en évidence les anomalies causées par l'insertion d'un message.

2.2.5.9 Dimensionality

La *Dimensionality* [8] (dimensionnalité) fait référence à la taille et à la complexité du vecteur de caractéristiques extrait d'une image. En stéganalyse moderne et en *deep learning*, la manipulation d'espaces à haute dimensionnalité permet de capturer des relations très fines entre les pixels, améliorant ainsi la précision de la détection.

2.3 Approches existantes

2.3.1 Deux voies pour la stéganalyse

L'approche choisie pour détecter un message dépend principalement des connaissances préalables dont dispose le stéganalyste sur l'algorithme d'insertion. On distingue généralement deux voies principales.

2.3.1.1 La stéganalyse ciblée (Targeted steganalysis)

La stéganalyse ciblée intervient lorsque l'on connaît précisément l'algorithme ou l'outil de stéganographie qui a été utilisé pour dissimuler le message dans le support.

- **Objectif :** Détecter les traces laissées par une méthode de stéganographie bien spécifique.
- **Principe :** Le détecteur est conçu et entraîné en exploitant les faiblesses connues de cet algorithme particulier. Cela permet d'obtenir une excellente précision de détection pour ce cas précis, mais le système perd son efficacité face à une méthode d'insertion différente ou inconnue.

2.3.1.2 La stéganalyse aveugle (Blind steganalysis)

La stéganalyse aveugle correspond à la situation la plus fréquente et la plus complexe en pratique, où le stéganalyste intercepte un média suspect sans avoir aucune information sur la méthode stéganographique employée.

- **Objectif :** Détecter la présence d'un message caché sans connaître l'algorithme sous-jacent.
- **Principe :** Au lieu de chercher une signature spécifique, le système apprend à repérer des anomalies statistiques globales provoquées par l'action de dissimuler des données. Grâce aux approches modernes basées sur l'apprentissage profond (*deep learning*), cette méthode permet de créer des détecteurs polyvalents capables d'identifier un contenu secret, même face à de nouvelles techniques d'insertion.

2.3.2 Quelques techniques de stéganographie

Pour bien comprendre comment la stéganalyse détecte les secrets, il est nécessaire de connaître les méthodes utilisées pour les cacher. Dans l'état de l'art, les techniques se divisent principalement entre le domaine spatial (les pixels bruts) et le domaine fréquentiel (les coefficients compressés).

2.3.2.1 LSB (Least Significant Bit) [32]

La méthode du bit le moins significatif (*Least Significant Bit* ou LSB) s'est popularisée au début des années 1990 avec le développement de la stéganographie. C'est la technique la plus classique et la plus intuitive du domaine spatial.

- **Principe** : Elle consiste à remplacer le bit de poids faible des pixels d'une image par les bits du message secret. Par exemple, modifier la valeur d'un pixel de 255 à 254 change sa couleur de manière tellement infime que l'œil humain ne peut pas s'en rendre compte.
- **Limite** : Bien qu'elle soit très simple à mettre en œuvre, la méthode LSB basique détruit les propriétés statistiques de l'image. Comme le démontre l'analyse classique de Westfeld et Pfitzmann [34], ces modifications laissent des traces très visibles pour les outils de stéganalyse automatique.

2.3.2.2 WOW (Wavelet Obtained Weights)

WOW est une méthode moderne et adaptative du domaine spatial, qui fait partie de la famille des stéganographies de contenu adaptatif (*content-adaptive*).

- **Principe** : Contrairement au LSB qui modifie les pixels au hasard, WOW utilise des banques de filtres (ondelettes) pour analyser l'image et identifier les zones complexes, comme les textures riches ou les contours nets.
- **Intérêt** : Développé par Holub et Fridrich [16], WOW calcule un coût d'insertion pour chaque pixel. Il cache le secret uniquement là où le bruit visuel est déjà élevé, rendant la détection statistique extrêmement difficile pour le stéganalyste.

2.3.2.3 S-UNIWARD (Spatial Universal Wavelet Relative Distortion)

S-UNIWARD est actuellement l'un des algorithmes de stéganographie adaptative les plus puissants et les plus redoutables du domaine spatial.

- **Principe** : Tout comme WOW, il cherche à minimiser la distorsion globale causée par l'insertion du message. S-UNIWARD utilise une fonction de coût universelle basée sur des coefficients d'ondelettes directionnels pour évaluer les zones de l'image.

- **Intérêt** : Conçu par Holub et al. [17], cet algorithme excelle à cacher l'information dans les zones texturées et difficiles à modéliser mathématiquement. S-UNIWARD est la référence incontournable de l'état de l'art pour évaluer la robustesse des nouveaux réseaux de neurones en stéganalyse.

2.3.2.4 Gaussian Shading [36]

Gaussian Shading [36] est une technique moderne de tatouage numérique (*watermarking*), conçue spécifiquement pour les modèles de génération d'images par diffusion (tels que Stable Diffusion). Contrairement aux approches traditionnelles qui modifient l'image après sa création, *Gaussian Shading* intervient en amont, lors de la phase de distribution du bruit initial. En appliquant une mise en forme et une randomisation contrôlée sur les composantes de bruit gaussien, cette méthode permet d'enfouir une signature invisible et mathématiquement prouvable sans dégrader la qualité sémantique ou visuelle de l'image finale générée [37]. Du point de vue de la stéganalyse, l'analyse de ces motifs s'avère particulièrement complexe car la signature se fond parfaitement dans les propriétés statistiques naturelles du bruit latent.

2.3.2.5 La méthode mas-GRDH [18]

La méthode **mas-GRDH** [18] est une approche de stéganographie DM-GIS qui se distingue en cachant l'information non pas dans l'image finale, mais directement dans la dynamique du modèle de diffusion. Son principe repose sur l'injection du message secret au cœur du **bruit résiduel** prédit par l'U-Net à chaque étape de débruitage. En modifiant subtilement ce résidu, le message est lissé par le *scheduler* tout au long de la trajectoire de la PF-ODE. L'information cachée se fond ainsi naturellement dans les textures et les détails de l'image synthétisée. Cette stratégie permet à GRDH d'offrir une excellente imperceptibilité visuelle et de contourner efficacement les stéganalyseurs classiques.

2.3.3 Taxonomie de la stéganalyse

Dans cette partie, nous allons présenter la taxonomie de la stéganalyse, car cela permet de bien comprendre comment s'organise ce domaine et de voir clairement les différentes approches existantes selon le schéma suivant :

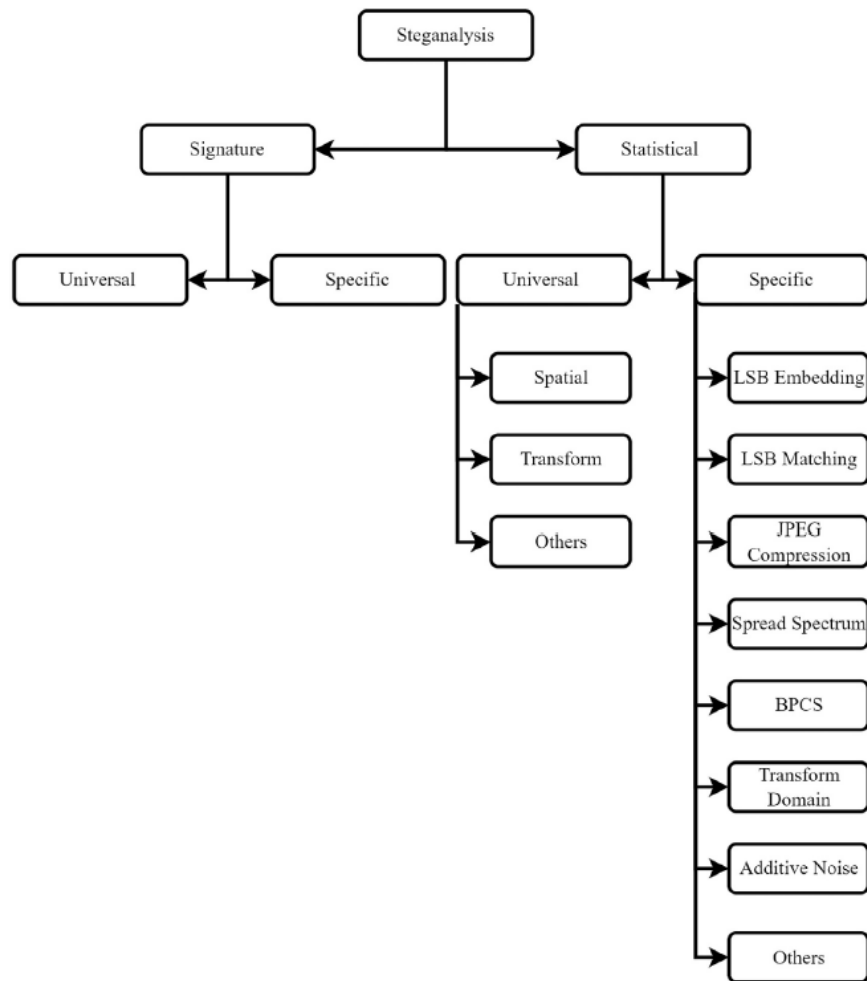


Figure 2.2 : Taxonomie de la stéganalyse d'images [8]

Comme le montre la Figure 2.2, la stéganalyse se sépare en deux grandes familles (basée sur les signatures ou statistique), qui se divisent chacune de manière symétrique en approches spécifiques (*Specific*) ou universelles (*Universal*).

2.3.3.1 La stéganalyse basée sur les signatures (Signature-based)

Cette approche cherche des empreintes fixes ou des anomalies de structure laissées par des logiciels de stéganographie connus. Elle se divise en deux voies :

- **Specific signature steganalysis :** Elle cible un outil précis en exploitant ses défauts structurels connus.
- **Universal signature steganalysis :** Elle cherche à détecter plusieurs outils à la fois.

2.3.3.2 La stéganalyse statistique (Statistical-based)

Cette approche analyse les perturbations mathématiques et les modifications de pixels ou de coefficients provoquées par l'insertion d'un secret. Elle se divise en deux voies :

- **Specific statistical steganalysis** : Elle utilise des algorithmes dédiés pour traquer les déformations statistiques uniques d'une méthode d'insertion bien précise.
- **Universal statistical steganalysis** : C'est l'approche "aveugle", aujourd'hui dominée par le *Deep Learning*. Sans rien connaître de l'algorithme utilisé, elle apprend à repérer n'importe quelle anomalie statistique ou bruit microscopique suspect dans l'image.

2.3.4 SPAM et SRM : Rich Models en Stéganalyse (méthodes classiques)

Avant l'arrivée de l'apprentissage profond et des réseaux de neurones, les modèles dits « riches » (*Rich Models*) représentaient les approches classiques les plus puissantes pour détecter les messages cachés. Ces méthodes reposent sur l'extraction manuelle de caractéristiques statistiques (*hand-crafted features*) calculées directement à partir des résidus de l'image.

On distingue principalement deux méthodes emblématiques :

- **SPAM (Subtractive Pixel Adjacency Matrix) [22]** : Cette méthode s'intéresse aux relations et aux dépendances entre les pixels voisins. Elle calcule les différences entre pixels adjacents pour isoler le bruit haute fréquence de l'image (le bruit d'insertion), puis utilise des matrices de transition (chaînes de Markov) pour modéliser statistiquement ces variations. Ce profil permet de voir si l'image a subi des modifications suspectes.
- **SRM (Spatial Rich Model) [12]** : SRM est l'évolution et la référence absolue des modèles riches pour le domaine spatial. Au lieu d'utiliser un seul type de filtre, SRM applique une très grande collection de filtres diversifiés (appelés sous-modèles) pour extraire une multitude de résidus de bruit différents.

C'est précisément avec des modèles comme SRM que survient le problème de dimensionnalité. En effet, les *features* explosent en dimension parce que ces modèles utilisent énormément de statistiques. Résultat : on peut obtenir entre 10 000 et 50 000 *features* [8], ce qui crée un problème de dimensionnalité majeur. Pour un humain ou un algorithme classique, c'est un volume énorme et très lourd à gérer.

2.3.5 Le changement de paradigme (Paradigm Shift)

C'est à partir de ce problème de dimensionnalité [8] qu'a eu lieu un véritable changement de paradigme (*paradigm shift*) dans le domaine de la stéganalyse. Face à la lourdeur des modèles riches classiques, les chercheurs ont arrêté d'essayer de concevoir manuellement des filtres statistiques de plus en plus complexes.

Ce changement s'est fait vers l'apprentissage profond (*Deep Learning*) et les réseaux de neurones convolutifs (CNN). Au lieu qu'un humain passe du temps à extraire des milliers de caractéristiques figées, on donne désormais l'image directement à un réseau de neurones. C'est la machine qui apprend toute seule, de bout en bout, à fabriquer ses propres filtres et à repérer les caractéristiques importantes pour détecter le secret, de manière beaucoup plus légère, automatique et performante. Ce changement de paradigme est bien illustré dans la figure 2.3

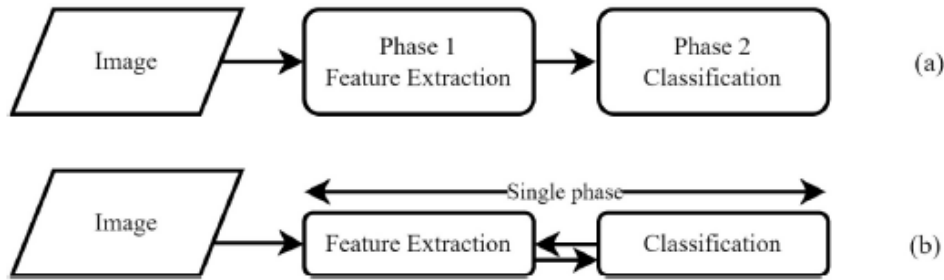


Figure 2.3 : Le changement de paradigme : de la stéganalyse classique à l'apprentissage profond [8]

- **A : L'approche traditionnelle (Pipeline classique)**
- **B : L'approche par Deep Learning**

2.3.6 Historique de la stéganalyse basé sur les CNN

Les approches basées sur les réseaux de neurones convolutionnels (CNN) en stéganalyse ont commencé à émerger en 2014 avec la proposition de SCAE [30], considéré comme le premier stéganalyseur intégrant une architecture CNN. Depuis, de nombreuses architectures ont été développées, chacune introduisant de nouvelles idées et apportant des améliorations progressives en termes de performances de détection. Au fil des années, ces travaux ont donné naissance à une grande diversité de stéganalyseurs profonds, illustrant l'évolution rapide du domaine. Les principaux stéganalyseurs basés sur les CNN sont présentés dans le tableau 2.1

Tableau 2.1 : Évolution historique et analyse technique des modèles CNN de stégalyse.

Modèle	Année & Réf.	Architecture	Apports et Particularités	Limites / Faiblesses
SCAE	2014 [30]	9 couches conv, max pooling, 1 FC.	Première tentative d'utilisation des CNN via des auto-encodeurs non supervisés.	Performances faibles, à peine au niveau du modèle SRM classique.
QianNet	2015 [23]	5 couches conv, Gaussienne, 3 FC.	Introduction d'une fonction d'activation Gaussienne pour capter les bruits faibles.	Dépend fortement d'un filtre passe-haut fixe et non optimisé au départ.
XuNet	2016 [35]	1 couche pré-trmt, 5 conv, 1 FC.	Intégration d'une couche Valeur Absolue (ABS) et de la <i>Batch Normalization</i> .	Sensible au <i>Cover-Source Mismatch</i> (changement de base d'images).
YeNet	2017 [38]	8 couches conv, 1 FC.	Utilisation de l'activation TLU pour ignorer les grands pixels et cibler le message.	L'extraction reste limitée par l'usage de filtres initiaux figés.
YedroudjNet	2018 [39]	30 filtres SRM, 5 conv, 3 FC.	Intègre les 30 filtres SRM au début pour bloquer l'image et n'analyser que le bruit.	Risque de surapprentissage à cause des 3 couches fully-connected (FC).
SRNet	2019 [2]	8 couches conv (sans pooling), 2 FC.	Premier modèle 100% autonome de bout en bout (<i>End-to-End</i>), sans filtre manuel.	Calcul très lourd, nécessite énormément d'images pour s'entraîner.
ZhuNet	2020 [42]	Architecture pyramidale.	Introduction de noyaux de débruitage apprenants qui remplacent les filtres fixes de SRM.	Spécialisé pour le domaine spatial, moins adapté au format JPEG.

FC : pour Fully Connected (couche entièrement connectée)

2.3.7 Architectures des modèles QianNet, XuNet et YeNet

Afin d'illustrer ces différents stéganalyseurs, il n'est pas possible de présenter en détail l'ensemble des architectures proposées dans la littérature en raison de leur grand nombre et de leur diversité. Nous nous limitons donc à trois architectures représentatives, présentées à titre illustratif afin de donner un aperçu de la structure générale des stéganalyseurs basés sur les CNN. Les architectures de QianNet [23], XuNet [35] et YeNet [38] sont respectivement représentées par les Figures 2.4, 2.5 et 2.6.

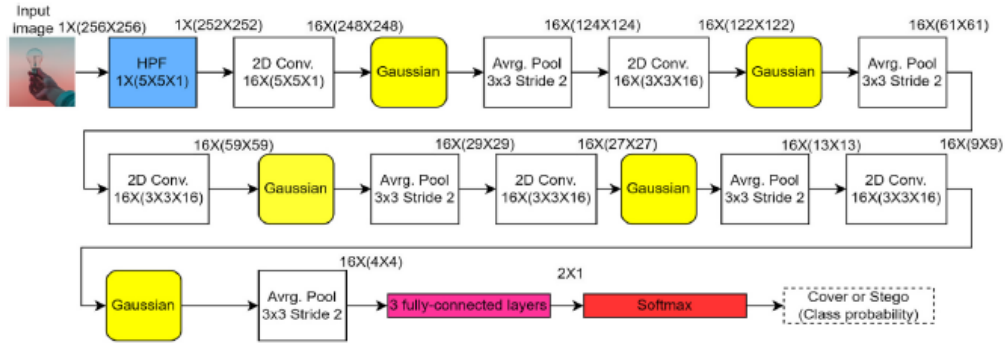


Figure 2.4 : Architecture globale du réseau QianNet [8]

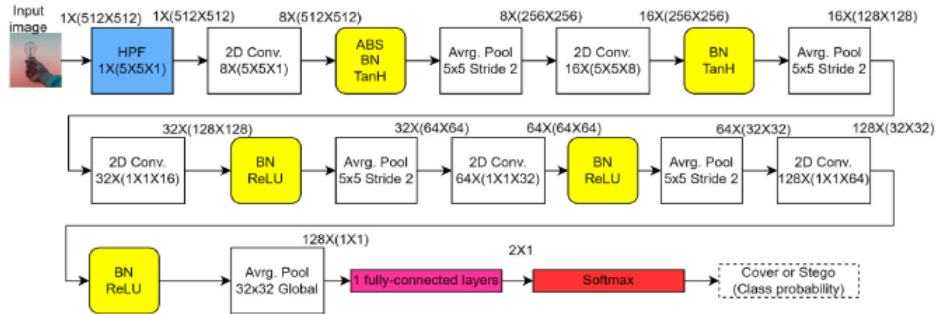


Figure 2.5 : Architecture globale du réseau XuNet [8]

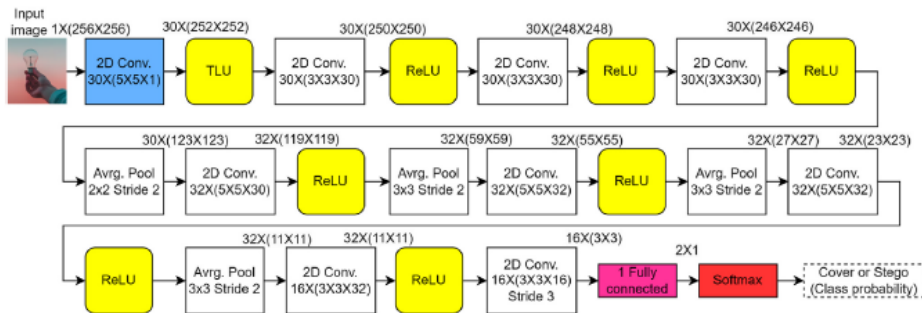


Figure 2.6 : Architecture globale du réseau YeNet [8]

2.3.8 La stéganalyse basée sur les GAN

Une évolution plus récente de la stéganalyse utilise les réseaux antagonistes génératifs, ou *GANs* (Generative Adversarial Networks). Un GAN [14] est un système composé de deux modèles de deep learning qui s'affrontent : un **générateur** qui crée de fausses images (ou des images modifiées), et un **discriminateur** qui apprend à faire la différence entre les vraies images et les fausses. En stéganalyse, cette architecture est détournée pour créer une sorte de jeu du chat et de la souris : le générateur s'entraîne à cacher des messages de la manière la plus invisible possible, tandis que le discriminateur devient un stéganalyste ultra-performant, forcé d'apprendre automatiquement à repérer les moindres anomalies générées par son adversaire.

2.4 NS-Dser

Contexte

- La stéganographie générative cache un message secret dans une image générée, s'affranchissant ainsi du besoin d'une image *cover* préexistante.
- Les méthodes les plus récentes et performantes de l'état de l'art s'appuient principalement sur les modèles de diffusion.
- Les messages secrets sont directement dissimulés au cœur de l'espace du bruit (*noise space*) du modèle génératif.

Questions posées :

1. **Comment les méthodes DM-GIS actuelles fonctionnent-elles ?** Où cachent-elles le message ?
2. **Ces méthodes sont-elles vraiment sécurisées** comme elles le prétendent ?
3. **Peut-on concevoir un stéganalyste** capable de les détecter ?

2.4.1 NS-DSer : Vers une Stéganalyse dans l'Espace de Bruit

Face à l'échec de la stéganalyse conventionnelle dans le domaine spatial (l'espace des pixels), le cadre *NS-DSer* [41] (*Noise Space-based Diffusion Steganalyzer*) propose un changement de paradigme en déplaçant le champ d'analyse directement vers l'espace de bruit latent.

Il s'agit d'une approche à la fois simple et puissante. Alors que les stéganalystes précédents opéraient dans l'espace des images, un environnement vaste, complexe et de très haute dimension, *NS-DSer* choisit de travailler au cœur même de l'espace du bruit, dont les propriétés statistiques fondamentales s'avèrent beaucoup plus simples et directes à modéliser pour la détection d'anomalies.

2.4.2 L'espace des distributions des images

Pour comprendre comment les ordinateurs analysent les images, on peut imaginer un immense espace virtuel, une sorte de carte géante, où chaque point représente une image ou un type de bruit. Dans cet espace, (voir la figure 2.7), les choses ne sont pas rangées au hasard : elles se regroupent par familles ou par *distributions*.

Une distribution, c'est simplement un groupe ou une catégorie d'objets qui se ressemblent et partagent les mêmes caractéristiques statistiques. Dans notre espace, il y a de la place pour toutes les familles possibles : la distribution des images de chats, la distribution des photos de paysages, mais surtout, quatre familles très importantes pour nous :

- P_c : La distribution de toutes les images saines (*Covers*).
- P_s : La distribution de toutes les images modifiées (*Stegos*).
- Q_c : La distribution des bruits naturels présents dans les images saines.
- Q_s : La distribution des bruits artificiels ajoutés par la stéganographie.

Chaque famille forme une sorte de "nuage" ou de zone sur notre carte. L'objectif pour un stéganalysateur est de réussir à voir si une image appartient au nuage P_c ou au nuage P_s .

Concrètement, lorsqu'un utilisateur prend une image saine et y cache un message secret, l'image change de nature. Ses statistiques changent. Visuellement, cela signifie que l'image voyage : elle quitte sa famille d'origine pour entrer dans une autre. En clair, dès qu'une image devient porteuse d'un secret, elle passe automatiquement de la distribution P_c à la distribution P_s .

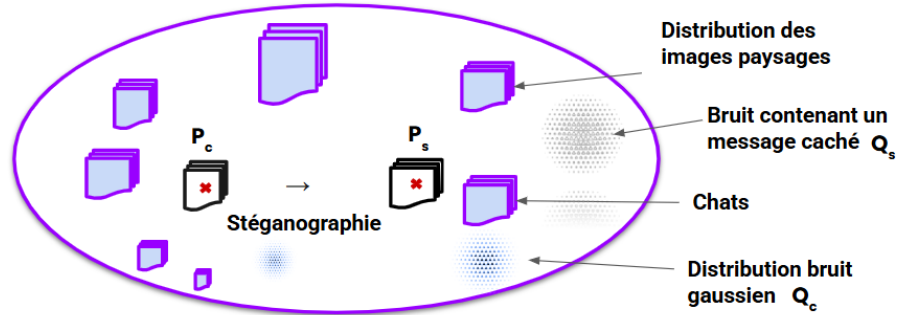


Figure 2.7 : Représentation schématique des différentes distributions (P_c , P_s , Q_c , Q_s) dans l'espace des images et déplacement d'une image de P_c vers P_s .

2.4.3 L'équivalence des divergences : le fondement de NS-DSer

Pour expliquer pourquoi il est plus intelligent de chercher le message caché dans le bruit plutôt que dans l'image, l'article NS-DSer [41] utilise un outil de mesure mathématique appelé la **divergence de Kullback-Leibler** [19] (ou divergence KL).

2.4.3.1 Qu'est-ce que la divergence KL ?

En mathématiques, la divergence KL [19] permet de mesurer l'écart ou la différence entre deux groupes de données (deux distributions).

- Si l'écart vaut **0**, cela signifie que les deux groupes sont rigoureusement identiques.
- Plus cet écart est **grand**, plus il est facile pour un ordinateur de faire la différence entre les deux groupes.

Dans notre cas, la divergence KL mesure la distance entre le groupe des images saines (P_c) et le groupe des images contenant un secret (P_s).

2.4.3.2 L'égalité prouvée par NS-DSer

La grande découverte de l'article *NS-DSer* est de prouver mathématiquement que, dans un modèle de diffusion, l'écart ne change pas durant la création de l'image. Il existe une égalité parfaite :

$$D_{KL}(P_c \parallel P_s) = D_{KL}(Q_c \parallel Q_s) \quad (2.1)$$

Où :

- $D_{KL}(P_c \parallel P_s)$ est l'écart entre les images saines et les images stégo.
- $D_{KL}(Q_c \parallel Q_s)$ est l'écart entre le bruit sain et le bruit stégo.

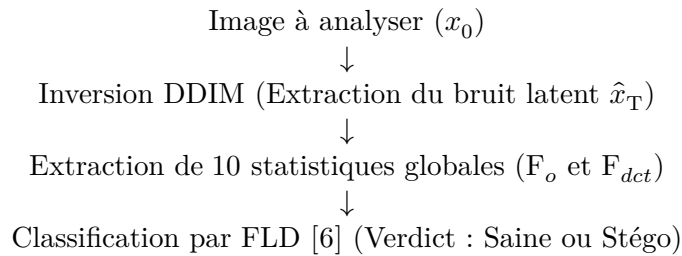
Pourquoi c'est une idée puissante ?

Cette égalité signifie que la quantité de traces laissée par le message caché est **exactement la même** dans l'image finale et dans le bruit d'origine. Les modifications ne se perdent pas en chemin.

Puisque l'écart à détecter est strictement égal des deux côtés, il est beaucoup plus facile pour notre stéganalysateur de travailler directement dans l'espace du bruit ($Q_c \parallel Q_s$). En effet, l'espace des images est un espace de très haute dimension rempli de détails visuels (des formes, des couleurs, des textures) qui perturbent les calculs. Le bruit, lui, est mathématiquement très simple, propre et standard. En allant fouiller dans le bruit, *NS-DSer* efface tous les détails inutiles de l'image pour se concentrer uniquement sur la fraude.

2.4.4 Le pipeline de l'architecture NS-DSer

Le framework *NS-DSer* cherche des indices non pas sur l'image elle-même, mais dans l'histoire de sa création (l'espace du bruit). Son processus global suit le cheminement suivant :



2.4.4.1 Étape 1 : L'extraction du bruit par Inversion DDIM

La toute première étape consiste à prendre l'image suspecte reçue, que l'on appelle x_0 , et à essayer de retrouver le bruit d'origine qui a servi à la fabriquer. Pour faire cela sans perdre d'information, *NS-DSer* utilise un mécanisme mathématique appelé l'inversion DDIM.

Contrairement aux méthodes classiques qui essaient de deviner le bruit à tâtons, cette inversion fait remonter le temps au modèle de diffusion de manière totalement déterministe (étape par étape, à l'envers). À la fin de ce voyage retour, on obtient un vecteur appelé le bruit latent extrait, noté $\hat{x}_T$.

2.4.4.2 Étape 2 : Le calcul des statistiques

Une fois que le bruit latent $\hat{x}_T$ est récupéré, il faut l'analyser pour voir s'il cache des anomalies statistiques. Si l'image est saine, ce bruit doit ressembler à une distribution gaussienne parfaite, très propre. Si un message a été caché, des petites déformations invisibles à l'œil nu vont perturber cette régularité.

Pour mettre en lumière ces changements, le pipeline calcule des statistiques globales sur le bruit extrait. Ceci permet de résumer le comportement du bruit et de préparer le terrain pour le diagnostic final.

2.4.4.3 Étape 3 : La classification

La dernière étape est celle de la décision. Les statistiques calculées sont données en entrée à un classifieur.

Puisque tout le "bruit visuel" et la complexité des textures de l'image originale ont été éliminés lors de la première étape, ce classifieur n'a pas besoin d'être un modèle gigantesque ou lourd. Il lui suffit d'observer les anomalies dans les statistiques du bruit pour rendre son verdict : l'image analysée est soit classée comme **Saine** (Cover), soit comme **Modifiée** (Stego).

2.4.5 Le vecteur de statistiques globales et le FLD

Pour détecter la fraude, *NS-DSer* déploie une stratégie statistique très intelligente. Au lieu de regarder de petites zones locales, il aplatit (*flatten*) complètement le bloc de bruit extrait pour l'analyser de manière globale à travers deux domaines différents.

2.4.5.1 Les statistiques extraites

Dans le domaine d'origine du bruit, le modèle extrait un premier vecteur de caractéristiques noté $F_o = [\mu, \sigma^2, \gamma_1, \gamma_2, \text{IQR}]$. Ces 5 indicateurs mathématiques permettent de vérifier si le bruit respecte parfaitement la loi gaussienne :

- **La moyenne (μ)** : Elle indique le centre de la distribution. Pour un bruit gaussien pur, elle doit être quasiment égale à 0.
- **La variance (σ^2)** : Elle mesure l'étalement global des valeurs autour de la moyenne.

- **La dissymétrie / *Skewness* (γ_1)** : Elle mesure la symétrie de la cloche. Si $\gamma_1 = 0$, la cloche est parfaitement équilibrée. Une valeur différente montre que le message caché a penché la distribution d'un côté.
- **L'aplatissement / *Kurtosis* (γ_2)** : Il indique si les valeurs extrêmes (les anomalies) sont plus ou moins fréquentes que dans une vraie gaussienne. Une modification stéganographique va souvent créer des pics anormaux.
- **L'écart interquartile / *IQR* ($IQR = Q_3 - Q_1$)** : Il mesure la dispersion des 50 % des valeurs centrales. Dans une distribution gaussienne standardisée, on doit avoir un $IQR \approx 1,349$. Si cette valeur change, c'est le signe qu'un message perturbe le bruit.

2.4.5.2 Les statistiques extraites dans le domaine transformé (DCT)

Comme certaines méthodes DM-GIS préfèrent dissimuler leurs messages dans les fréquences du bruit plutôt que dans ses valeurs directes, *NS-DSer* applique une **DCT** (*Discrete Cosine Transform* ou Transformée en Cosinus Discrète) sur le bruit.

Une fois le bruit projeté dans ce domaine fréquentiel, le modèle extrait exactement le même ensemble de 5 statistiques : $F_{dct} = [\mu, \sigma^2, \gamma_1, \gamma_2, IQR]$. Cela permet de piéger les algorithmes stéganographiques, peu importe leur stratégie d'insertion.

2.4.5.3 Comment le FLD [6] utilise ce vecteur de 10 caractéristiques

Au total, en fusionnant le domaine original et le domaine DCT, chaque image est résumée par un vecteur de 10 caractéristiques ($F_o + F_{dct}$).

C'est ici que le **FLD** (*Fisher's Linear Discriminant*) [6] intervient. Au lieu de travailler dans l'espace géant de l'image, il travaille dans cet espace simplifié à seulement 10 dimensions. Le FLD va chercher l'axe mathématique idéal qui permet de séparer le groupe des vecteurs **Cover** (où $\mu \approx 0$ et $IQR \approx 1,349$, etc.) du groupe des vecteurs **Stego** (qui présentent des anomalies sur ces 10 indicateurs). En maximisant l'écart entre ces deux groupes, le FLD trace une frontière de décision (voir la figure 2.8) ultra-précise pour classer l'image.

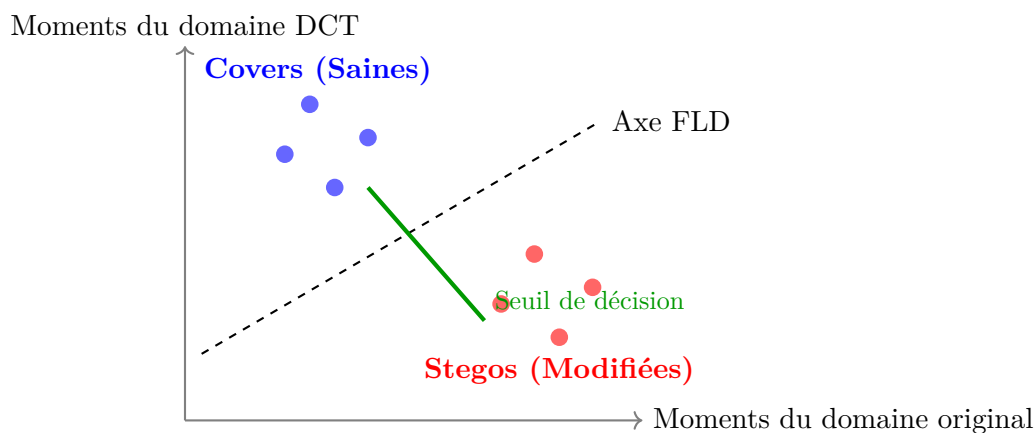


Figure 2.8 : Représentation simplifiée du FLD : Les statistiques des images se séparent en deux groupes distincts de part et d'autre du seuil de décision.

2.4.6 Mécanismes d'insertion des méthodes DM-GIS

Les méthodes de stéganographie par génération d'images basées sur les modèles de diffusion (*DM-GIS*) reposent sur la manipulation du processus de débruitage déterministe (souvent via un modèle de type *DDIM*). L'analyse de l'état de l'art permet de distinguer deux paradigmes majeurs pour dissimuler un message secret au sein de l'espace latent, comme illustré dans la Figure 2.9 :

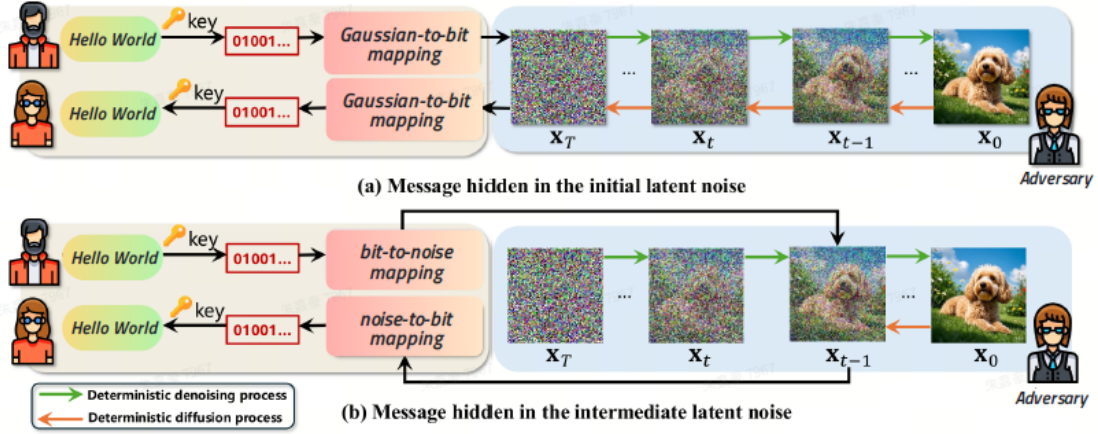


Figure 2.9 : Deux approches d'insertion de messages dans les frameworks DM-GIS : (a) dissimulation dans le bruit initial x_T , et (b) dissimulation au sein d'un état latent intermédiaire x_t .

1. **Insertion dans le bruit latent initial (x_T)** : Dans ce premier scénario (Figure 2.9a), les bits du message secret sont convertis en une distribution gaussienne à l'aide d'un algorithme de correspondance (*Gaussian-to-bit mapping*) et d'une clé secrète. Ce message modifié constitue directement le bruit blanc gaussien initial x_T . Le modèle applique ensuite son processus de débruitage déterministe pour générer l'image finale x_0 . Lors de l'extraction, le destinataire applique le processus inverse de diffusion déterministe pour remonter de x_0 jusqu'à x_T et décoder le message.
2. **Insertion dans le bruit latent intermédiaire (x_t)** : Dans ce second scénario (Figure 2.9b), le processus commence à partir d'un bruit initial standard. L'insertion du message s'effectue à une étape temporelle intermédiaire spécifique t au cours du processus de débruitage déterministe. Les bits du message sont convertis et projetés (*bit-to-noise mapping*) pour modifier l'état latent intermédiaire x_t . Le débruitage se poursuit ensuite normalement jusqu'à x_0 . À la réception, l'extraction nécessite de recalculer le chemin déterministe à l'envers depuis x_0 jusqu'à l'étape t cible (*noise-to-bit mapping*) afin de récupérer les données secrètes.

2.4.7 Le compromis entre sécurité et précision d'extraction

Dans cette partie, les auteurs de l'article mettent en lumière une règle fondamentale de la stéganographie générative : il est impossible d'avoir un système parfait sur tous les plans. Chaque réglage est un compromis.

Concrètement, les auteurs démontrent le mécanisme suivant :

- Si l'on cherche à ce que le destinataire reçoive le message avec une **précision d'extraction maximale** (sans aucune erreur de bit) ;
- On est obligé de forcer le trait et de **modifier plus fortement la distribution du bruit** d'origine.
- Par conséquent, ces modifications laissent des traces plus visibles, ce qui **augmente la détectabilité** de la fraude par les stéganalysateurs.

2.4.7.1 La limite théorique des méthodes DM-GIS

De ce fait, un modèle DM-GIS fait face à un triangle de contraintes et ne peut pas obtenir simultanément ces trois critères :

1. Une **sécurité maximale** (le message est totalement indétectable).
2. Une **extraction parfaite** (le message est retrouvé sans aucune erreur).
3. Une **haute qualité d'image** (l'image générée reste belle et naturelle).

Il existe une véritable limite théorique : si l'on améliore l'un de ces aspects, on dégrade forcément l'un des deux autres. Le concepteur d'un outil de stéganographie doit donc choisir le curseur idéal en fonction de ses priorités.

2.4.8 Évaluation expérimentale : Scénarios, résultats et limites de NS-DSer

Pour valider l'efficacité de *NS-DSer*, les performances du modèle ont été évaluées à travers quatre scénarios de difficulté croissante, face aux meilleurs stéganalysateurs actuels.

2.4.8.1 Les quatre scénarios de test

Les tests ont été structurés pour simuler des situations réelles de plus en plus complexes :

- **Scénario 1 (Le plus simple)** : Toutes les images proviennent du même modèle d'intelligence artificielle. Les images *cover* et les images *stego* sont à 100 % générées.
- **Scénario 2** : Les images *cover* ne sont plus seulement des images générées ; elles incluent désormais aussi des images naturelles (de vraies photos réelles).
- **Scénario 3** : En plus des conditions du scénario 2, les images *stego* et les images *cover* générées proviennent de modèles de diffusion différents avec des paramètres variés. C'est le scénario des données hétérogènes.
- **Scénario 4 (Le plus difficile)** : C'est le scénario du monde réel où les attaquants peuvent utiliser différentes méthodes DM-GIS pour cacher leurs messages, en utilisant des modèles et des paramètres variés.

2.4.8.2 Analyse des résultats obtenus

Face aux outils traditionnels travaillant sur l'image (XuNet, SRNet, SiaStegNet, UCNet), *NS-DSer* montre une supériorité globale très nette sur l'ensemble des tests grâce à son analyse dans l'espace du bruit :

- **Scénarios 1 à 3 :** Pour la majorité des méthodes de stéganographie qui modifient le bruit (comme *MC*, *MB*, *LDStega*, *CRoSS*), *NS-DSer* obtient des scores de détection exceptionnels, frôlant très souvent la perfection entre **96 % et 99,99 %**. Là où les autres stéganalyses perdent en efficacité à cause de l'hétérogénéité des données (scénarios 2 et 3), *NS-DSer* reste stable et robuste.
- **Scénario 4 :** Dans ce cas de figure, *NS-DSer* surclasse nettement tous ses concurrents en atteignant une précision globale de **86,26 %**, contre seulement 74,80 % pour SRNet et 57,96 % pour XuNet (comme résumé dans le Tableau 2.2).

Tableau 2.2 : Précision de détection globale (%) des stéganalyses sous le Scénario #4.

Stéganalyste	XuNet	SRNet	SiaStegNet	UCNet	NS-DSer (Ours)
Précision (%)	57,96	74,80	75,80	73,00	86,26

2.4.8.3 La limite de NS-DSer : La préservation de distribution

Les résultats mettent également en lumière une limite théorique et expérimentale majeure du modèle. Les méthodes dites *distribution-preserving* (comme *MN*, *G-S*, *StegaDDPM* et *mas-GRDH*) cachent le message secret sans altérer la distribution gaussienne du bruit d'origine.

Face à ces méthodes spécifiques, les performances de *NS-DSer* s'effondrent pour tomber aux alentours de **50 % de précision**, équivalent au hasard (voir le tableau 2.10). Tous les autres stéganalyses de l'état de l'art subissent exactement le même échec.

Ce résultat est d'une importance capitale : il valide expérimentalement le fait que **si une méthode stéganographique ne perturbe pas la distribution du bruit, elle reste mathématiquement indétectable**, même pour un outil spécialisé dans l'espace latent comme *NS-DSer*.

<i>Scenario #1</i>	MN	MC	MB	G-S	StegaDDPM	LDStega	GSD	mas-GRDH	CRoSS	Overall
XuNet	49.17	52.25	58.86	51.38	50.67	89.41	50.79	50.88	83.63	0.8255
SRNet	49.71	80.95	97.29	50.21	51.92	98.75	49.25	51.88	88.40	0.9039
#1 SiaStegNet	50.33	91.52	98.71	50.75	50.96	99.90	50.38	50.13	93.20	0.9283
UCNet	49.67	92.90	99.60	51.50	50.54	99.95	49.67	50.70	93.65	0.9281
NS-DSer (ours)	52.21	99.78	99.99	50.95	50.05	99.99	99.58	51.13	85.70	0.9742
XuNet	59.68	61.65	70.21	60.18	60.35	91.25	60.21	60.19	81.13	0.7635
SRNet	70.28	86.95	97.90	70.80	70.25	99.96	70.25	70.00	87.50	0.7392
#2 SiaStegNet	74.10	91.80	98.55	74.20	73.95	99.95	73.00	74.00	91.45	0.7141
UCNet	73.10	90.55	99.15	74.05	73.50	99.90	73.80	74.10	93.50	0.7200
NS-DSer (ours)	62.42	99.73	99.99	62.27	62.83	99.99	99.60	62.98	90.28	0.8633
XuNet	61.14	62.94	61.96	58.50	60.19	89.73	59.39	59.68	72.40	0.7476
SRNet	69.05	76.60	92.80	69.06	69.30	99.55	69.00	69.20	77.90	0.7243
#3 SiaStegNet	73.75	83.40	94.40	73.10	73.95	99.90	73.50	73.00	77.40	0.6941
UCNet	73.85	86.35	96.20	74.35	74.15	99.80	74.65	74.45	79.85	0.6949
NS-DSer (ours)	59.50	98.18	99.05	59.45	59.57	99.99	96.15	59.53	83.48	0.8817

Figure 2.10 : Résultats de détection (%) et évaluation globale du stéganalysateur NS-DSer par rapport à l'état de l'art à travers les différents scénarios.

2.5 Conclusion

Pour résumer tout ce que nous venons de voir, l'analyse du framework *NS-DSer* nous a permis de comprendre un élément fondamental : la stéganalyse moderne ne doit plus se focaliser uniquement sur les pixels visibles de l'image. En déplaçant l'enquête directement dans l'espace du bruit latent, nous arrivons à capturer des anomalies statistiques qui étaient auparavant totalement invisibles pour les outils classiques.

Les tests sur les quatre scénarios ont prouvé la puissance de cette approche, mais ils ont aussi révélé une limite importante : si un algorithme cache son message sans déformer la cloche de Gauss du bruit (méthodes *distribution-preserving*), le système devient inefficace et les chances de détection tombent à une sur deux (hasard total).

C'est précisément à partir de ce constat et de ces limites théoriques que se dessine la suite de notre travail. Les connaissances acquises à travers cette étude de l'état de l'art posent les bases nécessaires pour concevoir nos propres outils.

La première partie de ce mémoire, dédiée à l'étude théorique et à l'analyse de l'existant, est à présent terminée. Nous ouvrons maintenant la seconde grande partie de notre manuscrit, intitulée « **Contribution** ».

Dans cette nouvelle partie, nous allons passer de la théorie à la pratique. Nous aborderons dans un premier temps la conception générale puis l'implémentation concrète de notre propre solution logicielle. Enfin, nous terminerons par une présentation et une discussion détaillée des résultats obtenus lors de nos expérimentations, afin de mesurer l'efficacité réelle de notre apport face aux défis actuels de la stéganographie.

Deuxième partie

Contributions

CHAPITRE 3

APPROCHE PROPOSÉE

3.1 Introduction

Dans ce chapitre, nous présentons l’approche adoptée afin d’étudier et d’améliorer la détection des images générées par stéganographie basée sur les modèles de diffusion. Notre démarche repose sur deux étapes principales. Dans un premier temps, nous réimplémentons la méthode de référence NS-DSer [41] de la manière la plus fidèle possible à l’article original, afin d’obtenir une base expérimentale solide. Dans un second temps, nous proposons plusieurs extensions visant à renforcer cette baseline, notamment par l’ajout de nouvelles caractéristiques statistiques, l’exploration de différents classifieurs et la construction de modèles apprenant explicitement la normalité du bruit latent associé aux images *cover*.

3.2 La baseline

La base de notre démarche s’appuie sur une réimplémentation du stéganalyseur de référence *Noise Space-based Diffusion Steganalyzer* (NS-DSer). Avant d’envisager la moindre modification structurelle ou d’introduire d’autres architectures, notre premier objectif a été de reproduire fidèlement l’approche décrite dans le papier d’origine [41].

Notre approche suit ce même principe. Pour chaque image, nous effectuons les étapes suivantes :

1. *Encoder* l’image dans l’espace latent de Stable Diffusion ;
2. *Inverser* le processus de diffusion afin d’obtenir une approximation du bruit latent initial ;
3. *Extraire* des caractéristiques statistiques à partir de ce bruit ;
4. *Entraîner* un classifieur afin de distinguer les images *cover* des images *stego* ;

5. *Enrichir* progressivement cette représentation avec de nouvelles couches de caractéristiques.

Ce référer à la figure 3.1 qui représente une vue globale de notre approche pour bien visualiser ces étapes.

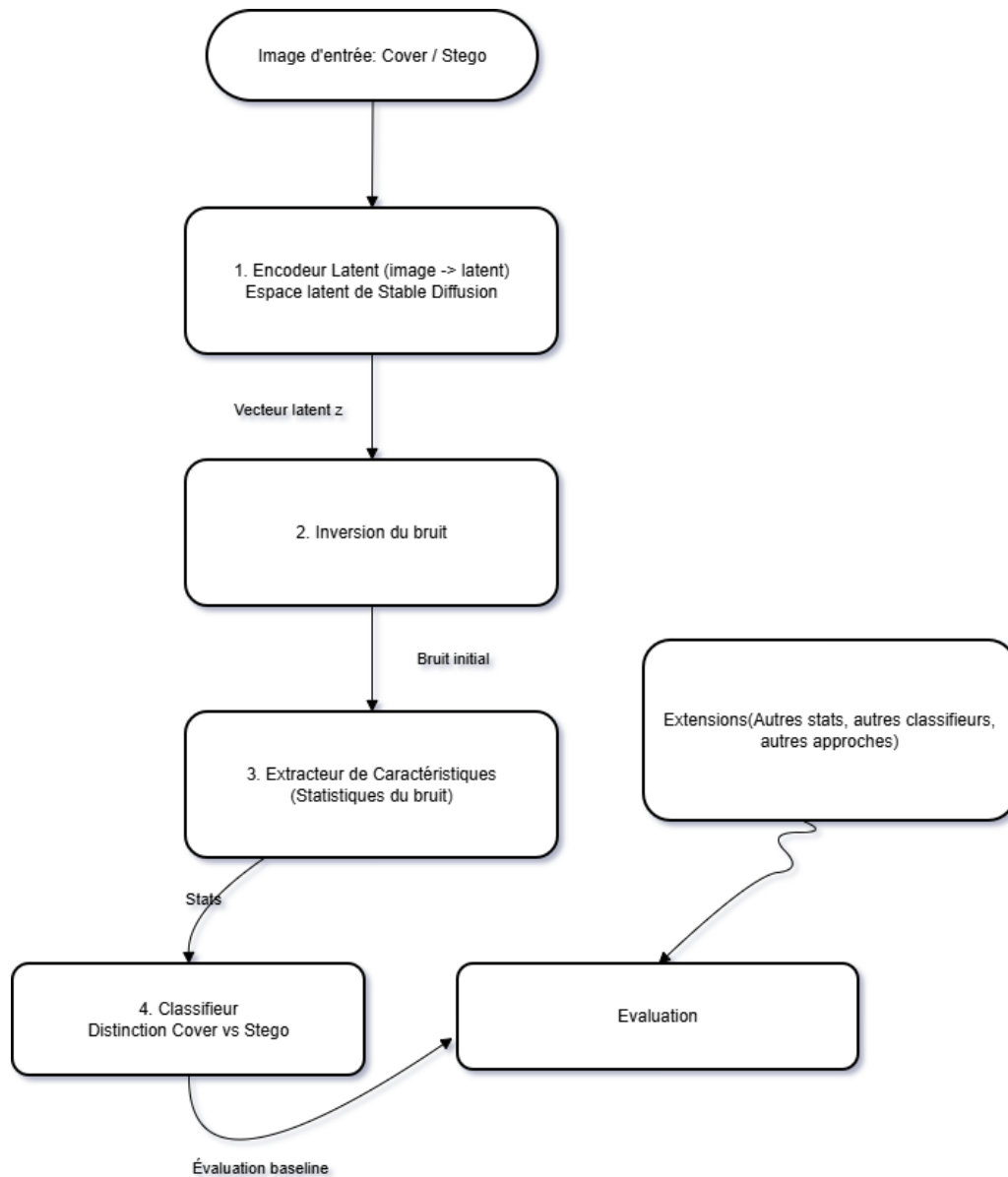


Figure 3.1 : Diagramme de l'approche : de l'encodage de l'image à l'enrichissement du classifieur.

Cette organisation permet d'avoir une *baseline* conforme à l'article de NS-DSer [41], puis de mesurer précisément l'apport de chaque extension proposée. Disposer de cette reproduction fidèle nous permet d'atteindre deux objectifs critiques :

- **Valider notre environnement** : S’assurer que nos scripts de génération, d’inversion et d’évaluation sont cohérents avec l’état de l’art.
- **Établir une base de comparaison fiable** : Disposer d’une véritable ligne de base (*baseline*), à partir de laquelle chaque exploration ou modification architecturale pourra être mesurée avec précision.

3.3 Défis de l’approche expérimentale

L’évaluation et l’amélioration des performances de détection face à la stéganographie générative soulèvent des contraintes techniques et méthodologiques majeures. Ce domaine de recherche, encore émergent, se heurte à des barrières d’infrastructure et de données qui ont fortement conditionné notre protocole expérimental.

3.3.1 L’émergence d’un domaine nouveau et l’absence de datasets de référence

Le premier verrou réside dans la nouveauté même du domaine DM-GIS (*Diffusion Model-based Generative Image Steganography*). Contrairement à la stéganalyse spatiale ou fréquentielle classique, qui dispose de bases de données standardisées et éprouvées de longue date (telles que BOSSBase ou BOWS2), il n’existe pas à ce jour de jeu de données standardisé et universellement adopté pour la stéganalyse dans l’espace de bruit des modèles de diffusion. Cette absence de référence oblige chaque chercheur à concevoir et à valider sa propre infrastructure de données, ce qui complexifie la reproductibilité immédiate et l’évaluation comparative directe.

3.3.2 La double surcharge computationnelle : génération et inversion

La nécessité de fabriquer notre propre *dataset* engendre un coût de calcul extrêmement lourd, structuré autour de deux étapes massives et consécutives :

1. **La génération initiale** : Pour constituer une classe d’images saines (*cover*) et une classe d’images modifiées (*stego*), il est indispensable de solliciter des modèles génératifs de grande envergure, dans notre cas *Stable Diffusion*. Ce processus requiert des dizaines de pas de débruitage (*sampling steps*) par image synthétisée, ce qui représente une charge de calcul phénoménale pour générer un volume statistiquement représentatif (plusieurs milliers d’images).
2. **L’inversion DDIM** : Une fois les données générées, la phase de stéganalyse (qu’il s’agisse de la reproduction de la baseline NS-Der ou de nos propres architectures) nécessite de remonter de l’image pixélique vers son bruit latent d’origine. Cette opération d’inversion DDIM équivaut à un second passage complet à travers le réseau U-Net pour chaque image du dataset. Nous faisons donc face à un double coût computationnel critique (Génération → Inversion).

3.4 La génération des données et constitution du dataset

Pour valider notre approche de manière rigoureuse, la constitution d'un jeu de données adéquat représente la première étape de notre méthodologie. Face à l'absence de bases de données standardisées pour la stéganalyse dans les modèles de diffusion, nous décrivons ici comment nous avons généré ces images.

Le processus repose sur l'utilisation d'un modèle de diffusion de grande envergure (tel que *Stable Diffusion*) alimenté par un ensemble de descriptions textuelles (*prompts*). Pour chaque concept sémantique généré, le pipeline se sépare en deux branches distinctes afin de former nos deux classes d'étude :

- **La classe saine (*Cover*)** : Elle est constituée des images synthétisées directement par le modèle de diffusion standard, représentant la distribution naturelle des images générées sans altération.
- **La classe stéganographiée (*Stego*)** : Elle est obtenue en appliquant une méthode d'insertion de l'état de l'art (par exemple, des altérations dans l'espace latent ou des techniques de type *Gaussian Shading*) au cours du processus de génération ou de post-génération immédiate. Le message secret est ainsi intimement lié au processus de synthèse de l'image.

L'approche méthodologique impose que les deux sous-ensembles partagent exactement les mêmes configurations de départ (mêmes seeds, mêmes *prompts*, mêmes résolutions et mêmes nombres de pas de débruitage) tel que le montre la figure 3.2. Cette stricte symétrie est indispensable pour garantir que le stéganalyste apprenne à détecter uniquement la signature du message dissimulé et non des biais sémantiques ou des artefacts liés à la génération textuelle.

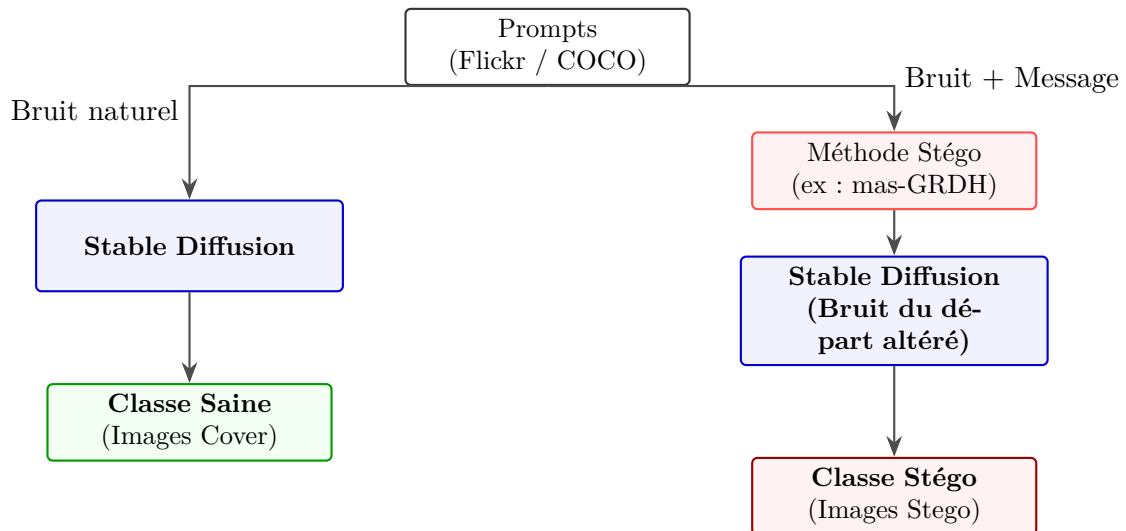


Figure 3.2 : Pipeline de génération du jeu de données pour les classes Cover et Stego.

3.4.1 Choix des méthodes de stéganographie

Dans ce travail, nous générons des jeux de données basés sur **trois techniques de stéganographie**, chacune répondant à un objectif bien précis :

- **Gaussian Shading et mas-GRDH** : L'objectif est d'améliorer les résultats du stéganalysesur NS-DSer, car celui-ci n'arrive pas à détecter les méthodes qui ne perturbent pas la distribution gaussienne du bruit.
- **La méthode MC** : L'objectif de cette troisième technique est distinct, puisqu'elle sert uniquement à valider notre *baseline*.

3.5 Première Extension - Extraction des caractéristiques internes de U-Net

Comme souligné précédemment, le bruit latent brut possède des propriétés proches d'une distribution gaussienne blanche, ce qui rend l'extraction de motifs spatiaux par des convolutions classiques inefficace. Pour contourner ce verrou, notre approche consiste à ne pas se limiter à l'espace de bruit final, mais à intercepter les flux de données internes du réseau U-Net lors de la phase d'inversion (voir la figure 3.3).

En plaçant des points de capture (*hooks*) sur des couches stratégiques du U-Net (notamment dans les blocs de convolution intermédiaires ou les couches d'attention), nous extrayons des cartes de caractéristiques (*feature maps*) de haute dimensionnalité. Ces activations internes contiennent à la fois l'information structurale de l'image en cours de reconstruction et les micro-distorsions induites par la présence du message secret.

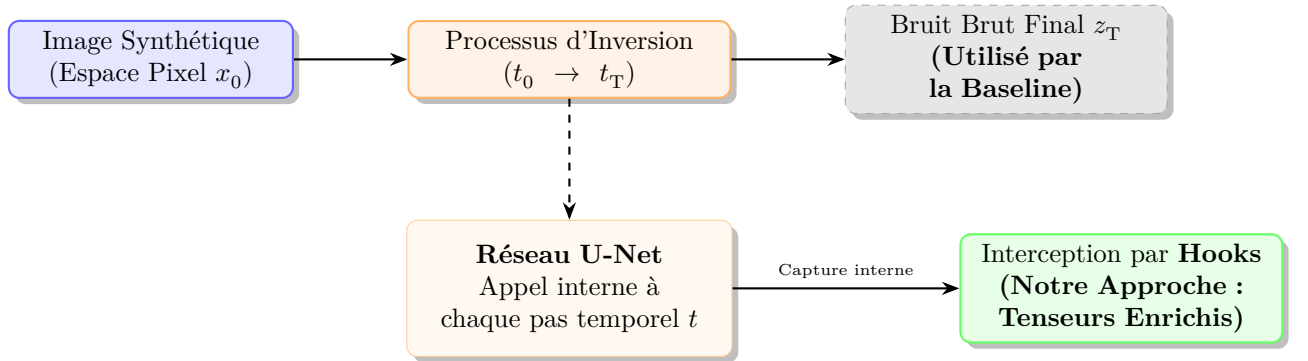


Figure 3.3 : Mécanisme d'extraction au cours de l'inversion : interception des activations internes du U-Net par Hooks.

3.5.1 Architecture globale de cette approche

Dans le cadre de cette étude, nous explorons une architecture hybride qui combine trois sources d'informations distinctes. L'objectif est d'évaluer si l'apport de caractéristiques internes ou spatiales peut compléter l'analyse du stéganalysesur *NS-DSer*. Le schéma global de cette structure est détaillé dans la figure 3.4.

3.5.1.1 Les trois modules d'extraction de caractéristiques

Notre réseau prend en entrée trois types de données complémentaires, traités en parallèle par trois blocs spécifiques :

- **NoiseCNN (Entraîné)** : Un réseau de neurones convolutif composé de trois blocs de convolution principaux (*Conv2d_Block*). Ce module traite le bruit latent brut pour extraire d'éventuels motifs spatiaux, avant de réduire la dimensionnalité via un *Flatten* et une couche linéaire pour obtenir un vecteur de taille 128.
- **UNetHooks (Gelé)** : Ce bloc intercepte les cartes de caractéristiques internes de U-Net pendant l'inversion à l'aide de trois points de capture (*Hook_Down*, *Hook_Mid*, et *Hook_Up*). Ces activations passent par un pooling adaptatif et des couches linéaires avant d'être concaténées pour former un vecteur de taille 192. Comme le modèle Stable Diffusion n'est pas réentraîné, ce module reste gelé.
- **StatsNS-DSer** : Ce bloc récupère directement les 10 caractéristiques statistiques habituellement calculées par *NS-DSer*. Elles sont passées dans une couche linéaire suivie d'une activation ReLU et d'un Dropout pour fournir un vecteur projeté de taille 32.

3.5.1.2 Fusion des caractéristiques

Une fois les traitements parallèles terminés, les vecteurs issus de chaque branche sont assemblés au niveau du module **Fusion**. L'opération consiste en une concaténation simple des trois dimensions précédentes ($128 + 192 + 32$), ce qui permet de regrouper au sein d'un unique vecteur de taille 352 l'ensemble des descripteurs statistiques, spatiaux et internes du modèle de diffusion.

3.5.1.3 Classification par approche neuronale (FinalMLP)

Contrairement à *NS-DSer* qui s'appuie sur un classifieur linéaire par analyse discriminante de Fisher (FLD), nous avons choisi de tester une approche entièrement neuronale pour la classification.

Le vecteur fusionné est ainsi envoyé dans un Perceptron Multicouche nommé **FinalMLP**. Ce réseau enchaîne trois couches d'interconnexion linéaires successives (réduisant la dimension de 352 à 128, puis à 32, et enfin à 1), pour se terminer par une fonction d'activation *Sigmoid*. Ce choix permet de voir si le modèle peut apprendre des relations non linéaires pour distinguer la classe *cover* de la classe *stego*.

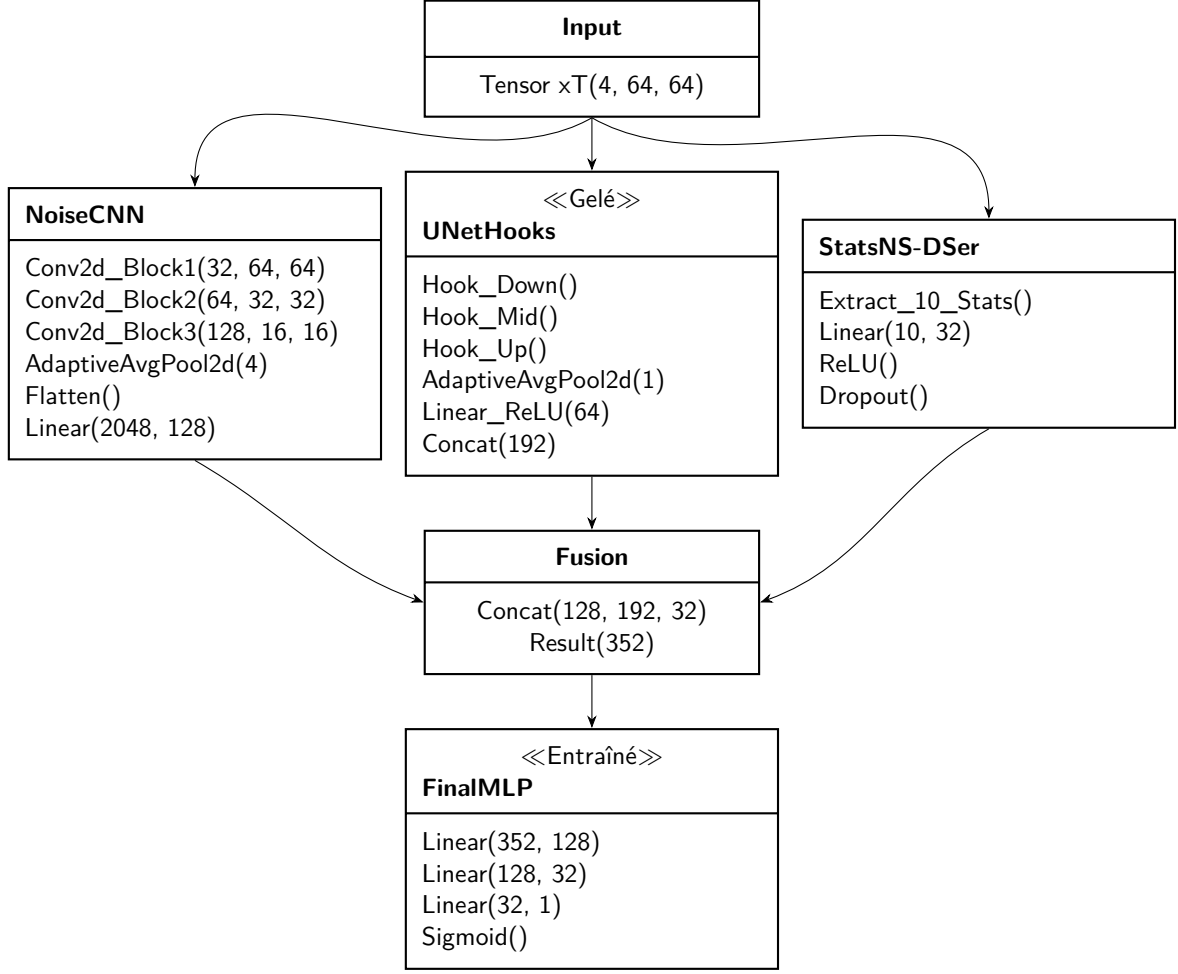


Figure 3.4 : Architecture globale de la première approche

3.6 Deuxième extension - Explorations de différents classifieurs et statistiques

Conformément à l'article original de NS-DSer [41], nous utilisons d'abord un classifieur de type FLD [6]. Ce choix est cohérent avec la faible dimension du vecteur NS-DSer, qui ne contient que dix caractéristiques. Le principe du FLD Ensemble consiste à entraîner plusieurs classifieurs Fisher Linear Discriminant sur des sous-ensembles aléatoires de caractéristiques, puis à agréger leurs décisions. Dans cette deuxième extension nous voulons renforcer cette baseline, notamment par l'ajout de nouvelles caractéristiques statistiques, l'exploration de différents classifieurs et la construction de modèles apprenant explicitement la normalité du bruit latent associé aux images cover.

3.6.1 Ajout de nouvelles caractéristiques sur le bruit latent

L'hypothèse est que les 10 statistiques globales de NS-DSer ne suffisent plus si le bruit a été modifié pour imiter un bruit naturel (gaussien). Dans ce cas, les premiers calculs (comme la moyenne ou la variance) ont l'air totalement normaux, mais il reste des corrélations et des structures suspectes entre les pixels que ces indicateurs basiques ne voient pas.

Pour capturer ces structures cachées, nous extrayons plusieurs familles de caractéristiques supplémentaires directement depuis les cartes de caractéristiques (feature maps) intermédiaires :

- **Des statistiques par canal latent :** Pour chaque canal de caractéristiques individuel, nous calculons séparément la moyenne et la variance en réduisant les dimensions spatiales. Cela permet de repérer si certains filtres spécifiques du réseau réagissent de manière anormale.
- **Des corrélations entre canaux :** Nous calculons une matrice de covariance entre les différents canaux latents. L'objectif est de mesurer comment les motifs abstraits détectés par le réseau co-évoluent, révélant des dépendances mutuelles suspectes induites par la falsification.
- **Des autocorrélations spatiales locales :** En comparant chaque canal avec lui-même après l'avoir décalé de quelques pixels, nous analysons la structure de sa texture. Un bruit parfaitement naturel ne montre généralement aucune corrélation après décalage, contrairement aux bruits artificiels qui conservent une régularité spatiale.
- **Des mesures spectrales basées sur la FFT et la DCT :** Nous appliquons une transformée de Fourier (FFT) en deux dimensions pour analyser le spectre de puissance des caractéristiques. Extraire l'énergie des hautes et des basses fréquences permet de détecter les artefacts périodiques microscopiques et les anomalies de lissage laissés par les algorithmes de génération.
- **Des statistiques locales calculées sur des patches :** À l'aide d'opérations de découpage local (*unfold*), nous extrayons des sous-régions de tailles variées. Calculer la moyenne et la variance sur ces fenêtres locales permet de capturer la structure du bruit à différentes échelles, allant du grain très fin (micro-structure) aux variations spatiales plus larges (macro-structure).

3.6.2 Déploiement de classifieurs avancés

L'ajout massif de nouvelles caractéristiques rend le modèle linéaire *Ensemble FLD* sous-optimal, car ce dernier peine à modéliser des frontières de décision complexes dans un espace de grande dimension. Nous avons donc remplacé le *FLD* par une suite d'architectures de classification avancées, sélectionnées pour leur capacité à gérer la non-linéarité :

1. **Support Vector Machine (SVM) :** Nous utilisons un *SVM* équipé d'une fonction de base radiale. Ce noyau permet de projeter mathématiquement le vecteur de caractéristiques enrichi dans un espace de dimension infinie, permettant de tracer un

hyperplan séparateur optimal entre les distributions de bruit *cover* et *stego* qui sont linéairement inséparables dans l'espace d'origine.

2. **Random Forest (Forêt Aléatoire) [3]** : Nous avons implémenté un ensemble d'arbres de décision cartographiant l'importance de chaque caractéristiques. Cette architecture est spécifiquement intégrée pour sa résistance au surapprentissage (*overfitting*) et sa capacité à gérer le bruit inhérent aux vecteurs extraits du domaine latent.
3. **XGBoost (eXtreme Gradient Boosting) [5]** : Nous utilisons cet algorithme d'amplification de gradient pour optimiser itérativement les erreurs de classification. *XGBoost* construit séquentiellement des arbres de décision faibles, où chaque nouvel arbre est mathématiquement contraint de corriger les résidus (erreurs) de prédiction de la somme des arbres précédents.
4. **Multi-Layer Perceptron (MLP)** : Un réseau de neurones artificiel profond composé de trois couches denses (*Fully Connected*) équipées de fonctions d'activation *ReLU* et de couches de régularisation *Dropout*. Le *MLP* est chargé d'apprendre les interactions complexes de haut niveau entre les composantes spatiales, fréquentielles et les corrélations inter-canaux du nouveau vecteur.
5. **Logistic Regression** : La Régression Logistique est un algorithme d'apprentissage supervisé utilisé pour la classification binaire (comme distinguer une image *Cover* d'une image *Stego*). À partir des caractéristiques d'entrée, elle calcule un score combiné qu'elle transforme en une probabilité entre 0 et 1 via la fonction sigmoïde, traçant ainsi une frontière de décision linéaire nette pour séparer les deux classes selon un seuil.

3.6.3 Apprentissage de la normalité : Approches PCRM et RPCM

En complément de l'extraction de caractéristiques statistiques prédéfinies, notre approche explore une nouvelle stratégie : apprendre directement à quoi ressemble un bruit latent "normal".

L'idée principale n'est plus d'essayer de modéliser la stéganographie, mais de construire un modèle de référence uniquement à partir des images saines (*cover*). Dès lors, toute image dont le bruit s'éloigne de ce modèle de référence est considérée comme une anomalie, et donc potentiellement stéganographiée (*stego*). Pour modéliser la normalité du bruit latent via la détection d'anomalies [4], nous proposons deux méthodes : la première, que nous baptisons PCRM (PCA Cover Residual Model), exploite l'erreur de reconstruction par ACP. La seconde, que nous nommons RPCM (Random Projection Cover Model), évalue la déviation statistique par projections aléatoires [1].

3.6.3.1 Le modèle PCRM

Le modèle PCRM utilise l'Analyse en Composantes Principales (ACP) comme un filtre de normalité. Au lieu de s'en servir pour classer, on s'en sert pour apprendre la géométrie d'une image saine.

Phase d'apprentissage : Nous entraînons l'ACP exclusivement sur les bruits cover. L'algorithme identifie les axes principaux qui forment la structure "naturelle" du bruit. Il crée ainsi un espace sur-mesure pour les images innocentes.

Phase de détection : Pour tester une image suspecte, nous la projetons dans cet espace ACP, puis nous essayons de reconstruire son bruit d'origine. C'est ici que la détection opère : si l'image est stéganographiée, elle contient des micro-altérations que le modèle ne connaît pas (puisqu'il n'a vu que des images cover). Le modèle va donc échouer à reconstruire l'image parfaitement. La différence entre le bruit original et le bruit reconstruit est appelée l'erreur de reconstruction (ou résidu). Un résidu élevé est le signe direct qu'une image cache une anomalie stéganographique.

3.6.3.2 Le modèle RPCM

Le modèle RPCM repose sur une technique de projection aléatoire. Le bruit latent étant un objet mathématique très complexe et de grande dimension, cette méthode propose de l'analyser sous différents "angles".

Phase d'apprentissage : Nous projetons les bruits latents des images cover dans plusieurs directions aléatoires. Pour chaque direction, nous calculons la moyenne et l'écart-type. Cela nous donne une cartographie précise des limites d'un bruit sain.

Phase de détection : Lorsqu'une nouvelle image est analysée, son bruit est projeté dans ces mêmes directions. Si l'image contient un message caché, la structure de son bruit sera légèrement modifiée. Ces altérations vont provoquer un écart significatif par rapport à la moyenne apprise. Plus le score d'écart est grand, plus l'image a de chances d'être classée stego.

3.7 Conclusion

L'approche proposée dans ce mémoire repose donc sur une progression expérimentale en plusieurs niveaux. Nous commençons par réimplémenter NS-DSer de manière fidèle afin d'obtenir une baseline fiable et comparable à l'article de référence. Ensuite, nous introduisons une approche neuronale puis enrichissons cette baseline par l'ajout de nouvelles caractéristiques extraites du bruit latent. Enfin, nous explorons des modèles plus avancés qui cherchent à apprendre explicitement la distribution des bruits cover.

CHAPITRE 4

IMPLÉMENTATION ET RÉSULTATS EXPÉRIMENTAUX

4.1 Introduction

Après avoir présenté les bases théoriques et les différentes approches proposées, nous passons dans ce chapitre à leur mise en œuvre concrète. Nous y décrivons les choix de conception et d’implémentation adoptés pour développer les modèles étudiés ainsi que les stratégies d’entraînement utilisées. Nous présentons ensuite les résultats obtenus afin d’évaluer les performances des méthodes proposées et d’analyser leur apport par rapport aux approches de référence.

4.2 Environnements de Développement, Matériel et Outils

^a

L’inférence de modèles de diffusion (Stable Diffusion) et l’entraînement de réseaux de neurones sont des tâches extrêmement coûteuses en mémoire vidéo (VRAM) et en puissance de calcul. Nous avons donc réparti la charge de travail sur deux plateformes d’exécution distinctes, comme illustré dans la figure 4.1.

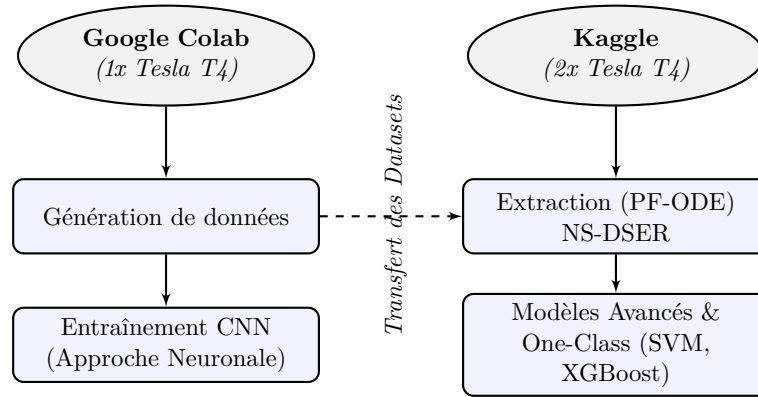


Figure 4.1 : Répartition de l'architecture logicielle et matérielle entre les environnements Cloud.

4.2.1 Google Colab (Génération et Approche Neuronale)

Google Colab (figure 4.2) est un environnement cloud basé sur les notebooks Jupyter, offrant une intégration native avec Google Drive. Nous avons utilisé cet environnement principalement pour la génération des bases de données et l'entraînement de notre première extension (le CNN).

- **Configuration matérielle** : L'environnement d'exécution était configuré avec un mono-GPU **NVIDIA Tesla T4** doté de 16 Go de VRAM, couplé à environ 12 Go de RAM système.
- **Justification** : Cette configuration est optimale pour des flux séquentiels comme la génération d'images paire par paire et l'entraînement du réseau convolutif. L'intégration au Drive a permis le stockage persistant et immédiat des données.

La figure montre l'interface d'un notebook Jupyter sur Google Colab. Le titre du notebook est 'O6_cnn_steganalyzer_v2.ipynb'. La barre de menu en haut inclut 'Fichier', 'Modifier', 'Affichage', 'Insérer', 'Exécution', 'Outils' et 'Aide'. À gauche, une barre latérale contient des icônes pour la table des matières, les fichiers, les cellules, les entrées de console, les variables et le terminal. Le code Python visible dans la cellule est le suivant :

```

!pip install diffusers transformers accelerate safetensors pycryptodome scipy scikit-learn -q

from google.colab import drive
drive.mount('/content/drive')

import os

GITHUB_TOKEN = ""
GITHUB_USER = "RebhiIdir"
REPO_NAME = "DStegan"
REPO_DIR = f"/content/{REPO_NAME}"

if os.path.exists(REPO_DIR):
    %cd (REPO_DIR)
    !git pull origin main
else:
    !git clone https://github.com/{GITHUB_TOKEN}/{GITHUB_USER}/{REPO_NAME}.git
    %cd (REPO_DIR)

!git config user.name "RebhiIdir"
!git config user.email "idir.rebhi@fei.umto.dz"

# créer les dossiers Drive si absents
PROJECT_DIR = "/content/drive/MyDrive/steganalysis_project"
for d in [
    f"{PROJECT_DIR}/datasets/cover_noises",
    f"{PROJECT_DIR}/datasets/stego_noises",
    f"{PROJECT_DIR}/checkpoints",
    f"{PROJECT_DIR}/results",

```

Figure 4.2 : Aperçu de l'interface de notre environnement de développement sur Google Colab.

4.2.2 Kaggle (Extraction NS-Dser et Deuxième Extension)

Kaggle (figure 4.3) est une plateforme spécialisée en science des données offrant des environnements conteneurisés hautement performants. Nous l'avons assignée à la tâche la plus lourde : l'inversion PF-ODE des images pour extraire le bruit latent, et l'entraînement des modèles Machine Learning avancés.

- **Configuration matérielle :** L'environnement Kaggle a été configuré en multi-GPU, utilisant **deux cartes NVIDIA Tesla T4** (soit 2×16 Go de VRAM) et 30 Go de RAM.
- **Justification :** L'implémentation de la fonction d'inversion charge simultanément le gigantesque réseau UNet et l'Auto-encodeur. Pour éviter les erreurs fatales de saturation mémoire (*Out-Of-Memory*), nous avons implémenté une répartition explicite sur les deux GPU.

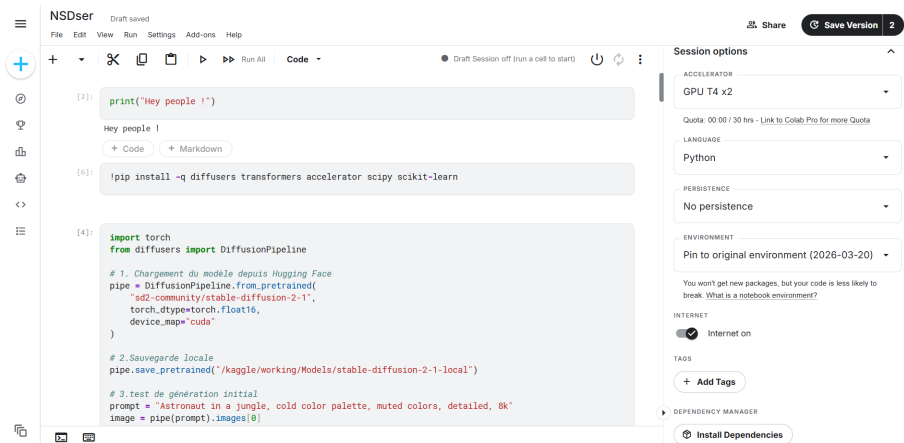


Figure 4.3 : Aperçu de l'environnement Kaggle en configuration multi-GPU (Accelerator : $2 \times$ T4) pour l'extraction de la baseline.

4.2.3 Outils et Bibliothèques Logiciels

La réalisation technique de ce projet s'articule autour d'un écosystème logiciel robuste, principalement centré sur le langage Python et ses bibliothèques dédiées à l'intelligence artificielle.



Python : Il s'agit du langage de programmation central de notre implémentation. Apprécié pour sa clarté syntaxique et sa flexibilité, Python dispose du plus vaste écosystème mondial de bibliothèques orientées data-science et calcul scientifique, ce qui en a fait un choix incontournable pour la structuration de notre code.



PyTorch : Ce framework open source d'apprentissage en profondeur, développé par le laboratoire FAIR (Meta), constitue le socle technologique fondamental de nos travaux. Il permet la manipulation de graphes de calcul dynamiques et gère de manière transparente l'accélération matérielle (CUDA) des tenseurs. Nous l'avons utilisé pour exécuter les modèles de diffusion, manipuler l'espace latent et développer l'architecture de notre réseau CNN.



Scikit-learn : Bibliothèque de référence en Python pour l'apprentissage automatique classique (*Machine Learning*). Elle a été intensivement utilisée dans notre deuxième extension pour implémenter les classifieurs comme SVM et random forest.



Diffusers (Hugging Face) : Cette bibliothèque open source fournit l'API standardisée pour instancier et manipuler les composants des modèles de diffusion. Elle nous a permis de charger le VAE, le réseau UNet, et d'implémenter les équations d'inversion mathématiques (*schedulers*).

4.3 Pipeline Central : Fonctions d'Extraction et d'Inversion

4.3.1 Chargement de l'architecture Stable Diffusion 2.1

L'étape préliminaire de notre extraction consiste à initialiser les modèles pré-entraînés dans notre environnement. La figure 4.4 présente la capture du code Python que nous avons écrit pour charger spécifiquement l'architecture de Stable Diffusion 2.1 via la bibliothèque `diffusers`, en répartissant dynamiquement les poids du modèle en demi-précision (`float16`) sur les unités de calcul disponibles.

```
import torch
from diffusers import DiffusionPipeline

# 1. Chargement du modèle depuis Hugging Face
pipe = DiffusionPipeline.from_pretrained(
    "sd2-community/stable-diffusion-2-1",
    torch_dtype=torch.float16,
    device_map="cuda"
)

# 2. Sauvegarde locale
pipe.save_pretrained("/kaggle/working/Models/stable-diffusion-2-1-local")

# 3. test de génération initial
prompt = "Astronaut in a jungle, cold color palette, muted colors, detailed, 8k"
image = pipe(prompt).images[0]

# Optionnel : sauvegarde aussi l'image de test pour vérifier
image.save("/kaggle/working/test_astronaut.png")

print("MODÈLE SAUVEGARDÉ ET PRÊT EN LOCAL !")
```

Figure 4.4 : Extrait de code illustrant le chargement et la répartition matérielle des poids de Stable Diffusion 2.1.

4.3.2 Logique d'Inversion et Extraction

L'extraction du bruit latent (le cœur de la baseline NS-Dser et de nos extensions) repose sur une séquence d'opérations mathématiques implémentées via PyTorch (voir figure 4.5).

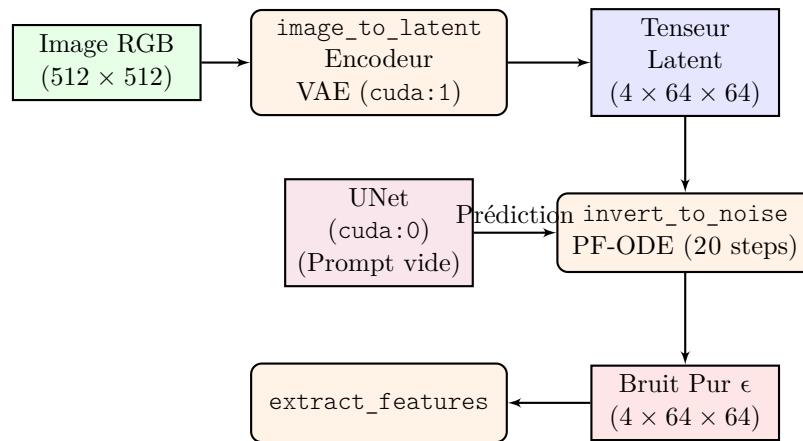


Figure 4.5 : Schématisation de l'inversion et de l'extraction.

Notre code est architecturé autour de trois fonctions fondamentales :

- **image_to_latent(path)** : Cette fonction charge l'image RGB depuis le disque, vérifie sa résolution stricte à 512×512 pixels. Le tenseur est ensuite normalisé dans la plage $[-1, 1]$. Enfin, l'image est passée dans l'encodeur du VAE pour être compressée dans l'espace latent.
- **invert_to_noise(latent)** : Il s'agit de la fonction critique du système. Elle configure un HeunDiscreteScheduler sur 20 pas d'inférence. Le code inverse dynamiquement la liste des timesteps pour simuler le processus à l'envers. Le UNet prédit le résidu de bruit, et le solveur estime l'état latent au pas de bruit supérieur, jusqu'à obtenir le tenseur de bruit pur final.
- **extract_features(noise_latent)** : Cette fonction rapatrie le bruit sur le CPU. Elle implémente le calcul des moments statistiques exigés par la baseline NS-Dser (moyenne, variance, asymétrie, kurtosis et IQR).

4.3.3 Le choix du scheduler Heun

Pour des raisons expérimentales liées à la phase d'implémentation, le HeunDiscreteScheduler a été privilégié au détriment du scheduler DDIM. En pratique, l'intégration l'inversion DDIM standard s'est révélée complexe à stabiliser sur un nombre réduit de pas dans notre environnement de développement. L'utilisation du scheduler de Heun s'est alors imposée comme une alternative particulièrement pertinente : ça nous a permis d'obtenir une configuration stable et fonctionnelle en seulement 20 pas, garantissant ainsi la viabilité et la fluidité de notre pipeline d'extraction.

Bien que ces deux schedulers ne produisent pas des résultats strictement identiques en termes de valeurs numériques, cet écart n'altère pas la cohérence globale de notre étude. L'objectif fondamental reste inchangé : extraire le bruit latent.

4.4 Génération et Organisation Structurée des Données

La préparation des données a été effectuée sur Colab. Nous avons généré les paires d'images (voir la figure 4.6) en utilisant les prompts des bases Flickr8K et COCO. Pour chaque image, la même graine aléatoire (*seed*) a été utilisée pour garantir un appariement parfait. Trois techniques stéganographiques ont été isolées dans des datasets distincts, nécessitant l'utilisation d'architectures génératives spécifiques : le modèle **Stable Diffusion v2.1** a été déployé pour générer les bases liées au MC et au *mas-GRDH*, tandis que le modèle **Stable Diffusion v1.5** a été utilisé pour le dataset *Gaussian Shading* (ce dernier étant réservé exclusivement à l'évaluation de l'approche neuronale).

Comme l'illustre la figure 4.7, le partage strict du texte de conditionnement et du bruit initial permet d'obtenir un contenu sémantique identique.

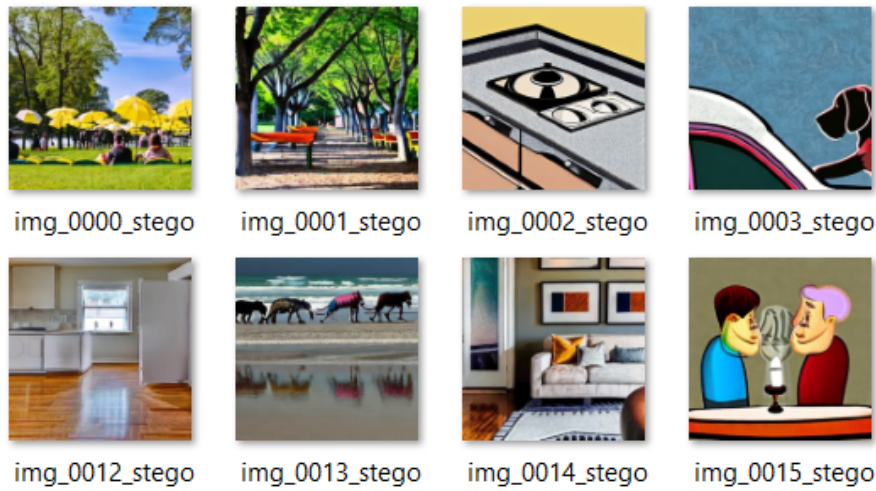


Figure 4.6 : Aperçu des paires d'images générées. L'utilisation stricte du même *prompt* et de la même graine aléatoire garantit un contenu visuel extrêmement similaire entre les classes.

Toutefois, l'altération du tenseur latent lors du processus stéganographique induit des micro-modifications lors du décodage final. La figure 4.7 met en évidence cette comparaison : bien que le sujet principal reste visuellement identique, de légères variations structurelles différencient la version altérée de la version innocente.

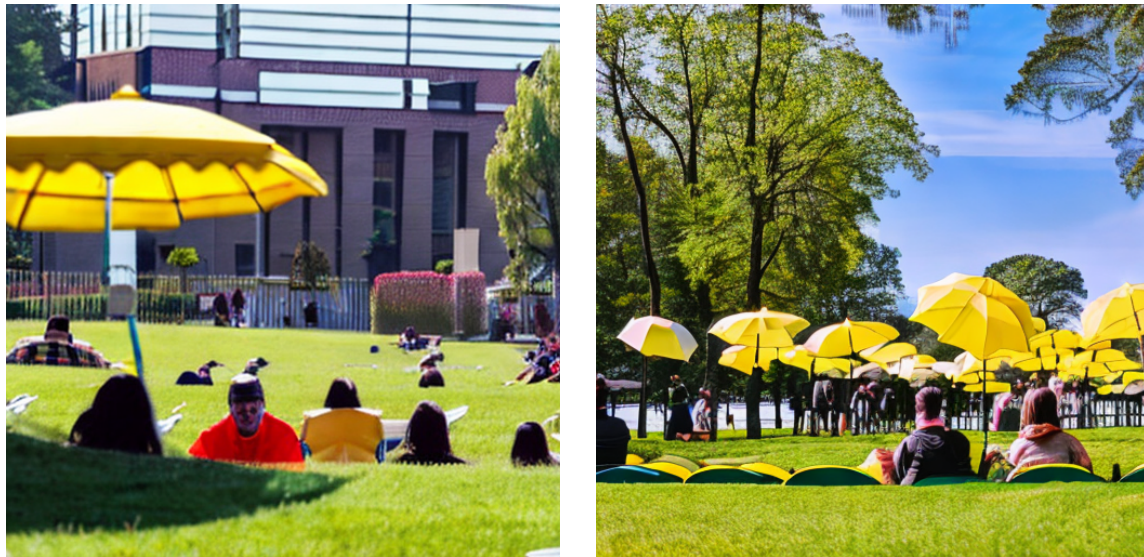


Image Cover

Image Stego

Figure 4.7 : Comparaison directe d'une paire générée à partir du même prompt. Malgré une forte similarité sémantique, le processus d'insertion engendre de légères variations dans les détails générés.

4.4.1 Implémentation des DataLoaders (Approche Neuronale)

Pour nourrir notre réseau de neurones convolutif (CNN), l'organisation des données sur le disque a été couplée à l'API `torch.utils.data`.

Donnees_Cover	23/05/2026 19:26	Dossier de fichiers
Donnees_Stego	23/05/2026 19:31	Dossier de fichiers
mas-GRDH	26/05/2026 02:19	Dossier de fichiers

Figure 4.8 : Organisation structurelle des données sur le disque, séparant rigoureusement les classes pour prévenir toute fuite d'information (*data leakage*) lors de l'entraînement.

- Les images ont été structurées en sous-répertoires stricts (voir figure 4.8).
- Les **DataLoaders** ont automatisé le traitement par lots (*batch_size* de 32), avec l'option `shuffle=True` et `num_workers=2` pour un pré-chargement asynchrone sans bloquer le GPU. Contrairement à la classification d'images standard, **aucune augmentation de données** (rotation, recadrage) n'a été appliquée, pour ne pas détruire le signal stéganographique microscopique.

4.5 Stratégie d'Entraînement et Hyperparamètres

4.5.1 Stratégie d'Entraînement de la Première Extension (CNN)

Le réseau convolutif a été entraîné de bout en bout avec les configurations suivantes :

- **Fonction de perte (Loss) :** Perte d'Entropie Croisée Binaire (`BCEWithLogitsLoss`).

- **Optimiseur et Taux d'apprentissage** : Optimiseur **Adam** avec un taux d'apprentissage initial de 1×10^{-4} .
- **Epochs et Early Stopping** : 50 epochs. Pour prévenir le surapprentissage, nous avons implémenté un arrêt prématuré (Early Stopping) de 15 epochs.

4.5.2 Stratégie d'Entraînement de la Deuxième Extension (Machine Learning & One-Class)

Pour les classifieurs avancés (XGBoost, SVM) et les modèles de détection d'anomalies :

- **Pré-traitement** : Application systématique d'un `StandardScaler` ajusté uniquement sur le jeu d'entraînement pour centrer et réduire les données ($\mu = 0, \sigma = 1$).
- **Recherche d'Hyperparamètre** : Validation croisée à 5 plis. Pour le SVM RBF, exploration de la pénalité C et du noyau γ . Pour XGBoost, optimisation de la profondeur maximale (`max_depth`) et du nombre d'estimateurs.

4.6 Résultats Expérimentaux et Évaluation

L'évaluation de nos systèmes de stéganalyse a été menée de manière rigoureuse sur nos jeux de données générés (2000 échantillons par technique : 1000 *cover* et 1000 *stego*). Face à la complexité et à la subtilité de la stéganographie générative, les résultats mettent en évidence les limites des approches actuelles, tout en dégageant de nouvelles pistes de recherche.

4.6.1 Évaluation de la Baseline NS-DSer

Nous avons dans un premier temps évalué l'approche de référence NS-DSer sur nos données. La figure 4.9 illustre les performances obtenues sur la technique de *Message Concealment* (MC), tandis que la figure 4.10 présente les résultats face à la méthode *mas-GRDH*.

```

Training complete.

=====
NS-DSEr – FLD ENSEMBLE RESULTS
=====
Accuracy : 0.9952
Precision : 1.0000
Recall    : 0.9905
F1-Score  : 0.9952
AUC-ROC   : 1.0000

Confusion Matrix:
[[105  0]
 [ 1 104]]

Classification Report:
              precision    recall  f1-score   support

     0       0.9906       1.0000       0.9953         105
     1       1.0000       0.9905       0.9952         105

 accuracy                   0.9952         210
 macro avg       0.9953       0.9952       0.9952         210
 weighted avg    0.9953       0.9952       0.9952         210

```

Figure 4.9 : Résultats de l'évaluation de la baseline NS-DSEr sur le jeu de données MC.

Discussion sur la méthode Message Concealment (MC)

Les tests menés sur la méthode (MC) ont donné une précision de détection de 99,52 %. Ce résultat s'avère particulièrement cohérent avec les performances rapportées dans l'étude de référence NS-DSEr [41]. Une telle précision s'explique par la nature même de la méthode MC : bien qu'elle agisse sur le bruit latent lors de la génération, elle crée des anomalies statistiques grossières et uniformes que notre pipeline de stéganalyse parvient à capturer sans difficulté. Obtenir un score aussi proche, malgré l'utilisation de jeux de données distincts, conforte et valide notre implémentation technique. Cela démontre la fiabilité de notre chaîne d'extraction pour détecter les formes d'insertions directes et atteste de la bonne configuration de notre pipeline par rapport à l'état de l'art.

```

Training complete.

=====
NS-DSEr - FLD ENSEMBLE RESULTS
=====
Accuracy : 0.5200
Precision : 0.5227
Recall : 0.4600
F1-Score : 0.4894
AUC-ROC : 0.5245

Confusion Matrix:
[[58 42]
 [54 46]]

Classification Report:

```

	precision	recall	f1-score	support
0	0.5179	0.5800	0.5472	100
1	0.5227	0.4600	0.4894	100
accuracy			0.5200	200
macro avg	0.5203	0.5200	0.5183	200
weighted avg	0.5203	0.5200	0.5183	200

Figure 4.10 : Résultats de l'évaluation de la baseline NS-DSEr sur le jeu de données *mas-GRDH*.

Discussion sur la méthode mas-GRDH

À l'inverse, les tests menés sur la méthode *mas-GRDH* (figure 4.10) ont donné une accuracy d'environ 52 %. Ce résultat est là aussi tout à fait cohérent avec les résultats de référence de NS-DSEr, où les performances observées se situent également aux alentours de 50 %. Le score de détection de 52 % obtenu sur la méthode *mas-GRDH* confirme la prédiction théorique liée aux approches distribution-preserving. En effet, ces méthodes étant conçues pour préserver la distribution statistique originale des images, elles tendent par principe vers l'indéfectabilité. Ce résultat, proche d'une classification aléatoire, valide ainsi la cohérence théorique de notre modèle face à ce type de stéganographie. Cela confirme la redoutable efficacité de *mas-GRDH*. Contrairement à MC, cette méthode de nouvelle génération n'introduit pas d'anomalies grossières, mais fonde le message secret de manière extrêmement subtile dans la structure même du bruit, rendant la détection statistique totalement impuissante.

4.6.2 Évaluation de la Première Extension : Approche Neuronale (CNN)

Notre réseau de neurones convolutif a été entraîné pour apprendre directement les motifs spatiaux du bruit. La figure 4.11 résume les métriques de cette première extension.

Train : 3200 | Val : 400 | Test : 400

Entraînement NSDSerPlus sur 2000 paires...

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Val P_E
1	0.6958	0.4978	0.6939	0.4875	0.5000
-> Meilleur modele sauvegarde (epoch 1)					
2	0.6939	0.4963	0.6935	0.5075	0.5000
-> Meilleur modele sauvegarde (epoch 2)					
3	0.6949	0.4863	0.6935	0.4825	0.5000
4	0.6919	0.5072	0.6934	0.4975	0.4908
-> Meilleur modele sauvegarde (epoch 4)					
5	0.6944	0.4997	0.6937	0.4950	0.4976
6	0.6943	0.4869	0.6937	0.4900	0.5000
7	0.6931	0.5134	0.6939	0.4925	0.4991
8	0.6934	0.5059	0.6941	0.4875	0.4975
9	0.6936	0.4994	0.6940	0.4950	0.4951
10	0.6927	0.5181	0.6941	0.4775	0.4975
11	0.6932	0.4991	0.6939	0.4575	0.4975
12	0.6931	0.5038	0.6939	0.4850	0.4944
13	0.6915	0.5156	0.6941	0.4525	0.4946
14	0.6924	0.5191	0.6941	0.4600	0.4976

Early stopping a l epoch 14
Meilleur modele : epoch 4

Entraînement termine !

Figure 4.11 : Performances de classification de notre architecture CNN (Première Extension) sur les résidus de bruit latent.

Ces résultats mettent en évidence une différence de détection majeure selon la méthode stéganographique employée. D'un côté, notre approche de référence détecte parfaitement la technique de Message Concealment (MC) avec une précision de 99%, ce qui valide la fiabilité de notre solution. De l'autre côté, face à la technique mas-GRDH, les performances s'effondrent à environ 52%, soit l'équivalent d'un choix aléatoire. Notre première extension basée sur un réseau de neurones convolutif (CNN) a d'ailleurs stagné à ce même seuil de 50. Malgré la capacité théorique du Deep Learning à extraire des caractéristiques complexes, les résultats confirment la difficulté extrême de la tâche : les modifications induites par le processus stéganographique dans l'espace latent sont si infimes qu'elles se confondent avec le bruit naturel du processus de diffusion, limitant ainsi la précision du classifieur.

4.6.3 Évaluation de la Deuxième Extension : Modèles Avancés et Nouvelles Caractéristiques

Sur l'espace initial de 10 statistiques, la Régression Logistique et XGBoost plafonnent autour de 52,5% de précision (proche d'une prédiction aléatoire). L'approche par *Stacking* a même sous-performé, s'effondrant à une précision de 46,0% (AUC de 0.43), traduisant un surapprentissage massif du bruit ou une incapacité à généraliser. Des modèles puissants comme le SVM RBF ou les ExtraTrees n'ont pas non plus réussi à obtenir les résultats espérés.

Une piste prometteuse : L'apport des matrices RPCM

Bien que la majorité des approches se soient soldées par un échec, une configuration spécifique a montré des résultats encourageants. La combinaison des caractéristiques tabulaires et des descripteurs RPCM (*tabular_plus_rpcm*), classifiée par une **Régression Logistique pénalisée**, a obtenu les meilleures performances de toutes nos expériences :

- **Précision de test : 57,00%**

- **F1-Score** : 0.5521
- **ROC-AUC** : 0.5500

Ce résultat (voir la figure 4.12), bien qu'insuffisant pour un déploiement industriel fiable, est le plus prometteur de notre étude. Il suggère que l'analyse des corrélations de phase et des dépendances spatiales inter-canaux (capturées par le RPCM) recèle une infime partie du signal stéganographique altéré par le décodeur VAE. C'est la seule piste qui parvient à s'élever nettement et de manière constante au-dessus du hasard mathématique.

```

=====
tabular_plus_rpcm
Dim: 1296
=====
tabular_plus_rpcm | LogReg_C0.03 val=0.5600 test=0.5350 auc=0.5420 f1=0.5373
tabular_plus_rpcm | LogReg_C0.1 val=0.5650 test=0.5250 auc=0.5459 f1=0.4865
tabular_plus_rpcm | LogReg_C0.3 val=0.5600 test=0.5500 auc=0.5473 f1=0.5213
tabular_plus_rpcm | LogReg_C1 val=0.5650 test=0.5700 auc=0.5500 f1=0.5521
tabular_plus_rpcm | ElasticNet_03 val=0.5400 test=0.5300 auc=0.5508 f1=0.5155

```

Figure 4.12 : Résultats RPCM.

Conclusion

Ce chapitre a permis de présenter notre environnement technique, nos choix d'implémentation, ainsi que d'évaluer les performances de nos modèles. Si l'approche de référence détecte parfaitement la méthode MC (99,52 %), elle s'avère impuissante face à la subtilité de mas-GRDH (52 %), un constat d'échec également partagé par notre modèle neuronal (CNN).

Dans ce contexte difficile, l'intégration des matrices RPCM est la seule à montrer un léger signal avec 57% d'accuracy. Bien que ce score reste insuffisant pour une détection fiable, il indique que l'analyse des dépendances spatiales constitue une piste intéressante pour orienter de futurs travaux.

CONCLUSION GÉNÉRALE

1 Synthèse

Ce travail de mémoire marque l’aboutissement d’une recherche rigoureuse autour de la stéganalyse et des nouvelles techniques de dissimulation. Notre objectif principal était de mesurer si les outils de détection actuels (la stéganalyse) sont capables de repérer des messages cachés via des technologies de pointe, à savoir la stéganographie générative dans l’espace latent.

Pour y parvenir, nous avons avancé par étapes. Dans un premier temps, nous avons présenté le sujet et posé un panel théorique complet pour bien comprendre le fonctionnement des modèles de diffusion. Ensuite, nous avons passé en revue l’état de l’art de manière approfondie. Ce travail d’analyse nous a permis de bien cerner les forces et les faiblesses des approches existantes, et de poser des bases solides pour nos propres expérimentations.

Dans la phase pratique, nous avons implémenté un pipeline complet basé sur la méthode de référence NS-Dser, qui consiste à inverser la trajectoire de diffusion pour extraire le bruit latent des images. Nos tests ont mis en évidence un contraste très net. D’un côté, nous avons obtenu un succès total sur la méthode classique *Message Concealment* (MC) avec une précision de détection de 99 %, ce qui valide notre code et confirme l’efficacité de l’analyse statistique sur les insertions directes. De l’autre côté, face à la technique moderne *mas-GRDH*, notre précision s’est effondrée à 52 % (du pur hasard). Ce résultat confirme la prédiction théorique d’indéfectabilité propre aux approches dites *distribution-preserving* ; ces dernières préservant par principe la distribution statistique originale des images, elles rendent la classification structurellement proche du hasard, validant ainsi la cohérence théorique de notre modèle.

Ce mémoire montre que la stéganographie générative évolue aujourd’hui beaucoup plus vite que les outils de détection traditionnels, ce qui nous oblige à repenser notre manière de sécuriser les images.

2 Perspectives

Nous envisageons plusieurs perspectives aux travaux présentés tout au long de ce mémoire.

Afin de consolider les observations de cette étude et d'apporter des réponses face à l'échec de la stéganalyse traditionnelle contre les approches génératives, plusieurs axes de recherche concrets doivent être explorés.

Les perspectives peuvent aussi être présentées sous forme de liste :

- **Évaluation sur des images réelles** : Il sera crucial d'évaluer le comportement de notre pipeline d'inversion sur de vraies photographies (non synthétiques) pour mesurer l'impact du bruit de capteur réel par rapport au bruit théorique des modèles de génération de contenu.
- **Variations des architectures de diffusion** : Actuellement restreint à Stable Diffusion, notre système devra être confronté à des images issues d'autres générateurs (comme Midjourney ou DALL-E 3) afin de vérifier si nos observations statistiques et nos métriques de détection se généralisent d'un modèle à un autre.
- **Changement de paradigme vers la sanitisation** : Face à l'extrême difficulté de détecter les messages cachés par *mas-GRDH*, l'avenir de la défense repose sur le nettoyage automatique et préventif des flux d'images aux passerelles de sécurité. Au lieu de chercher à détecter l'invisible, l'objectif est de neutraliser le canal secret.
- **Intégration de la méthode ADS (Adversarial Diffusion Sanitization)** : En nous inspirant des travaux récents de Frants et Agaian [11], nous envisageons d'implémenter un processus qui applique systématiquement à chaque image un cycle très court de diffusion (un seul pas de bruitage/débruitage) couplé à une micro-perturbation basée sur les quaternions, détruisant ainsi la structure microscopique du message sans altérer la qualité visuelle de l'image.

3 Disponibilité du code et reproductibilité

L'ensemble de ce projet de recherche est entièrement disponible en ligne sur GitHub. Vous pouvez visualiser l'intégralité du code source, des scripts et de l'architecture générale à l'adresse suivante : <https://github.com/RebhiIdir/DStegan.git>.

Il est important de noter que dans son état actuel, l'environnement global reste difficilement reproductible en un seul clic à cause des nombreuses dépendances matérielles et des configurations de bibliothèques spécifiques. Cependant, le projet est totalement transparent, complet et consultable. Pour faciliter la compréhension et l'accès à certaines étapes clés de notre travail, plusieurs notebooks importants ont été déposés sur Kaggle et Google Colab. Le tableau 4.1 rassemble les liens vers ces ressources et jeux de données.

Plateforme / Composant	Lien d'accès ou Notebook
Dépôt GitHub Principal	https://github.com/RebhiIdir/DStegan.git
Notebook Kaggle	Code de référence NS-DSer : https://www.kaggle.com/code/idirrebhi/nsdser
Google Colab (Données mas-GRDH)	Génération et préparation des données <i>mas-GRDH</i> : https://colab.research.google.com/drive/1VayTd9MSf_RxZmBAg9sTLwXmzXzBpSDL?usp=sharing
Google Colab (Extension 1)	Implémentation du CNN et des <i>hooks</i> du UNet : https://colab.research.google.com/drive/1r7A-xs2FrWvubkwBnHGwmOcWfun3WmaE?usp=sharing
Google Colab (Gaussian Shading)	Génération des données de test <i>Gaussian Shading</i> : https://colab.research.google.com/drive/104siaNVnEL5UEvyOH_7e1hvkTK1UGgGp?usp=sharing
Datasets	Lien vers les données (covers et stego) https://drive.google.com/drive/folders/13-byY7SyrEgkCZvn-GYbNgRFf3-3WUAf?usp=sharing

Tableau 4.1 : Liste des liens externes vers les codes sources, notebooks de génération et environnements d'exécution

Annexes

ANNEXE A

ARCHITECTURE DÉTAILLÉE DE L'APPROCHE

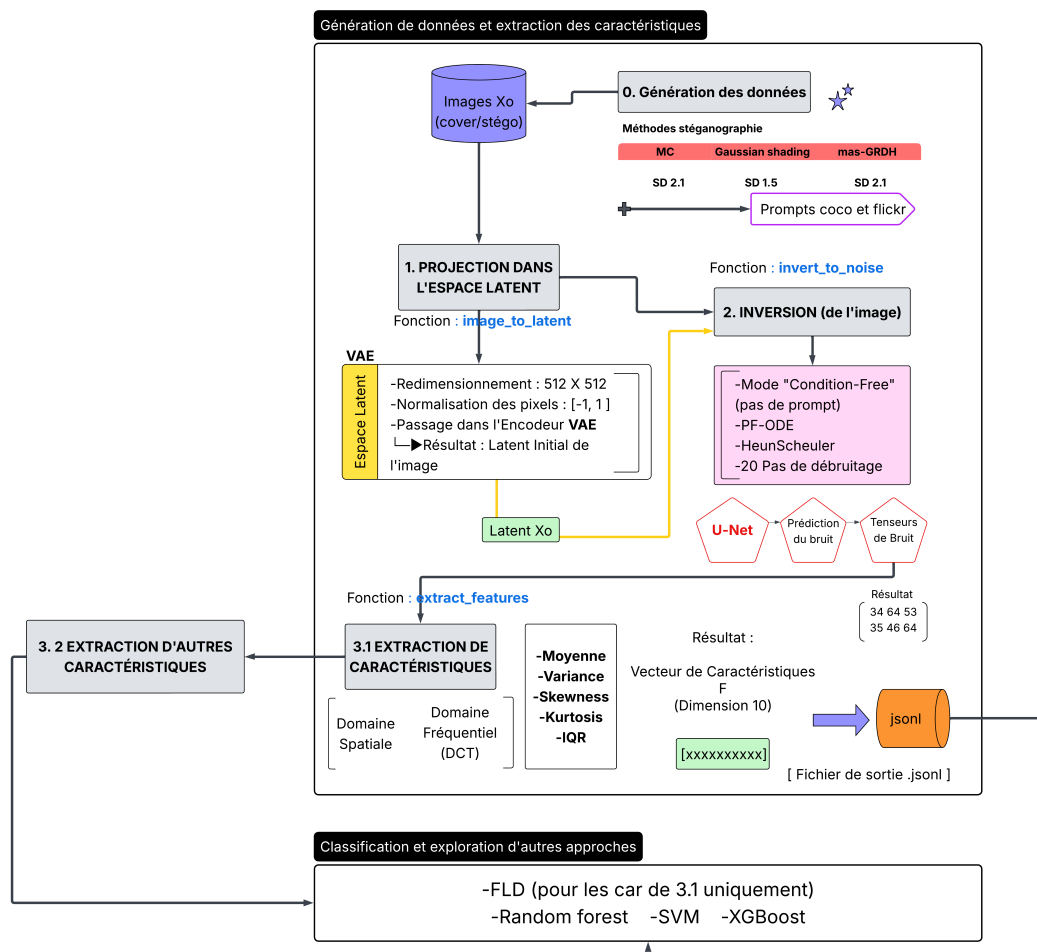


Figure A.1 : Architecture détaillée de l'approche proposée. De la génération des données à la classification

ANNEXE B

RÉSULTATS BRUTS DE CLASSIFICATION SUR LA BASE MAS-GRDH

Cette annexe rassemble l'ensemble des mesures brutes obtenues lors de nos expérimentations de classification face à l'algorithme de stéganographie générative *mas-GRDH*. Afin de maximiser les chances de détection, nous avons testé trois configurations de vecteurs de caractéristiques (*features*) sur neuf classificateurs différents.

B.1 Tableau comparatif des performances

Le tableau B.1 présente les scores d'exactitude (*accuracy*), d'aire sous la courbe ROC (*AUC*) et de *F1-score* pour chaque modèle. Les résultats confirment de manière flagrante que peu importe l'algorithme choisi ou l'enrichissement des caractéristiques, les performances stagnent aux alentours de 50 % à 56 %, ce qui équivaut à un choix purement aléatoire.

B.2 Matrices de corrélation et analyses graphiques

La figure B.1 regroupe les graphiques d'analyse de notre vecteur de caractéristiques (divisés en deux captures pour des raisons d'affichage : la distribution en haut et la matrice de corrélation en bas). On remarque tout de suite qu'il n'y a aucune différence visible ni aucun motif particulier entre les images saines (*cover*) et les images modifiées (*stego*). Les données se mélangent complètement, ce qui explique graphiquement pourquoi tous nos modèles de Machine Learning échouent et n'arrivent pas à faire la distinction.

Configuration des descripteurs	Classificateur	Accuracy	AUC	F1-score
NS-DSer Baseline (10 statistiques)	FLD	0,5200	0,5245	0,4894
	LogReg	0,5250	0,5451	0,4809
	SVM (RBF)	0,4700	0,5170	0,4952
	RandomForest	0,4800	0,4975	0,4951
	ExtraTrees	0,4900	0,5180	0,5096
	GradientBoosting	0,4750	0,4742	0,4928
	MLP	0,5000	0,5422	0,5050
	XGBoost	0,5200	0,5276	0,5385
	LightGBM	0,4850	0,5245	0,4926
Noise Enhanced Features (122 statistiques de bruit)	FLD	0,5550	0,5476	0,5742
	LogReg	0,5650	0,5538	0,5797
	SVM (RBF)	0,5450	0,4432	0,5604
	RandomForest	0,4850	0,4745	0,5024
	ExtraTrees	0,4450	0,4717	0,4478
	GradientBoosting	0,4600	0,4870	0,4953
	MLP	0,5600	0,5520	0,5769
	XGBoost	0,4800	0,4905	0,5048
	LightGBM	0,4800	0,4799	0,4902
NS-DSer + Noise Features (Combinaison : 132 caractéristiques)	FLD	0,5600	0,5551	0,5644
	LogReg	0,5650	0,5658	0,5756
	SVM (RBF)	0,5300	0,4579	0,5437
	RandomForest	0,4600	0,4880	0,5000
	ExtraTrees	0,5200	0,5008	0,5429
	GradientBoosting	0,4650	0,4683	0,4729
	MLP	0,5500	0,5754	0,5408
	XGBoost	0,5150	0,4823	0,5268
	LightGBM	0,5000	0,4893	0,5000

Tableau B.1 : Comparaison brute des performances de détection sur la base mas-GRDH (2 000 échantillons)

	feature_set	model	accuracy	precision	recall	f1	roc_auc	confusion_matrix
19	NSDser_10stats_plus_noise_features	LogReg	0.565	0.561905	0.59	0.575610	0.56580	[[54, 46], [41, 59]]
10	Noise_enhanced_features_only	LogReg	0.565	0.560748	0.60	0.579710	0.55380	[[53, 47], [40, 60]]
18	NSDser_10stats_plus_noise_features	FLD	0.560	0.558824	0.57	0.564356	0.55510	[[55, 45], [43, 57]]
15	Noise_enhanced_features_only	MLP	0.560	0.555556	0.60	0.576923	0.55200	[[52, 48], [40, 60]]
9	Noise_enhanced_features_only	FLD	0.555	0.550459	0.60	0.574163	0.54760	[[51, 49], [40, 60]]
24	NSDser_10stats_plus_noise_features	MLP	0.550	0.552083	0.53	0.540816	0.57540	[[57, 43], [47, 53]]
11	Noise_enhanced_features_only	SVM_RBF	0.545	0.542056	0.58	0.560386	0.44320	[[51, 49], [42, 58]]
20	NSDser_10stats_plus_noise_features	SVM_RBF	0.530	0.528302	0.56	0.543689	0.45790	[[50, 50], [44, 56]]
1	NSDser_baseline_10stats	LogReg	0.525	0.530120	0.44	0.480874	0.54510	[[61, 39], [56, 44]]
7	NSDser_baseline_10stats	XGBoost	0.520	0.518519	0.56	0.538462	0.52760	[[48, 52], [44, 56]]
0	NSDser_baseline_10stats	FLD	0.520	0.522727	0.46	0.489362	0.52450	[[58, 42], [54, 46]]
22	NSDser_10stats_plus_noise_features	ExtraTrees	0.520	0.518182	0.57	0.542857	0.50075	[[47, 53], [43, 57]]
25	NSDser_10stats_plus_noise_features	XGBoost	0.515	0.514286	0.54	0.526829	0.48230	[[49, 51], [46, 54]]
6	NSDser_baseline_10stats	MLP	0.500	0.500000	0.51	0.504950	0.54220	[[49, 51], [49, 51]]
26	NSDser_10stats_plus_noise_features	LightGBM	0.500	0.500000	0.50	0.500000	0.48930	[[50, 50], [50, 50]]
4	NSDser_baseline_10stats	ExtraTrees	0.490	0.490741	0.53	0.509615	0.51795	[[45, 55], [47, 53]]
8	NSDser_baseline_10stats	LightGBM	0.485	0.485437	0.50	0.492611	0.52450	[[47, 53], [50, 50]]
12	Noise_enhanced_features_only	RandomForest	0.485	0.485981	0.52	0.502415	0.47445	[[45, 55], [48, 52]]

(a) Distribution des caractéristiques de bruit (Partie supérieure)

3	NSDser_baseline_10stats	RandomForest	0.480	0.481132	0.51	0.495146	0.49745	[[45, 55], [49, 51]]
16	Noise_enhanced_features_only	XGBoost	0.480	0.481818	0.53	0.504762	0.49050	[[43, 57], [47, 53]]
17	Noise_enhanced_features_only	LightGBM	0.480	0.480769	0.50	0.490196	0.47990	[[46, 54], [50, 50]]
5	NSDser_baseline_10stats	GradientBoosting	0.475	0.476636	0.51	0.492754	0.47420	[[44, 56], [49, 51]]
2	NSDser_baseline_10stats	SVM_RBF	0.470	0.472727	0.52	0.495238	0.51700	[[42, 58], [48, 52]]
23	NSDser_10stats_plus_noise_features	GradientBoosting	0.465	0.466019	0.48	0.472906	0.46830	[[45, 55], [52, 48]]
21	NSDser_10stats_plus_noise_features	RandomForest	0.460	0.465517	0.54	0.500000	0.48800	[[38, 62], [46, 54]]
14	Noise_enhanced_features_only	GradientBoosting	0.460	0.464912	0.53	0.495327	0.48700	[[39, 61], [47, 53]]
13	Noise_enhanced_features_only	ExtraTrees	0.445	0.445545	0.45	0.447761	0.47170	[[44, 56], [55, 45]]

Saved benchmark: /kaggle/working/ns_dser_improvement_benchmark.csv

Best result:

	feature_set	model	accuracy	precision	recall	f1	roc_auc	confusion_matrix
19	NSDser_10stats_plus_noise_features	LogReg	0.565	0.561905	0.59	0.57561	0.5658	[[54, 46], [41, 59]]

(b) Matrice de corrélation des blocs de caractéristiques (Partie inférieure)

Figure B.1 : Visualisation globale du rapport d'analyse statistique effectué sur le vecteur complet (132 dimensions)

BIBLIOGRAPHIE

- [1] E. Bingham et H. Mannila, “Random projection in dimensionality reduction: applications to image and text data”, in *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 2001, pp. 245–250.
- [2] M. Boroumand, M. Chen, et J. Fridrich, “Deep residual network for steganalysis of digital images”, *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 5, pp. 1181–1193, 2018.
- [3] L. Breiman, “Random forests”, *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] V. Chandola, A. Banerjee, et V. Kumar, “Anomaly detection: A survey”, *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, p. 15, 2009.
- [5] T. Chen et C. Guestrin, “Xgboost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16. ACM, Août 2016, p. 785–794.
- [6] R. Cogranne, T. Denemark, et J. Fridrich, “Theoretical model of the fld ensemble classifier based on hypothesis testing theory”, in *2014 IEEE International Workshop on Information Forensics and Security (WIFS)*, 2014, pp. 167–172.
- [7] I. Cox, M. Miller, J. Bloom, J. Fridrich, et T. Kalker, *Digital watermarking and steganography*. Morgan Kaufmann, 2007.
- [8] N. J. D. L. Croix, T. Ahmad, et F. Han, “Comprehensive survey on image steganalysis using deep learning”, *Array*, vol. 22, p. 100353, Juil. 2024.
- [9] J. Devlin, M.-W. Chang, K. Lee, et K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding”, 2019. En ligne: <https://arxiv.org/abs/1810.04805>
- [10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, et N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale”, in *International Conference on Learning Representations*, 2021.

- [11] V. Frants et S. Aghaian, “Training-free color-aware adversarial diffusion sanitization for diffusion stegomalware defense at security gateways”, 2025. En ligne: <https://arxiv.org/abs/2512.24499>
- [12] J. Fridrich et J. Kodovsky, “Rich models for steganalysis of digital images”, *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 3, pp. 868–882, 2012.
- [13] B. Ghogh et A. Ghodsi, “Diffusion models: Tutorial and survey”, 07 2024.
- [14] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, et Y. Bengio, “Generative adversarial networks”, 2014. En ligne: <https://arxiv.org/abs/1406.2661>
- [15] J. Ho, A. Jain, et P. Abbeel, “Denoising diffusion probabilistic models”, 2020. En ligne: <https://arxiv.org/abs/2006.11239>
- [16] V. Holub et J. Fridrich, “Designing steganographic distortion using directional filters”, in *IEEE International Workshop on Information Forensics and Security (WIFS)*, 2012, pp. 234–239.
- [17] V. Holub, J. Fridrich, et T. Denemark, “Universal distortion function for steganography in an arbitrary domain”, *EURASIP Journal on Information Security*, vol. 2014, pp. 1–13, 2014.
- [18] X. Hu, S. Li, Q. Ying, W. Peng, X. Zhang, et Z. Qian, “Establishing robust generative image steganography via popular stable diffusion”, *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8094–8108, 2024.
- [19] S. Kullback et R. A. Leibler, “On information and sufficiency”, *The Annals of Mathematical Statistics*, vol. 22, no. 1, pp. 79–86, 03 1951.
- [20] Y. Lecun, Y. Bengio, et G. Hinton, “Deep learning”, *Nature*, vol. 521, no. 7553, pp. 436–444, Mai 2015.
- [21] K. O’Shea et R. Nash, “An introduction to convolutional neural networks”, 2015. En ligne: <https://arxiv.org/abs/1511.08458>
- [22] T. Pevný, P. Bas, et J. Fridrich, “Steganalysis by subtractive pixel adjacency matrix”, in *Proceedings of the 11th ACM Workshop on Multimedia and Security*, ser. MM&Sec ’09. New York, NY, USA: Association for Computing Machinery, 2009, pp. 75–84.
- [23] Y. Qian, J. D. Dong, W. Wang, et T. Tan, “Deep learning for steganalysis via convolutional neural networks”, in *Media Watermarking, Security, and Forensics 2015*, vol. 9409. SPIE, 2015, pp. 101–112.
- [24] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, et I. Sutskever, “Learning transferable visual models from natural language supervision”, 2021. En ligne: <https://arxiv.org/abs/2103.00020>
- [25] A. Radford, K. Narasimhan, T. Salimans, et I. Sutskever, “Improving Language Understanding by Generative Pre-Training.”
- [26] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, et B. Ommer, “High-resolution image synthesis with latent diffusion models”, 2022. En ligne: <https://arxiv.org/abs/2112.10752>

- [27] A. Selvaraj, A. Ezhilarasan, S. L. J. Wellington, et A. R. Sam, “Digital image steganalysis: A survey on paradigm shift from machine learning to deep learning based techniques”, *IET Image Processing*, vol. 15, no. 2, pp. 504–522, Fév. 2021.
- [28] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, et S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics”, 2015. En ligne: <https://arxiv.org/abs/1503.03585>
- [29] J. Song, C. Meng, et S. Ermon, “Denoising diffusion implicit models”, 2022. En ligne: <https://arxiv.org/abs/2010.02502>
- [30] S. Tan et B. Li, “Stacked convolutional auto-encoders for steganalysis of digital images”, in *2014 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA)*. IEEE, 2014, pp. 1–4.
- [31] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, et G. Lample, “Llama: Open and efficient foundation language models”, 2023. En ligne: <https://arxiv.org/abs/2302.13971>
- [32] R. van Schyndel, A. Tirkel, et C. Osborne, “A digital watermark”, in *Proceedings of 1st International Conference on Image Processing*, vol. 2, 1994, pp. 86–90 vol.2.
- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, et I. Polosukhin, “Attention is all you need”, 2023. En ligne: <https://arxiv.org/abs/1706.03762>
- [34] A. Westfeld et A. Pfitzmann, “Attacks on steganographic systems”, in *Information Hiding: Third International Workshop, IH’99*. Springer, 1999, pp. 61–76.
- [35] G. Xu, “Structural design of convolutional neural networks for steganalysis”, *IEEE Signal Processing Letters*, vol. 23, no. 5, pp. 708–712, 2016.
- [36] Z. Yang, K. Zeng, K. Chen, H. Fang, W. Zhang, et N. Yu, “Gaussian shading: Provable performance-lossless image watermarking for diffusion models”, 2024. En ligne: <https://arxiv.org/abs/2404.04956>
- [37] —, “Gaussian Shading: Provable Performance-Lossless Image Watermarking for Diffusion Models”, Mai 2024, arXiv:2404.04956 [cs]. En ligne: <http://arxiv.org/abs/2404.04956>
- [38] J. Ye, J. Ni, et Y. Yi, “Deep learning hierarchical representations for image steganalysis”, *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 11, pp. 2545–2557, 2017.
- [39] M. Yedroudj, F. Comby, et M. Chaumont, “Yedroudj-net: An efficient cnn for spatial steganalysis”, *IEEE ICASSP*, pp. 2092–2096, 2018.
- [40] J. Yu, X. Zhang, Y. Xu, et J. Zhang, “Cross: Diffusion model makes controllable, robust and secure image steganography”, 2023. En ligne: <https://arxiv.org/abs/2305.16936>
- [41] J. Zhu, Z. Chen, J. Liu, L. Yang, Y. Zhou, W. Luo, et X. Xie, “Rethinking Security of Diffusion-based Generative Steganography”, Fév. 2026, arXiv:2602.10219 [cs]. En ligne: <http://arxiv.org/abs/2602.10219>

-
- [42] Y. Zhu *et al.*, “Image steganalysis based on neural architecture search”, *Signal Processing: Image Communication*, vol. 89, p. 115980, 2020.

LISTE DES FIGURES

1.1	Architecture simplifiée d'un réseau de neurones multicouches (MLP).	5
1.2	Modèle mathématique d'un neurone artificiel : entrées pondérées, biais, sommation et activation.	6
1.3	Comparaison entre la linéarité et la non-linéarité (Fonction Sigmoidale).	7
1.4	Représentation simplifiée de la descente de gradient.	13
1.5	Schéma de la boucle d'apprentissage d'un réseau de neurones.	15
1.6	Architecture d'un réseau de neurones convolutionnel (CNN) appliqué à la reconnaissance de chiffres manuscrits [21].	17
1.7	Architecture générale du modèle Transformer (Encodeur-Décodeur) [33].	18
1.8	Représentation de la loi normale centrée en $\mu = 0$ avec un écart-type σ	20
1.9	Représentation d'une chaîne de Markov appliquée au processus de diffusion. Les flèches pleines décrivent le processus direct (ajout de bruit q), et les flèches pointillées rouges décrivent le processus inverse (génération/débruitage p_θ).	21
1.10	Représentation visuelle du processus direct (<i>Forward Process</i>) et du processus inverse (<i>Generative Backward Process</i>) [13].	24
1.11	Processus d'inférence de Stable Diffusion	25
1.12	Image synthétique générée par le modèle <i>Stable Diffusion v1.5</i> après $T = 20$ étapes d'échantillonnage, basée sur le prompt : « <i>a cat sitting on a chair</i> ».	26
1.13	Architecture globale d'un modèle de diffusion dans l'espace latent (<i>Latent Diffusion Model</i>) [26].	27
2.1	Illustration des concepts de Cover et Stego [27]	31
2.2	Taxonomie de la stéganalyse d'images [8]	36
2.3	Le changement de paradigme : de la stéganalyse classique à l'apprentissage profond [8]	38
2.4	Architecture globale du réseau QianNet [8]	40
2.5	Architecture globale du réseau XuNet [8]	40
2.6	Architecture globale du réseau YeNet [8]	40

2.7	Représentation schématique des différentes distributions (P_c, P_s, Q_c, Q_s) dans l'espace des images et déplacement d'une image de P_c vers P_s	42
2.8	Représentation simplifiée du FLD : Les statistiques des images se séparent en deux groupes distincts de part et d'autre du seuil de décision.	45
2.9	Deux approches d'insertion de messages dans les frameworks DM-GIS : (a) dissimulation dans le bruit initial x_T , et (b) dissimulation au sein d'un état latent intermédiaire x_t	46
2.10	Résultats de détection (%) et évaluation globale du stéganalysateur NS-DSer par rapport à l'état de l'art à travers les différents scénarios.	49
3.1	Diagramme de l'approche : de l'encodage de l'image à l'enrichissement du classifieur.	52
3.2	Pipeline de génération du jeu de données pour les classes Cover et Stego.	54
3.3	Mécanisme d'extraction au cours de l'inversion : interception des activations internes du U-Net par Hooks.	55
3.4	Architecture globale de la première approche	57
4.1	Répartition de l'architecture logicielle et matérielle entre les environnements Cloud.	62
4.2	Aperçu de l'interface de notre environnement de développement sur Google Colab.	62
4.3	Aperçu de l'environnement Kaggle en configuration multi-GPU (Accelerator : $2 \times T4$) pour l'extraction de la baseline.	63
4.4	Extrait de code illustrant le chargement et la répartition matérielle des poids de Stable Diffusion 2.1.	64
4.5	Schématisation de l'inversion et de l'extraction.	65
4.6	Aperçu des paires d'images générées.	66
4.7	Comparaison directe d'une paire générée (Cover vs Stego).	67
4.8	Organisation structurelle des données sur le disque, séparant rigoureusement les classes pour prévenir toute fuite d'information (<i>data leakage</i>) lors de l'entraînement.	67
4.9	Résultats de l'évaluation de la baseline NS-DSer sur le jeu de données MC.	69
4.10	Résultats de l'évaluation de la baseline NS-DSer sur le jeu de données <i>mas-GRDH</i>	70
4.11	Performances de classification de notre architecture CNN (Première Extension) sur les résidus de bruit latent.	71
4.12	Résultats RPCM.	72
A.1	Architecture détaillée de l'approche proposée. De la génération des données à la classification	78
B.1	Visualisation globale du rapport d'analyse statistique effectué sur le vecteur complet (132 dimensions)	81

LISTE DES TABLEAUX

1.1	Comparaison des principales fonctions d'activation	11
2.1	Évolution historique et analyse technique des modèles CNN de stéganalyse.	39
2.2	Précision de détection globale (%) des stéganalyseurs sous le Scénario #4. .	48
4.1	Liste des liens externes vers les codes sources, notebooks de génération et environnements d'exécution	75
B.1	Comparaison brute des performances de détection sur la base mas-GRDH (2 000 échantillons)	80

LISTE DES ABRÉVIATIONS, SIGLES ET ACRONYMES

ACP *Analyse en Composantes Principales.*

API *Application Programming Interface.*

BERT *Bidirectional Encoder Representations from Transformers.*

CLIP *Contrastive Language-Image Pretraining.*

CNN *Convolutional Neural Network.*

COCO *Common Objects in Context.*

CPU *Central Processing Unit.*

CUDA *Compute Unified Device Architecture.*

DCT *Discrete Cosine Transform.*

DDPM *Denoising Diffusion Probabilistic Models.*

DL *Deep Learning.*

DM-GIS *Diffusion Model-based Generative Image Steganography.*

FLD *Fisher Linear Discriminant.*

GAN *Generative Adversarial Network.*

GPT *Generative Pre-trained Transformer.*

GPU *Graphics Processing Unit.*

IA *Intelligence Artificielle.*

IQR *Interquartile Range.*

LDM *Latent Diffusion Models.*

LLaMA *Large Language Model Meta AI.*

MC *Message Concealment.*

MLP *Multi-Layer Perceptron.*

MSE *Mean Squared Error.*

NS-DSer *Noise Space Diffusion Steganalyzer.*

PCA *Principal Component Analysis.*

PCRM *PCA Cover Residual Model.*

PF-ODE *Probability Flow Ordinary Differential Equation.*

RAM *Random Access Memory.*

ReLU *Rectified Linear Unit.*

RNN *Recurrent Neural Network.*

RPCM *Random Projection Cover Model.*

SD *Stable Diffusion.*

SGD *Stochastic Gradient Descent.*

SVM *Support Vector Machine.*

VAE *Variational Autoencoder.*

VRAM *Video Random Access Memory.*